CPE334
Software Engineering

Detailed Midterm Study Notes

Lectures 1–7 + Lecture 10 Exam Blueprint

จากความต้องการที่คลุมเครือ สู่ซอฟต์แวร์ที่ตรวจสอบได้

วิธีใช้หนังสือ อ่าน concept เพื่อเข้าใจเหตุผล แล้วฝึกสร้าง artifact จาก scenario โดยระบุ assumption, decision และ evidence ทุกครั้ง

KMUTT • Computer Engineering

ฉบับอ่านสอบกลางภาค | ปรับตัวอย่างและชุดฝึกสอบ ตุลาคม 2026

สารบัญ

1 Introduction: จาก AI ที่เขียนโค้ด สู่ซอฟต์แวร์ที่ตรวจสอบได้ 1

2 Requirement Engineering: จากคำขอสู่ requirement ที่ทดสอบได้ 9

3 Specifications, SDD, Testing และ TDD 17

4 UML และการจัดทำ SDS / STS 23

5 Security in Software Engineering 30

6 Project Management: เริ่มโครงการ วางแผน และประมาณการ 36

7 Project Execution: backlog, Scrum, delivery และ Git workflow 43

8 Applied Example: POS clarification และ Kaching SDS 50

9 แผนภาพอ่านเร็ว: จากโจทย์สู่ชิ้นงาน 57

10 Integrated Worked Casebook: จากโจทย์สู่หลักฐาน 65

11 Lecture 10: Midterm Exam Blueprint 75

12 วิธีทำข้อสอบ: ตอบให้เห็นเหตุผลและหลักฐาน 77

13 Git Exam Reference: คิดเป็น transaction 81

14 Git Worked Transactions: ExamScore repository 83

15 ทบทวนก่อนสอบ: แผนที่ 4 หน้า 91

A สูตรและแนวเขียนคำตอบแบบเร็ว 94

แนวทางและขอบเขต

เล่มนี้เรียบเรียงจาก Lecture 1–7 เป็นแหล่งเนื้อหาหลัก และใช้ Lecture 10 เป็นแหล่งรูปแบบโจทย์สอบ/การฝึกเท่านั้น ตัวอย่าง Kaching SDS, Lecture 3 ChatGPT Sessions และ UML Example Exam Problem ใช้เป็น applied examples โดยไม่แทนนิยามจาก lecture. คำอธิบายที่ช่วยเชื่อมแนวคิดจะติดป้าย *Explanation / Intuition*; ข้อสรุปเกี่ยวกับรูปแบบสอบติดป้าย *Exam Blueprint* หรือ *Exam Insight*.

Source labels

Lecture Content = แนวคิด/กฎจาก lecture; **Exam Blueprint** = โครงข้อสอบที่ Lecture 10 ระบุ; **Applied Example** = ตัวอย่างจากไฟล์เสริม; **Explanation / Intuition** = คำอธิบายประกอบ; **Exam Insight** = วิธีคิดเพื่อทำข้อสอบซึ่ง derive จาก source; **Extra Reference** = ข้อมูลเพิ่มภายนอก (ฉบับนี้ไม่ใช้เป็น authority).

ขึ้นงาน | เส้นทางฝึกสอบหลังอ่านบทหลัก

ส่วน Exam blueprint และ Answer workflow สรุปวิธีทำโจทย์จากชีทและวิธีรับมือโจทย์เปลี่ยนเงื่อนไข หลังทบทวน 12 ชุด ให้ทำไฟล์ข้อสอบจำลองที่แยกจากเล่มนี้ก่อนเปิดเฉลย สัดส่วน 25/25/50 อิง Lecture 10 ส่วนข้อสอบและเกณฑ์ตรวจที่เขียนเพิ่มใช้เพื่อฝึก

ศัพท์ช่วยอ่านก่อนสอบ

เล่มนี้คงศัพท์อังกฤษที่อาจารย์ใช้ เพื่อให้เชื่อมกับชีทและโจทย์ได้ ตารางนี้แปลความหมายที่ต้องใช้คิดคำตอบ เมื่อเจอศัพท์ ให้ถามต่อว่ามันเปลี่ยนสิ่งที่ต้องเขียน วาด หรือทดสอบอย่างไร

ศัพท์	อ่านให้เข้าใจและใช้ตอบอย่างไร
Artifact / Contract	ชิ้นงานที่ตรวจได้ เช่น ตาราง ภาพ สเปก หรือ Test Case; Contract คือข้อตกลงพฤติกรรมและขอบเขตที่แต่ละส่วนต้องรักษา
Scope / Constraint	ขอบเขตว่าทำอะไรและไม่ทำอะไร; ข้อจำกัดบังคับทางเลือก เช่น เวลา Stack หรือแหล่งข้อมูล
Fact / Assumption	Fact คือข้อมูลที่โจทย์ยืนยัน Assumption คือสิ่งที่สมมติเพื่อเดินต่อ ต้องเขียนให้เห็นและใช้สอดคล้องกัน
Baseline / Clarification	Baseline คือข้อตกลงตั้งต้นที่ใช้เทียบการเปลี่ยนแปลง Clarification คือการถามเพื่อปิดความกำกวมก่อนยึดเป็นกฎ
Derive / Rationale	Derive คือสร้างคำตอบจากข้อมูลและกฎ Rationale คือเหตุผลว่าทำไมเลือกคำตอบนั้น
Observable / Verifiable	มีผลที่สังเกตหรือตรวจได้ เช่น จำนวน Record ไม่ใช่เพียงคำว่าเร็ว ดี หรือทำงานถูก
Traceability / Evidence	เชื่อมที่มาของ Requirement ไป Design, Task, Code, Test และหลักฐานผลจริง
Boundary / Guard	Boundary คือขอบเขตระบบหรือจุดเปลี่ยนความเชื่อถือ Guard คือเงื่อนไขก่อนเลือกเส้นทางหรือเปลี่ยนสถานะ
Lifeline / Partition	Lifeline คือผู้ร่วมใน Sequence ที่อ่านตามเวลา Partition หรือ Lane แบ่งผู้รับผิดชอบ Action ใน Activity
Side effect / Invariant	ผลที่ต้องเกิดตาม Action เช่นส่ง Email; กฎที่ต้องรักษาเสมอ เช่น success หมายถึงข้อมูลถูกบันทึกจริง
Transaction / Rollback	กลุ่มการเปลี่ยนข้อมูลที่ต้องสำเร็จร่วมกัน หากล้มต้องย้อนกลับไม่ทิ้งข้อมูลครึ่งหนึ่ง
Idempotent / Reconcile	ทำซ้ำแล้วไม่ให้ผลสำคัญซ้ำ เช่นไม่ลด Stock สองครั้ง; ตรวจเทียบผลชำระเดิมเมื่อยังไม่ทราบผลแน่ชัด
Dependency / Buffer	งานที่ต้องเสร็จก่อนเริ่มงานถัดไป; เวลาสำรองเพื่อรับความเสี่ยง ไม่ซ่อนเป็นระยะทำงานปกติ
Effort / Duration	ปริมาณแรงงาน เช่นคน-ชั่วโมง ต่างจากเวลาที่งานกินไปตามปฏิทินหรือวันทำงาน
Milestone / Increment	จุดเหตุการณ์สำคัญที่ไม่กินเวลางาน; ส่วนงานที่ส่งมอบเพิ่มและใช้ตรวจผลได้
AC / DoD	AC คือเงื่อนไขรับ Feature/Story เฉพาะข้อ DoD คือเกณฑ์คุณภาพร่วมก่อนทีมเรียกงานว่าเสร็จ

รูปแบบตอบที่ใช้ได้หลายหัวข้อ: ข้อมูลโจทย์ → กฎที่เกี่ยวข้อง → คำตอบ/ภาพ → เหตุผลและสมมติฐาน → วิธีตรวจพร้อมผลที่คาดหวัง

เส้นเรื่องของวิชาและวิธีอ่านเล่ม

วิชานี้เดินจากการเข้าใจปัญหาไปสู่การส่งมอบงานที่ตรวจสอบได้ แต่ละ lecture เพิ่มคำตอบอีกหนึ่งขั้น ไม่ได้เริ่มระบบใหม่แยกจากกัน

Lecture	คำถามที่บทตอบ	สิ่งที่ต้องสร้างหรือใช้ตัดสินใจ
1	ทำไมการเขียน code ยังไม่เท่ากับส่งมอบ software?	เลือก process, แบ่งงาน, review และเก็บ evidence
2	ผู้ใช้ต้องการแก้ปัญหาอะไร?	แยก stakeholder, FR/BR/NFR, acceptance และข้อมูลที่ยังต้องถาม
3	ทีมจะรู้ตรงกันได้อย่างไรว่าต้องสร้างและตรวจอะไร?	เชื่อม requirement → design contract → test evidence
4	จะทำให้ behavior, data และหน้าจอเห็นภาพเดียวกันได้อย่างไร?	เลือก UML/UI ให้ตรงคำถาม แล้วตรวจ semantics
5	จุดใดเปลี่ยนจาก trusted เป็น untrusted และใครทำอะไรได้?	กำหนด identity, permission, boundary, control และ negative test
6	ขอบเขตและเวลาโครงการสมเหตุผลหรือไม่?	อนุมัติ value, แตก WBS, ระบุ owner, dependency, buffer และ risk
7	ทีมจะส่งงานเป็นส่วนย่อยและปรับจาก feedback อย่างไร?	ดูแล backlog, วาง Sprint, ติดตามงาน และเปลี่ยน Git refs อย่างตั้งใจ

รูปแบบการเล่าในแต่ละหัวข้อ

แต่ละ topic ใช้ลำดับ **โจทย์** → **หลักคิด** → **การตัดสินใจ** → **ชิ้นงาน** → **วิธีตรวจ** → **ข้อจำกัด**. อ่านตัวอย่างพร้อมเหตุผลก่อนจำรูปแบบคำตอบ; หากมีช่องว่างใน source ให้ทำเครื่องหมายเป็น *assumption* หรือ *unresolved* แทนการเติมเป็นข้อเท็จจริง.

ตัวอย่างพร้อมวิธีคิด | ตัวอย่างการเขียน: เปลี่ยนคำว่า “เร็ว” ให้ตรวจสอบได้

ข้อความต้นทาง: “The registration process should be quick.”
ทำไมยังไม่พอ: คำว่า *quick* ไม่มีหน่วยวัด ไม่มีช่วงเวลาที่เริ่ม/จบ และไม่ระบุภาระงานที่ใช้ทดสอบ.
ข้อมูลที่ต้องตกลง: เวลาเริ่มและจบของการวัด, สัดส่วนคำขอที่ต้องผ่านเกณฑ์, ปริมาณผู้ใช้พร้อมกัน และ threshold ที่ stakeholder ยอมรับ.
รูปแบบ requirement หลังตกลงค่า: “For the agreed workload, 95% of registration searches shall complete within [agreed threshold] seconds.” วงเล็บเหลี่ยมคือค่าที่ยังต้องตกลง ไม่ใช่ค่าที่ผู้เขียนเดา.
หลักฐาน: ระบุ load-test setup, วัด percentile เดียวกัน และเทียบผลกับ threshold ที่อนุมัติ.

สัญลักษณ์แหล่งข้อมูล

Lecture Content คือ สิ่งที่สอนใน Lecture 1–7; **Exam Blueprint** คือ โครงคะแนน/ชนิดชิ้นงานจาก Lecture 10; **Professor Sample** คือ รูปแบบโจทย์และพื้นที่ตอบจากตัวอย่างอาจารย์; **Applied Example** คือ กรณีที่ใช้ฝึก; **Explanation** คือคำอธิบายประกอบ; **Assumption** คือรายละเอียดที่โจทย์ไม่ได้ยืนยัน.

ดัชนีต้นฉบับและการแก้ความไม่ครบถ้วน

Source scope. Lecture 1–7 รวม 636 หน้า; Lecture 10 อีก 141 หน้า; supplementary examples อีก 59 หน้า. จำนวนสไลด์ไม่เท่ากับจำนวนหน้าหนังสือ เพราะมี cover/divider, repeated examples และภาพที่ขยายหลายหน้า. ฉบับนี้จัดตาม concept/artifact และเก็บตัวอย่างหลายระบบแยกกันเพื่อให้ตรวจย้อนกลับได้.

Source	Pages	ตำแหน่งและหัวข้อในหนังสือ
Lecture 1	46	บท 1: agents/product/SDLC/models/responsible use/DoD/full-stack lab; Git collaboration หน้า 31–46 ครบตั้งแต่ repo setup, branch, PR/review, staging ถึง release
Lecture 2	43	บท 2: complexity/elicitation/requirements/AC/proposal/validation/trace/API foundations
Lecture 3	59	บท 3: contracts/SRS-SDS-STS/SDD pipeline/feature design/testing/TDD; Kaching case
Lecture 4	161	บท 4: UML/UI/SDS/STS/engineering contracts; Visual Worked Examples; KU case
Lecture 5	40	บท 5: auth/session/authorization/RBAC-ABAC-ACL/vulnerabilities/backend tests
Lecture 6	116	บท 6: initiation/goals/charter/feasibility/stakeholders/org/WBS-RACI-Gantt/CPM-PERT/risk
Lecture 7	171	บท 7: execution/docs/Agile/Kanban/BDD-TDD/MVP/Scrum/stories/points/metrics/end-of-sprint
Lecture 10	141	Exam Blueprint/Playbooks; WorkshopHub/DormFix/ShuttleNow; Git context; repeated modeling/SDD prep
Sessions	28	POS clarification; revised FR/BR/NFR69-row trace; decisions vs assumptions
Kaching SDS	22	System-level SDS18sections; architecture/deployment/data/failure/recovery/quality targets
UML Example	9	KU scenario9steps, Activity, actual List/Form prototype, Class scope/assumption caveats

ชิ้นงาน | ช่องว่างเดิมที่แก้ในฉบับนี้

- เพิ่ม explanations/decisions/anti-patterns และ mini problems พร้อมเหตุผลท้ายทุกบทหลัก.
- เพิ่ม selected vector figures และคำอธิบายวิธีอ่าน; ใช้เฉพาะภาพที่ช่วยอธิบายแนวคิด/โจทย์ แทนการแปะชุด source slides ซ้ำทั้งชุด.
- แก้ Service Ticket ให้แยกผู้ปิดสองสถานะ; แยก Grades Submission จาก Course Registration; แยก generic POS refund จาก Kaching exclusions.
- เพิ่ม KU Sequence/Activity/UI จริงพร้อม field/action contract; อธิบาย multiplicity ที่ sample ขัดกับ blank Draft.
- เติม DormFix ทุก task/owner/duration/date/buffer; CPM forward/backward; PERT variance derivation.
- เพิ่ม Git collaboration flow จาก Issue ถึง release และ Git transactions ก่อน/หลัง พร้อม nodes/parents/refs/HEAD/local-remote semantics; Final Review 12 ชุดแยกหน้า.

ตรวจย้อนกลับ: ใน source bundle มี source_page_inventory.csv บันทึกทุกหน้าและ chapter route; content_inventory.md รวม topic/ coverage decisions; source_trace.md ระบุ แหล่ง/ บท; figure_manifest.md แยก crop/ redraw; qa_checklist.md รายงานสิ่งที่ตรวจจริง. Inventory route เป็นดัชนี ไม่ใช่หลักฐานว่า software ถูก implement/ test แล้ว. **ยังต้องใช้ข้อมูลโจทย์จริง:** DormFix RACI ไม่มี involvement meeting notes ราย task; performance thresholds ของ vague request และ provider protocols ที่ draft ไม่ระบุยังเป็น gap. ไม่แต่งค่าหรือ role assignments เพื่อให้ตารางเต็ม. Worked tests เป็น proposed expected checks เว้นมี execution evidence ระบุชัด.

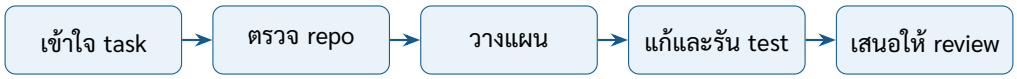
บทที่ 1

Introduction: จาก AI ที่เขียนโค้ด สู่ซอฟต์แวร์ที่ตรวจสอบได้

แกนบท: AI ลดต้นทุนการพิมพ์โค้ด แต่ไม่ได้ตัดสินแทนทีมว่าปัญหาคืออะไร สร้างของใด ปลอดภัยพอหรือไม่ และมีหลักฐานอะไรว่าใช้งานได้

1.1 AI coding agent ทำอะไรได้ และใครรับผิดชอบ

Chatbot ตอบจากข้อมูลนำเข้าเป็นหลัก ส่วน **AI agent** ทำงานไปสู่เป้าหมายโดยสังเกตสภาพแวดล้อม วางแผน ใช้เครื่องมือ ดูผลลัพธ์ ปรับการกระทำ และอาจเก็บบริบทข้ามการโต้ตอบได้ **AI coding agent** นำความสามารถนี้มาใช้กับ repository, ไฟล์, shell, test และ workflow พัฒนาโปรแกรม



อ่านภาพ: agent เคลื่อนจากบริบทไปสู่การเปลี่ยนแปลงที่ตรวจได้ การเห็นคำว่า “tests passed” อย่างเดียวไม่ยืนยันว่า test ครอบคลุม requirement หรือไม่มี regression; ผู้รับผิดชอบต้องดู diff, คำสั่งที่รัน, ผลทดสอบ และข้อจำกัดที่ยังเหลือ *Source: Lecture 1, slides 3, 6, 15–16*

Engineering decision

ใช้ AI ช่วยเร่ง implementation แต่ให้วิศวกรรมซอฟต์แวร์กำหนดขอบเขต สเปก การตรวจสอบ และการเปลี่ยนแปลงที่ยอมรับได้ มนุษย์ยัง accountable ต่อการตัดสินใจในทุก phase.

1.2 เขียน code ไม่เท่ากับสร้าง software product

มุมเปรียบเทียบ	เขียน code	วิศวกรรม product
เป้าหมาย	ทำตาม instruction	แก้ user problem ที่มีคุณค่า
หลักฐาน	ผ่านตัวอย่างเฉพาะหน้า	ผ่าน acceptance criteria ที่ตกลง
คุณภาพระยะยาว	อาจละเลย maintainer ในอนาคต	testable, secure, maintainable
หลังส่งมอบ	จบเมื่อเขียนเสร็จ	ต้อง operate, observe และ evolve

ตัวอย่าง: ฟังก์ชันบันทึกไฟล์อาจทำงานกับไฟล์ขนาดเล็กบนเครื่องนักพัฒนา แต่ยังไม่ใช่ product ที่พร้อมใช้ หากไม่มีสิทธิ์เข้าถึงที่ถูกต้อง ไม่มี error path ไม่มีหลักฐานว่าเก็บข้อมูลสำเร็จ หรือไม่มีวิธีเปลี่ยนเมื่อ schema โตขึ้น **คุณภาพจึงครอบคลุม behavior, ความปลอดภัย, การดูแล และการปฏิบัติการ** ไม่ใช่เพียง output ณ ครั้งเดียว

1.3 Engineering process และ SDLC

Lecture วางกระบวนการเป็นลำดับการตัดสินใจ: **Discover** ปัญหา/ผู้ใช้/คุณค่า → **Decide** scope/priority/risk → **Design** structure/interface → **Build** code/integration → **Verify** test/review → **Release** deploy/operate → **Learn** feedback/change. ลำดับนี้ช่วยไม่ให้ implementation กลายเป็นจุดเริ่มต้นโดยอัตโนมัติ

ชิ้นงาน | SDLC เป็นวงจรงาน ไม่ใช่ checklist ที่ห้ามย้อนกลับ

Phase	คำถามที่ต้องตอบ	ตัวอย่าง output/evidence
Planning	ทำไมควรทำ และอะไรเป็นข้อจำกัด	goal, feasibility, risk, scope
Requirements	ใครต้องการ behavior/quality ใด	stakeholder needs, FR/NFR/constraints
Specification	ตกลง behavior, interface, data, error อย่างไร	SRS, acceptance criteria
Design	แบ่งระบบและกำหนด boundary อย่างไร	architecture, data model, API/UI contract
Implementation	เปลี่ยน design เป็น code อย่างไร	code, review, unit tests
Testing	หลักฐานใดลด uncertainty	test result, defect record
Deployment	release และ rollback อย่างไร	deployed version, configuration
Maintenance	เรียนรู้และเปลี่ยน product อย่างไร	monitoring, fixes, updated docs

Phases ไม่จำเป็นต้องเกิดเป็นเส้นตรง: requirement อาจเปลี่ยนหลังผู้ใช้ทดลอง; test อาจเผย defect ซึ่งต้องย้อนแก้ design; deployment สร้างข้อมูลเพื่อปรับ scope. Model ต่างกันที่การจัดลำดับและจังหวะ feedback กิจกรรมหลักยังต้องเกิดอยู่

1.4 Specification คือสัญญาทางวิศวกรรม

Specification ระบุ behavior, interface, rules, data, errors และ quality constraints ให้ review และตรวจได้ ส่วน **acceptance criterion** คือเงื่อนไขที่ต้องเป็นจริงเพื่อยอมรับ feature/story. ทั้งคู่ร่วมกันสร้าง baseline ว่าจะสร้างอะไร “เสร็จ” หมายถึงอะไร และ verify อย่างไร

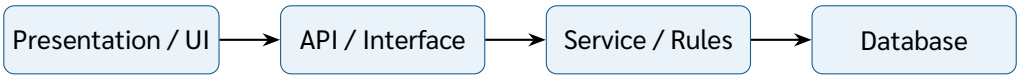
ตัวอย่างพร้อมวิธีคิด | ส่งงาน lab ผ่านระบบ

Requirement: นักศึกษาส่งงาน lab ออนไลน์ได้
Specification: ระบบรับไฟล์ .ipynb ขนาดไม่เกิน 25 MB ก่อน deadline, บันทึก timestamp และแสดง confirmation
Acceptance: Given ไฟล์ถูกต้องและยังไม่เลยกำหนด, When นักศึกษากด Submit, Then ระบบบันทึกไฟล์และแสดง confirmation ID.

เหตุผล: agent ที่ได้รับ specification สามารถวางแผน input, output, boundary case และ test ได้ ผู้ใช้จึง review ผลเทียบกับข้อตกลง แทนที่จะถามว่าโค้ดดูเหมือนจะทำงานหรือไม่

1.5 Design และ architecture ก่อน code ขยายความผิดพลาด

ตัวอย่างชั้นระบบใน lecture แสดง **UI** → **API** → **Service** → **Database**. Design artifacts ได้แก่ architecture diagram, data model, API/interface contracts, UI flows และ security boundaries. จุดประสงค์คือแบ่งความรับผิดชอบและระบุสิ่งที่ module แลกเปลี่ยนกันก่อนที่การเปลี่ยนแปลงจะกระทบหลายส่วน



การมี layer ไม่ได้แปลว่าทุกระบบต้องมี service แยกแบบเดียวกัน; architecture เป็นการตัดสินใจให้เข้ากับ requirement, constraint และขนาดของระบบ

Source: Lecture 1, slide 11; Lecture 3, slides 27–28

1.6 Process models: วิธีจัดกิจกรรมและส่งมอบ

แนวทาง	ใช้เมื่อ/จังหวะงาน	ความเสี่ยงหรือข้อแลกเปลี่ยน
Waterfall / predictive	วางแผนและควบคุมการเปลี่ยนแปลงก่อนดำเนินตาม phase	feedback จากของจริงมาช้า; การแก้ความเข้าใจผิดปลายทางแพง
Incremental	ส่งมอบส่วนเพิ่มที่ใช้งานได้ทีละ slice	ต้องวาง integration และจัดลำดับ increment ให้ value มาถึงก่อน
Iterative	ปรับปรุง solution เดิมจาก feedback ซ้ำ	ถ้าไม่มี goal/เกณฑ์ อาจวนปรับโดยไม่รู้ว่าเสร็จเมื่อไร
Agile / adaptive	คาดว่า change จะเกิด; เน้น collaboration, working software, feedback	ต้องมีส่วนร่วมสม่ำเสมอและตัดสินใจไว
Scrum	ใช้ sprint goal และ cadence/event เพื่อวางแผน/ตรวจ/ปรับ	ต้องรักษา accountability ของ role และนิยาม Done
Kanban	งานไหลเข้าต่อเนื่อง; สนใจ flow และการมองเห็นงาน	ต้องควบคุมงานค้างและข้อจำกัดกำลังคน

Incremental vs iterative: incremental เพิ่มความสามารถหรือส่วนของ product; iterative กลับมาปรับสิ่งเดิมให้ดีขึ้น ทั้งสองอย่างเกิดร่วมกันได้. **Predictive vs adaptive** อธิบายวิธีวางแผน/ตอบสนอง change; ไม่ใช่การบอกว่าแบบหนึ่งมี testing หรือ requirements แต่อีกแบบไม่มี

1.7 Definition of Done และการใช้ AI อย่างรับผิดชอบ

Definition of Done (DoD) เป็น checklist คุณภาพที่ตกลงก่อนนับงานว่าเสร็จ ไม่ใช่ความรู้สึกของผู้ทำ ตัวอย่างหลักฐานอาจรวม code review, tests, integration, documentation และการ deploy ตามขอบเขตที่ทีมกำหนด. DoD ใช้เป็น completion gate ระดับทีม; acceptance criteria ระบุเงื่อนไขของ feature/story เฉพาะรายการ

ข้อผิดพลาดที่พบบ่อย | ยอมรับผล agent โดยไม่ตรวจหลักฐาน

อาจได้ wrong target, false confidence, hidden breakage, security gap, dependency risk หรือ documentation drift. คำกล่าวของ AI ว่า “ทำครบ” ไม่ใช่ verification evidence. ตรวจ diff และ test ต่อ requirement; บันทึกสิ่งที่ยังไม่ทดสอบ.

1.8 AI delegation pattern ที่ review ได้

1. ให้ context: repository, ไฟล์, constraints, coding standard.
2. ให้ specification: input/output, edge cases, acceptance criteria.
3. ขอ plan ก่อนแก้: ระบุไฟล์ที่จะเปลี่ยนและ tests ที่จะเพิ่ม/รัน.
4. จำกัด increment: หนึ่ง feature, bug หรือ refactor ต่อรอบ.
5. review evidence: diff, commands, test result, documentation และ residual risks.

สรุปบท 1

- Agent ทำ plan/use tools/observe feedback ได้ แต่มนุษย์ยังตัดสิน goal, scope, acceptance และความเสี่ยง.
- Software product ต้องผ่านความต้องการและ quality ที่ตกลง พร้อมดูแลได้หลัง release.
- SDLC activities เป็น feedback system; process model จัดกิจกรรมต่างกันแต่ไม่ลบกิจกรรมออก.
- Specification และ acceptance criteria ช่วยให้ทั้งคนและ agent ทำงานจาก baseline เดียวกัน.

1.9 อ่าน SDLC เป็นชุดการตัดสินใจและ artifact

Lecture Content | Core-SE | L1 หน้า 3–17. การใช้ agent เป็นเพียงความสามารถในการลงมือกับ repository. Software Engineering ต้องเริ่มจากความต้องการและคุณค่าของผู้ใช้ เพราะโปรแกรมที่ทำงานตามคำสั่งผิดก็ยังผ่านตัวอย่าง local ได้. บทบาทมนุษย์จึงรวมการเลือกเป้าหมาย ขอบเขต ความเสี่ยงที่ยอมรับ เกณฑ์ acceptance และการรับผิดชอบผลส่งมอบ

Phase	Input / decision	Output และ quality gate
Planning	ปัญหา ผู้ใช้ งบ เวลา; คุ้มทำหรือไม่	goal/ scope/ risk; stakeholder เห็น คุณค่า และ feasibility
Requirements	คำขอและข้อจำกัด; ต้องทำอะไร	FR/ NFR/ constraints พร้อม source; clear/ testable/traceable
Specification	requirement ที่ตกลงแล้ว; behavior ที่แน่นอน	input/output/error/rule/AC; review ความหมายตรงกัน
Design	spec และ constraints; แบ่งส่วนอย่างไร	architecture/ data/ API/ UI/ security boundary; trade-off มีเหตุผล
Implementation	design/ contract; increment ใด เล็กพอ	code/tests/docs บน branch; diff ตรง scope
Testing	AC/STS และ increment; หลักฐานเพียงพอหรือยัง	test execution และ defect record; test ต้องตรวจพฤติกรรมจริง
Deployment	accepted build/config; ปล่อยอย่างไร	version/ environment/ migration/ rollback; smoke check ผ่าน
Maintenance	feedback/logs/new needs; ปรับอะไร	bug fix/refactor/change spec; regression ไม่เสีย

เหตุผล: โมเดลต่างกันจัดกิจกรรมต่างกัน แต่ไม่ได้ทำให้ requirement, design หรือ verification หายไป. ถ้า feedback พบว่าผู้ใช้ต้องการอย่างอื่น ต้องย้อนแก้ spec และ test ก่อนเพิ่ม code ต่อ

1.9.1 Chatbot กับ agent: feedback loop เปลี่ยนคุณภาพอย่างไร

Chatbot ส่งคำตอบจาก input เป็นหลัก; agent อ่านสถานะจริง วางแผน ใช้เครื่องมือ สังเกตผล และปรับการกระทำ. Coding agent จึงควร inspect repository ก่อนอ้างชื่อไฟล์หรือคำสั่งทดสอบ. Memory ช่วยรักษาบริบท แต่ไม่ได้ทำให้ความทรงจำถูกต้องเสมอ: เริ่ม session ใหม่ต้องอ่านเอกสารที่มี version และเทียบกับ code ปัจจุบัน

ตัวอย่างพร้อมวิธีคิด | คำสั่งที่ review ได้: category endpoint

Explanation / Intuition. แทนคำว่า “ทำ category feature” ให้ระบุ requirement ID, endpoint, data source และขอบเขต เช่น “implement FR-CAT-01, GET /api/categories, query PostgreSQL ผ่าน Prisma, ส่ง JSON และเรียงตาม name; เพิ่ม API test; UI ห้าม hard-code category.”

1. อ่าน route/schema/test เดิม: มี endpoint หรือ unique name อยู่แล้วหรือยัง?
2. เขียน plan: ไฟล์ที่จะเปลี่ยน, migration/seed ที่จำเป็น, negative case และคำสั่ง verify.
3. ทำ increment: query+API+UI+test; ไม่เพิ่ม admin editor เมื่อ scope ไม่ให้.
4. ตรวจ evidence: diff, คำสั่งที่รันจริง, exit/result, screenshot และข้อมูลจาก DB.
5. สรุป residual risk: เช่นยังไม่ทดสอบ production load; ห้ามเขียนว่า production-ready จาก unit pass อย่างเดียว.

Quality gate: reviewer เปลี่ยนชื่อ category ใน DB แล้ว UI ต้องแสดงค่าที่เปลี่ยน; วิธีนี้จับ fake implementation ที่ hard-code ได้

1.10 Process models: เลือกจาก uncertainty และ flow

Lecture Content | L1 หน้า 18–24. Predictive/adaptive เป็นมิติการวางแผน; incremental/iterative เป็นมิติการส่งมอบ. Predictive กำหนดและควบคุมการเปลี่ยนแปลงล่วงหน้า; adaptive รับ feedback แล้วปรับ. Incremental เพิ่ม working slice ใหม่; iterative ปรับ slice เดิมให้เหมาะขึ้น. โปรเจกต์เดียวสามารถเป็นทั้ง incremental และ iterative ได้

ชิ้นงาน | ตารางเลือกโมเดลพร้อมเหตุผล

Model	เหมาะเมื่อ	ประโยชน์	ความเสี่ยงที่ต้องจัดการ
Waterfall	requirements stable, งานคล้ายเดิม	phase/milestone ชัด; budget/traceability วางล่วงหน้า	feedback ช้า; change แพง; ควรมี change-control
Incremental	ใช้ core workflow ก่อนได้	value/learning มาเร็ว; ลด big-release risk	integration/architecture ต้องมีวินัย; increment ต้อง coherent
Scrum	discovery ต้องมี goal/ feedback เป็นรอบ	sprint goal, review, adaptation	participation ต้องจริง; Done ต้อง integrated/tested
Kanban	งานเข้าต่อเนื่อง; flow เป็นปัญหา	เห็นงานค้าง; จำกัด WIP; cycle time/throughput	policy ไม่ชัดหรือเปิดหลายงานพร้อมกันทำให้ค้าง

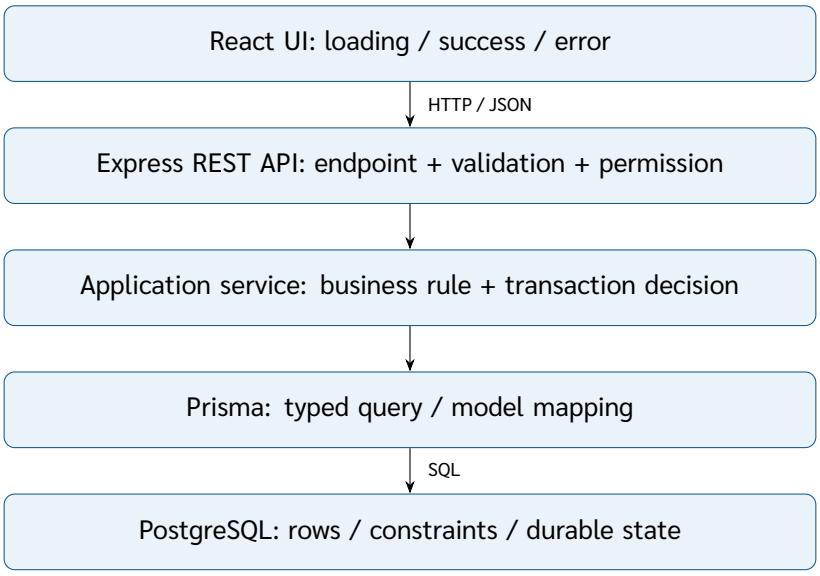
Agile values ตาม lecture: ให้ความสำคัญกับ individuals/interactions, working software, customer collaboration และ responding to change. คำว่า working software ไม่ยกเลิกเอกสารที่จำเป็นต่อการตกลงและตรวจสอบ. Spec ที่สั้นแต่ชัดมีประโยชน์กว่าประวัติ chat ยาวที่ไม่มี baseline

ตัวอย่างพร้อมวิธีคิด | สองมิติในระบบ ticket

Explanation / Intuition. Sprint 1 ให้ requester สร้างและดู ticket; Sprint 2 เพิ่ม staff queue: นี่คือ incremental เพราะมี capability ใหม่. หลัง requester ทดลองแล้วพบว่าปุ่มแนบไฟล์หาไม่เจอ จึงปรับ UI ของ Create Ticket: นี่คือ iterative. ถ้าเพิ่มแต่ database ใน sprint แรกโดยผู้ใช้ทำงานไม่ได้ ยังไม่ใช่ usable vertical slice

1.11 Full-stack และ vertical slice ที่อ่าน traceability ได้

Lecture Content | L1 หน้า 27–30; L2 หน้า 35–43. React แสดง state และส่ง request; Express รับ HTTP ตรวจ input และจัดการ business behavior; Prisma แปลง object query เป็น SQL; PostgreSQL เก็บ persistent records. API คือ contract ไม่ใช่ชื่อ library. ORM เป็น data-access layer ไม่ใช่ database และไม่แทน authorization



อ่านภาพ: ไล่ request ลงและ result ขึ้น; frontend ไม่คุย PostgreSQL โดยตรง. Vertical slice ของ “ดู categories” ต้องครอบ UI, endpoint, query/data และ evidence; แบ่งทีมตาม layer อย่างเดียวอาจส่งมอบ component โดยไม่มี user value

1.12 DoD, operation และความเสี่ยงของ AI

Lecture Content | L1 หน้า 14,17,24,42–46. Acceptance Criteria เป็น feature-specific; Definition of Done เป็น team-level gate ที่ใช้ร่วมทุก increment: AC ผ่าน, code review, tests เขียนและรัน, critical defect/security issue ไม่มี, docs ทัน code, merge/deploy ตาม policy. Release ต้องตรวจ build/config/secrets/migration, monitor และ rollback; maintenance ต้องแก้ code พร้อม test/doc ไม่ให้ documentation drift

ชิ้นงาน | Evidence checklist สำหรับ Issue → PR → release

- Issue มี reason/requirement/AC/dependency; branch ผูก Issue และเปลี่ยนเฉพาะ scope.
- PR มี diff, linked Issue, review result, test commands/results, UI proof และ known limits.
- Staging มี integrated tests; feature pass แยกกันยังไม่รับประกัน integration pass.
- Main เป็น accepted release; secret ไม่อยู่ Git; README มี install/run/test/env/branch policy.

ข้อผิดพลาดที่พบบ่อย | หกความเสี่ยงต้องผูกกับหลักฐาน

Wrong target ตรวจกับ requirement; false confidence ตรวจ test execution; hidden breakage ตรวจ edge/regression; security gap ตรวจสิทธิ์/secret; dependency risk ตรวจ package/command ที่เพิ่ม; documentation drift ตรวจ README/spec กับ code. การที่ agent บอก “works” ไม่มีข้อมูลเพียงพอปิด Issue

1.13 Git collaboration: จาก Issue ถึง release ที่ตรวจสอบได้

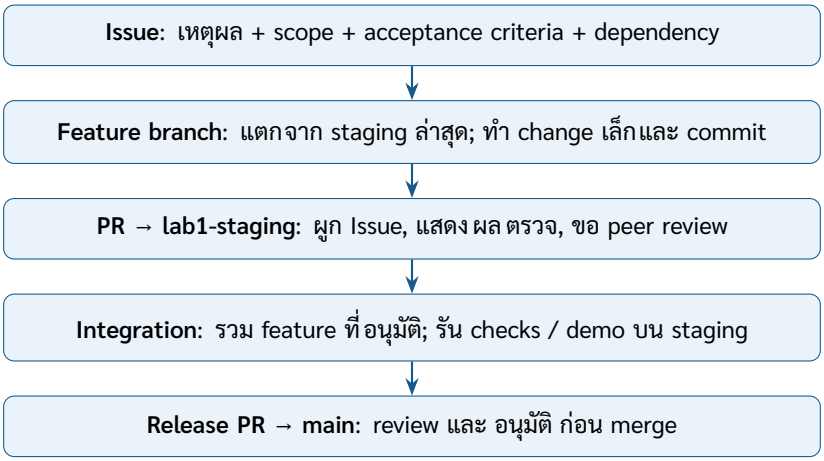
Lecture Content | L1 หน้า 31–46; Exam Insight เชื่อมกับ L10 หน้า 32. Version control เก็บประวัติว่าใครเปลี่ยนอะไร เมื่อไร และด้วยเหตุผลใด ช่วยย้อนกลับจากความผิดพลาด แยกการทดลองออกจากเส้นงานหลัก และให้ทีม review ก่อนรับการเปลี่ยนแปลง. Git เป็น distributed version control system; GitHub เป็นบริการโฮสต์ repository และทำงานร่วมกัน ไม่ใช่ตัว Git เอง.

ชิ้นงาน | คำที่ต้องแยกให้ชัด

คำ	ความหมายเมื่ออ่านสถานะงาน
Working tree	ไฟล์ที่กำลังแก้ในโฟลเดอร์ทำงาน; อาจมีการเปลี่ยนที่ยังไม่เลือกเข้า commit
Staging area / index	snapshot ของไฟล์ที่เลือกด้วย git add; ยังไม่ใช่ commit
Commit	snapshot ที่บันทึกใน local history พร้อม parent และข้อความอธิบายเหตุผล
Branch	ชื่ออ้างอิงที่ขยับตาม commit ของงานหนึ่งสาย; การสร้าง branch ไม่ได้สร้าง commit
Remote / origin	repository ที่แชร์กับทีม; origin มักเป็นชื่อ remote ไม่ใช่ branch
Pull Request (PR)	คำขอให้ตรวจและรวม branch; ใช้แนบ Issue, acceptance criteria, test evidence และ review

1.13.1 เส้นทาง Lab: feature ผ่าน staging ก่อนเข้า main

ในตัวอย่าง Lecture 1, main คือ release ที่ยอมรับแล้ว และ lab1-staging คือจุดรวมงานของ Lab. ทุก feature ส่ง PR เข้า staging; ห้ามส่ง feature branch ตรงเข้า main. หลังรวมแล้วต้องทดสอบ/demonstrate integrated work ก่อนเปิด PR จาก staging ไป main.



1.13.2 ทำงานหนึ่ง Issue โดยเริ่มจาก branch ล่าสุด

1. **เตรียม repository.** ใช้ git clone <url> เมื่อมี repository บน GitHub อยู่แล้ว; ใช้ git init เฉพาะโฟลเดอร์ใหม่ที่ยังไม่ถูก track. ตรวจตำแหน่งและสถานะด้วย git status, git branch, git log --oneline --decorate -5 และ git remote -v.
2. **ป้องกันไฟล์ที่ไม่ควรแชร์.** เพิ่ม .env, password/API key, database credential, node_modules/, build output และไฟล์ชั่วคราวลง .gitignore ก่อน stage. เก็บชื่อ environment variables ที่ต้องใช้ใน .env.example โดยใส่ placeholder เท่านั้น. README ควรบอก project purpose, install/run, environment, test และ branch/PR rules.
3. **เริ่มจาก integration ล่าสุด.** git switch lab1-staging; git pull origin lab1-staging; แล้วสร้าง git switch -c feature/1-project-foundation. การเริ่มจาก staging ล่าสุดช่วยลดการทำงานบน code เก่า.
4. **แก้และตรวจ scope.** ก่อน/หลังแก้ให้ดู git status; stage เฉพาะไฟล์ที่เกี่ยวข้อง เช่น git add README.md src/app.js; บันทึก commit ที่อธิบายเจตนา เช่น git commit -m "Add project foundation". คำสั่ง git add เปลี่ยน index; git commit จึงเพิ่ม node ให้ local history.
5. **เผยแพร่ branch และขอ review.** ครั้งแรกใช้ git push -u origin feature/1-project-foundation. เปิด PR เข้า lab1-staging, link Issue (เช่น Closes #1), วาง acceptance criteria, ระบุข้อที่ผ่านพร้อม test evidence และขอ peer review. ผู้ดูแลอาจ approve หรือขอแก้ไขก่อน merge.
6. **อัปเดตเมื่อ staging เดินหน้า.** ทำ git fetch origin; กลับไป feature branch; รวม origin/lab1-staging เข้า feature ด้วย git merge origin/lab1-staging; แก้ conflict หากมี, ตรวจสถานะและทดสอบซ้ำ, แล้ว git push เพื่ออัปเดต PR. Fetch ปรับข้อมูล remote-tracking ในเครื่อง; มันไม่ merge เข้า branch ทำงานให้เอง.
7. **ปิดงานและ release.** หลัง PR ผ่าน review ให้ merge feature เข้า staging และลบ feature branch ที่เสร็จแล้วตามนโยบายทีม. เมื่อ feature ของ Lab รวมและผ่าน checks/demo บน staging จึงเปิด PR staging → main และ merge หลังอนุมัติ. *Done* หมายถึง accepted by review + satisfies criteria + merged + evidence documented ไม่ใช่เพียง agent บอกว่าเสร็จ.

ข้อผิดพลาดที่พบบ่อย | Fast-forward กับ three-way merge

ถ้า target branch เป็น ancestor ของ feature tip และไม่มีงานใหม่บน target, fast-forward เพียงขยับ target ref ไปยัง commit เดิมที่ตามหลัง; ไม่เพิ่ม merge commit. ถ้าทั้งสอง branch มี commit แยกหลัง common ancestor, three-way merge สร้าง merge commit ที่มี parent สองตัว (เมื่อ merge สำเร็จ). การ merge ไม่ได้ทำให้ source branch ref เคลื่อนตาม target. วาด parent และ ref จากคำสั่งจริงทุกครั้ง.

ตัวอย่างพร้อมวิธีคิด | ตรวจความเข้าใจ: งาน feature ถูก push แล้วแปลว่า release หรือยัง?

ยัง. git push ส่ง commit ไป remote branch เพื่อให้ผู้อื่นตรวจได้; ยังไม่เท่ากับ peer review, merge เข้า lab1-staging, integrated checks, release PR หรือ merge เข้า main. ถ้าโจทย์ถามสถานะ ให้แยก local commit, remote-tracking ref, PR review และ release ref ออกจากกัน.

เชื่อมข้อสอบ: L10 ให้ทำนาย graph, อธิบาย repository state และเขียนคำสั่ง/วาดผล. บท *Git Exam Reference* และ *Git Worked Transactions* ฝึกแยก refs/HEAD/index และตอบแบบ before → commands → after → why.

1.14 สรุปบทและฝึกตอบก่อนสอบ

แนวคิดที่ต้องอธิบายได้: อ่าน SDLC เป็นการตัดสินใจเรื่องความต้องการ การออกแบบ การสร้าง และหลักฐานรับงาน AI ช่วยลงมือได้ แต่มนุษย์ยังรับผิดชอบเป้าหมายและคุณภาพ

สิ่งที่ต้องสร้างได้: Requirements, Specification, Design, Tests และหลักฐานส่งมอบ

ตัวอย่างพร้อมวิธีคิด | คำถามสั้นพร้อมเหตุผล

ความต้องการของ Campus App เปลี่ยนทุกสัปดาห์ ควรเลือกวิธีทำงานแบบใด?
แนวตอบ: เลือกแนวทาง Adaptive และส่งส่วนงานเล็กที่ใช้งานได้ครบ ใช้ Scrum เมื่อมีเป้าหมายและรอบทบทวนชัด หรือ Kanban เมื่องานเข้าต่อเนื่อง ระบุเหตุผลและ Definition of Done ก่อนเริ่ม

เชื่อมแนวสอบ: Part 1: เหตุผลของ Requirement; Part 3: พื้นฐาน Git

ตรวจความพร้อม: ลองอธิบายโดยไม่เปิดนิยาม แล้วสร้างชิ้นงานจากโจทย์ใหม่ ระบุสมมติฐานและวิธีตรวจผลให้ครบ

บทที่ 2

Requirement Engineering: จากคำขอสู่ requirement ที่ทดสอบได้

แกนบท: requirement engineering สร้าง shared understanding และลดโอกาสที่ stakeholder, analyst และ developer เข้าใจคำเดียวกันคนละแบบ การ “ชะลอการเขียน code” เป็นงานวิศวกรรม ไม่ใช่ความล่าช้า

2.1 ทำไม stakeholder request ยังไม่ใช่ specification

Software ซับซ้อนเพราะบริบทผู้เกี่ยวข้องต่างกัน ภาษาธรรมชาติกำกวม เป้าหมายชนกับเวลา/งบประมาณ และการแบ่งระบบเป็น module สร้างปัญหาการประสานงาน. ประโยคอย่าง “ระบบกิจกรรมในมหาวิทยาลัยต้องใช้ง่ายและ reliable” ยังไม่บอกว่าใครใช้, ทำงานใด, ข้อมูลมาจากไหน, เมื่อผิดพลาดเกิดอะไร และใครตัดสินว่าเสร็จ

ชิ้นงาน | แยกข้อความก่อนแปลงเป็นงาน

มุมมอง	คำถาม	ตัวอย่างข้อมูล
Facts	ต้นทางระบุชัดว่าอะไร	ผู้ใช้ browse หมวดกิจกรรม
Assumptions	ทีมกำลังเชื่ออะไรโดยยังไม่ยืนยัน	หมวดหมู่ถูกอ่านจากฐานข้อมูล
Questions	ต้องถามใครเพื่อปิดความกำกวม	ใครสร้าง/แก้ไขหมวดหมู่?
Constraints	ข้อจำกัดที่บังคับ solution	endpoint ที่ lab ให้ใช้มีเฉพาะ GET /api/categories

Assumption / unresolved: อย่าเปลี่ยน assumption ให้กลายเป็น requirement โดยไม่แจ้งผู้มีอำนาจตัดสินใจ

2.2 Stakeholder ก่อน feature

Stakeholder คือบุคคลหรือองค์กรที่ได้รับผลจากระบบหรือมี legitimate interest. Lecture แยกกลุ่มเพื่อช่วยค้นหาความต้องการและอำนาจยอมรับ:

- **End users:** ใช้ product และอธิบาย task ที่ต้องทำ.
- **Customer:** ได้ประโยชน์โดยตรงหรือโดยอ้อม และอาจเป็นผู้จ่าย.
- **Developers/testers/maintainers:** ทำให้ระบบเกิดขึ้น ตรวจ และดูแล.
- **Affected parties:** ได้ผลทางบวก/ลบแม้ไม่ได้กดใช้ระบบ.

ระบุผู้อนุมัติและผู้มี domain knowledge ด้วย; คนที่ขอ feature ไม่จำเป็นต้องเป็น end user หรือผู้รับผิดชอบ policy เสมอไป.

2.3 Requirement levels และชนิด

ชนิด	คำถาม/ผู้ใช้หลัก	ตัวอย่าง
Business requirement	องค์กรต้องบรรลุเป้าหมายอะไร	ลดเวลารอการลงทะเบียน
User requirement	ผู้ใช้ต้องการบริการ/ข้อจำกัดใด	นักศึกษาต้องดู workshop ที่เปิดรับสมัคร
System requirement	ทีมต้อง implement function/service/ constraint ใด	FR: ระบบแสดง workshop ที่เปิดรับสมัครให้ผู้ใช้ที่ sign in
Functional requirement (FR)	ระบบทำอะไร; observable behavior	สร้าง registration เมื่อผ่านเงื่อนไข
Non-functional requirement (NFR)	ทำงานด้วยคุณภาพ/เงื่อนไขใด	uptime มากกว่า 90% (ต้องระบุช่วง/วิธีวัดให้ชัด)
Domain / business rule (BR)	policy ใดควบคุมพฤติกรรม	ชื่อ category ต้อง unique
Acceptance criterion (AC)	จะตรวจ feature นี้อย่างไรจึงยอมรับ	Given/When/Then ที่ observable
Constraint	ทางเลือกใดถูกจำกัด	platform, law, budget, data source

FR อธิบาย capability; BR คือ policy/condition อาจใช้กับหลาย FR และมัก trace/test แยกได้. แยก BR เมื่อช่วย review, reuse หรือ change impact; ไม่จำเป็นสร้าง BR สำหรับทุกรายละเอียดเล็ก ๆ.

2.4 เขียน FR และ NFR ให้ตรวจได้

รูปแบบช่วยเริ่ม FR: The system shall + action + object + condition. ใส่ actor, boundary หรือ condition เมื่อจำเป็นต่อการตีความ. หลีกเลี่ยง “ระบบควรจะทำได้” โดยไม่บอก behavior ที่มองเห็นได้ และคำกว้างอย่าง simple, easy, reliable หากไม่มีเกณฑ์ตรวจ

ตัวอย่างพร้อมวิธีคิด | ตัวอย่างแปลง FR

ก่อน: “ค้นหาชื่อนักศึกษาได้.”
คำถาม: ใครค้น? ค้นจากข้อมูลใด? ต้องแสดงข้อมูลอะไร? มีสิทธิ์ค้นได้ทุกคนหรือไม่?
หลัง (เมื่อยืนยัน scope แล้ว): “The system shall show matching student records to an authorized coordinator when the coordinator submits a name query.”
เหตุผล: rewrite ระบุ actor, trigger, output และ authorization condition ซึ่งทำให้ derive test ได้. หาก source ยังไม่ยืนยันข้อมูลที่แสดง ให้คงเป็น question ไม่เติมเอง.

NFR ต้องมี quality attribute, measurement, operating condition และ threshold ที่ตกลงกัน ตัวอย่าง “registration ต้องเร็ว” ยังขาดจุดเริ่ม/จุดจบ, percentile, load และ threshold. ใช้ placeholder จนกว่าจะได้คำตอบ:

ชิ้นงาน | NFR rewrite pattern

The registration operation shall complete within [T seconds] for [specified percentile] under [specified load/ environment].
Assumption / unresolved: ค่า [T], percentile และ condition ต้องมาจาก stakeholder/measurement target; อย่าคิดตัวเลขขึ้นเพื่อให้ดู testable.

2.5 Quality checklist ของ requirement

คุณภาพ	ถามตัวเอง	สัญญาณปัญหา
Clear	มีความหมายเดียวหรือไม่	adjective คลุมเครือ / actor ไม่ชัด
Testable	reviewer ตรวจ output/evidence ได้ไหม	ไม่มี observable result / metric
Feasible	ทำได้ด้วยเวลา/เครื่องมือ/ทักษะหรือไม่	ต้องพึ่งสิ่งที่ไม่มีหรือขัด constraint
Relevant	เชื่อม stakeholder need/goal หรือไม่	feature ไม่มีเจ้าของประโยชน์
Atomic	เปลี่ยนและตรวจแยกได้ไหม	มีหลาย behavior เชื่อมด้วย “และ”
Traceable	มี ID, source, owner และ evidence path ไหม	ไม่รู้ว่าใครขอ/จะตรวจที่ไหน

2.6 Requirement Engineering loop

กระบวนการเป็น loop: elicit/clarify → analyze/classify → negotiate conflicts → document → validate → manage change/trace. คำตอบใหม่อาจบังคับให้ย้อนแก้ assumption หรือ scope; เก็บเหตุผลไว้เพื่อให้ทีมตอบได้ว่า “ทำไมเลือกทางนี้”

2.7 Acceptance criteria: เงื่อนไขรับงานเฉพาะ feature

รูปแบบ Given/When/Then ช่วยแยก initial state, action และ observable result. AC ควร specific, observable, testable และไม่ผูกกับ implementation ที่ไม่ใช่ข้อจำกัดของผู้ใช้. AC ไม่ได้แทน requirement ทั้งหมด; มัน operationalize การยอมรับของ feature หนึ่ง

ชิ้นงาน | Rule: จำกัดจำนวนการลงทะเบียน

Business Rule: นักศึกษาหนึ่งคนลงทะเบียน workshop ได้สูงสุด 3 รายการต่อ semester.
AC: Given นักศึกษามี registrations ครบ 3 รายการใน semester เดียวกัน; When นักศึกษาพยายามสร้างรายการที่ 4; Then ระบบไม่ยอมรับ registration เพิ่ม.
Verification: boundary test ที่ 2, 3 และ 4 รายการ พร้อมตรวจ semester scope.

Assumption / unresolved

source ไม่บอกว่า cancelled registration ยังนับรวม limit หรือไม่. ให้ถามหรือระบุ assumption ที่ใช้ในคำตอบ; test ต้องไม่ซ่อน policy นี้.

2.8 จากข้อความที่ไม่ atomic สู่ requirement set

ชิ้นงาน | Before → Transformation → Finished artifact

Before: “students cancel a registration and coordinators see participant lists.”
Why split: คนละ actor, capability, permission และ verification; การเปลี่ยนหนึ่งข้อไม่ควรทำให้ต้องแก้อีกข้อโดยไม่จำเป็น.
FR-04A: The system shall allow a student to cancel a permitted registration.
FR-04B: The system shall allow an authorized coordinator to view the participant list for a workshop.
ต่อด้วย AC แยกให้แต่ละ ID และเชื่อม ID ไปยัง test/evidence.

2.9 Lightweight proposal: ตัดสินใจว่าจะสร้างอะไร

Lecture เสนอ องค์ประกอบ 8 ช่อง: **Problem, Users, Solution, Expected Value, Scope, Risk, Effort, Evidence.** Proposal เป็นเครื่องมือชวนตัดสินใจ ไม่ใช่การการันตีโครงการ

องค์ประกอบ	คำถาม	WorkshopHub source review
Problem	pain ปัจจุบันคืออะไร	แบบฟอร์มแยกและรวมข้อมูลด้วยมือ
Users	ใครใช้/ได้รับผล	students และ coordinators
Solution	แนวทางแก้คืออะไร	WorkshopHub
Expected Value	ผลที่คาดหวังคืออะไร	ลด duplicate/error และงาน admin
Scope	อะไรอยู่/ไม่อยู่ในรอบแรก	free workshops ก่อน; payment/ certificates ภายหลัง
Risk	อาจล้มเหลวหรือเกิดผลเสียอะไร	ไม่ระบุใน source
Effort	ใช้ทรัพยากรประมาณเท่าไร	“เสร็จก่อน semester หน้า” เป็น deadline ไม่ใช่ effort estimate
Evidence	พิสูจน์ completion อย่างไร	ไม่ระบุใน source

Source Proposal → Gap Analysis → Recommendation

ทำเครื่องหมาย risk/evidence ว่าขาดใน proposal ต้นทาง แล้วเสนอให้เติมในฐานะ recommendation ของ study note. ห้ามเขียนเพิ่มแล้วทำให้ผู้อ่านเข้าใจว่าเป็นข้อมูลต้นฉบับ.

2.10 Traceability: ความต้องการต้องเดินทางไปถึงหลักฐาน

Traceability เชื่อม Stakeholder need → Requirement → Acceptance criteria → Specification → Design/UML → task/ backlog → implementation → test → evidence/release. หาก NFR ไม่มี threshold จะยังผูก planned test ได้ แต่ evidence status ต้องเป็น blocked/unresolved ไม่ใช่ pass

ชิ้นงาน | Traceability matrix แบบย่อ

Req	Source	AC	Artifact	Check	Evidence
FR-WRK-01	Coordinator	AC-WRK-01	Workshop list behavior	Functional test	Pending
NFR-WRK-01	Coordinator	Threshold unresolved	Performance condition	Measurement	Blocked: threshold unresolved
BR-WRK-01	University policy	AC-BR-01	Registration rule	Boundary test	Pending

สรุปบท 2

- เริ่มจาก stakeholders, facts, questions, assumptions และ constraints; ค่อยสร้าง feature.
- FR = behavior, NFR = quality/operating expectation, BR = domain policy, AC = feature-specific acceptance check.
- Requirement ที่ดี Clear, Testable, Feasible, Relevant, Atomic, Traceable.
- Unknown ต้องเห็นได้; source gap ไม่ใช่ใบอนุญาตให้ invent.
- Traceability ทำให้รู้ที่มา วิธีตรวจ และหลักฐานปลายทาง.

Exam Insight: เมื่อโจทย์สั่ง classify, quality-check, plan a check และ rewrite ให้ส่งคำตอบเป็นสี่ชั้นนี้ พร้อม Why, missing info และ assumption.

2.11 Complexity และกิจกรรมของวิศวกรที่ไม่ใช่ coding

Lecture Content | Core-SE | L2 หน้า 3–5. Domain complexity คือความยากของปัญหาจริง เช่นนโยบายลงทะเบียนหรือเงินคืน. Communication complexity เกิดจากคนมีความรู้คนละด้านและใช้คำเดียวกันต่างความหมาย. Constraint ทำให้ solution ที่ถูกต้องเพียงอย่างเดียวยังเลือกไม่ได้: ต้องเทียบ cost/time/conflict และ design goals เช่น reliability, maintainability, interoperability, usability และ verifiability

SE activity	ทำอะไร	ผลที่ review ได้
Modeling	จับ requirement model แล้วเทียบ solution model	โมเดลระบบตอบ need ไม่หลุด domain
Problem solving	เลือก solution อย่างเป็นระบบเมื่อมี change	alternatives/trade-off/decision
Knowledge acquisition	ถาม/ลอง/อ่าน; กลับไป unlearn ความเข้าใจเก่า	fact/assumption ที่อัปเดตพร้อม source
Rationale management	เก็บว่าทำไมเลือกวิธีนี้	reason ที่กลับมาตรวจได้เมื่อ assumption เปลี่ยน

เหตุผล: Divide and conquer ลดสิ่งที่ต้องคิดพร้อมกัน แต่เพิ่มความจำเป็นต้องระบุ interface ระหว่าง modules. แบ่งงานแล้วไม่ตกลง interface จะย้าย complexity ไปวัน integrate

2.12 Requirement มีสองแกนการจัดประเภท

Lecture Content | Exam-High | L2 หน้า 11–18. User/System/Business requirements บอกระดับและผู้อ่าน; FR/NFR/Domain rule บอกลักษณะเนื้อหา. ห้ามคิดว่า “user requirement ต้องเป็น FR เท่านั้น” หรือ “business requirement เท่ากับ business rule”. Business requirement ระบุ outcome ที่คนลงทุนต้องการ เช่นลดงานธุรการ; BR เป็นนโยบายที่ govern behavior เช่นชื่อ category ต้อง unique

ชิ้นงาน | จำแนกพร้อม purpose และ verification

Statement	Class	Purpose	Check
ลด duplicate registrations	Business objective	business value	เทียบ baseline กับช่วงหลังใช้งาน; ยังต้องกำหนด metric
Student ดู workshop ได้	User-level FR	user capability	scenario ดูรายการที่อนุญาต
API คืน categories จาก DB	System-level FR	implementable behavior	เพิ่ม DB row แล้ว API ต้องคืน row
Uptime มากกว่า 90%	NFR ตาม L2 example	quality condition	วัดช่วงเวลา/นิยาม downtime; ตัวเลขไม่ใช่กฎทุกระบบ
Category name unique	Domain rule	data invariant	insert ชื่อซ้ำต้องไม่สำเร็จ
React + TypeScript	Constraint	จำกัด solution space	ตรวจ stack/build configuration

คุณภาพตาม L2 หน้า 17: Clear, Testable, Feasible, Relevant, Atomic, Traceable. ส่วน validation หน้า 26 ใช้ Validity, Consistency, Completeness, Realism, Verifiability, Comprehensibility, Traceability, Adaptability. ทั้งสองชุดมีหน้าที่ต่างกัน: ชุดแรกตรวจ work item ก่อนทำ; ชุดหลังตรวจ requirement set ว่าระบบที่กำหนดตอบ customer จริงและเปลี่ยนได้หรือไม่

2.13 Elicitation → analysis → specification → validation → change

Lecture Content | L2 หน้า 19–27. Elicitation ใช้ interview/observation/ข้อมูลเดิมเพื่อเก็บ needs/pain/workaround; analysis จัดกลุ่ม แก้ conflict และ priority; specification ทำข้อตกลงที่ implement ได้; validation review/prototype/derive tests; change management เก็บ reason/owner/impact/decision โดยรักษา stable IDs

ตัวอย่างพร้อมวิธีคิด | อ่านคำขอ event app โดยไม่แอบเติม feature

คำขอ: students browse categories, know system working, easy/reliable.

1. **Facts:** มี category browsing และ system-health visibility. คำว่า easy/reliable เป็น quality expectations ยังวัดไม่ได้.
2. **Questions:** category มาจากไหน? ใครดูแล? sort อย่างไร? backend offline แสดงอะไร? Reviewer ยอมรับจากหลักฐานใด?
3. **Assumptions:** seed-only database และ student browser user ตาม Lab 1; ระบุ owner/confirmation path.
4. **Scope:** health/model/seed/API/button/loading/success/failure; purchase/payment/login/admin editing ยังนอก increment.
5. **Rewrite:** FR-CAT-01 คืนรายการ DB; AC แสดงผลจาก API เมื่อกด Check System; negative AC แสดง useful failure หาก backend unreachable.
6. **Verify:** DB/API/UI evidence เชื่อม IDs; mock UI test ตรวจ rendering แต่ต้องมี API/DB test เพื่อพิสูจน์ data source.

เหตุผล: แยก fact กับ assumption ช่วยไม่ให้แผนเริ่มจากสิ่งที่ stakeholder ไม่เคยขอ. ไอเดียใหม่บันทึก backlog ได้โดยไม่ใส่ release นี้ทันที

2.13.1 Acceptance Criteria ที่ทำเป็น test ได้

Given คือ state/precondition ที่จัดเตรียมได้; When คือ action/event; Then คือ observable outcome. AC ไม่ควรอธิบาย internals เมื่อผู้รับงานต้องการ behavior. “looks nice” ไม่พอ; “AI says complete” ไม่มี acceptance authority

ชิ้นงาน | AC และ boundary cases: categories

AC-CAT-1: Given backend/DB running และมี Workshop, Sport; When student กด Check System; Then UI แสดง status และสองชื่อที่ API คืน.
AC-CAT-2 (Explanation / Intuition เพิ่มเพื่อฝึก): Given DB ไม่มี categories; When request สำเร็จ; Then แสดง empty state ไม่ใช่ error. ต้องตกลงข้อความกับ stakeholder ก่อน baseline.
AC-CAT-3: Given backend unreachable; When กดปุ่ม; Then loading จบและแสดง useful failure. Test ไม่ควรผ่านถ้า spinner ค้าง.
Test design: normal/empty/failure คนละ data setup. ไม่ยึดหน้าตา screenshot หนึ่งภาพแทนสาม behavior

2.14 Validation และ change request แบบครบวงจร

ตัวอย่างพร้อมวิธีคิด | เพิ่ม admin category editing กลาง Lab 1

Explanation / Intuition. Customer ขอ feature ที่อยู่นอก scope เดิม. เริ่มด้วย change record CR-01: requester/reason/date/requirements affected. วิเคราะห์ API authorization, UI form, migration, tests, effort/dependency และ release risk. ให้ทางเลือก: defer ไป later backlog หรือ replace feature priority ต่ำภายใต้ deadline เดิม. Decision ต้องมี approver; update SRS/AC/design/tasks/trace/test เมื่อ accept. ห้ามทำเพิ่มเงียบ ๆ เพราะทำให้ scope และ evidence ไม่ตรงแผน

ชิ้นงาน | Rationale record

Decision: Lab 1 uses seed-only categories. **Alternatives:** admin editor หรือ hard-code UI. **Reason:** slice เล็กพอและยังพิสูจน์ DB → API → UI. **Consequences:** ต้องใช้ migration/seed; เปลี่ยนข้อมูลผ่าน UI ยังไม่ได้. **Revisit when:**

admin editing ถูก approve. **Evidence:** schema/seed/endpoint/test ไม่ใช่แค่ข้อความ Done

2.15 Lightweight proposal และ traceability ที่ใช้ได้จริง

Lecture Content | L2 หน้า 28–33. Proposal แปดช่อง: problem/users/solution/expected value/scope/risks/effort/evidence. Effort เริ่มจาก rough estimate เพื่อเลือก scope ไม่ใช่ deadline promise. เลือก MVP จาก value/risk/effort/dependency/learning โดยต้องเป็น coherent increment

ตัวอย่างพร้อมวิธีคิด | Feature proposal: Check System

Problem/users: ผู้เรียนและ reviewer ไม่ทราบ frontend ต่อ backend/DB ได้จริงหรือไม่. **Solution:** ปุ่มเดียวเรียก health และ categories. **Value:** ลด uncertainty ของ full stack ก่อน ticket workflow. **Scope:** read-only และ UI states; ไม่เพิ่ม login/payment. **Risk:** env/migration/seed ต่างกัน. **Effort:** API+DB, UI states, tests+evidence และ integration ต้อง estimate แยกก่อนตกลง. **Evidence:** passing endpoint test, idempotent seed, loading/success/failure proof และ reviewer acceptance.

ชิ้นงาน | Traceability row ที่ไม่จบแค่ feature name

Link	ตัวอย่าง
Need → Req	stakeholder category request → FR-CAT-01
Req → Design	GET /api/categories; Category model; UI states
Design → Task	Issue#3 model/seed/API; Issue#4 UI
Task → Code	feature branches/PR; schema, handler, App
Code → Test	API DB-backed test + UI behavior tests
Test → Release → Demo	result with commit ID; accepted main; screenshot/video

Quality check: link เปิดไป artifact จริงและ version ตรงกัน. ถ้า source เปลี่ยน requirement ต้องไล่ downstream impact; ถ้า code เพิ่ม behavior ต้องไล่กลับหา source หรือ explicit approved addition

2.16 Lab foundations: API, database และ UI evidence

Lecture Content | L2 หน้า 35–43. Client/server/docs แยก purpose; install dependencies ในแต่ละ package. Express create App() สร้าง app โดยไม่ bind port เพื่อให้ Supertest import ได้; app.listen() เป็น startup concern. HTTP method/path/status/body เป็น contract: GET อ่าน resource; POST สร้าง; 200/404/500 ต้องมีความหมาย ไม่ใช่คืน 200 ทุก failure

ชิ้นงาน | API contract และ test ที่ meaningful

GET /api/health → 200 JSON {status:"ok", service:"TokTickIT API"}.

Supertest assert status/content type/body. Vitest mount App, click Check System, await Workshop ที่ response คืน. Arrange/Act/Assert และหนึ่ง behavior ต่อ test ช่วยจับ defect ไม่ผูกกับ implementation internals

Prisma workflow: initialize schema/config → .env connection → migrate → generate client → query/seed. Category name ใช้ unique constraint; upsert ทำ seed ซ้ำโดยไม่ duplicate. React state แยก idle/loading/success/error, clear previous error และหยุด loading หลัง outcome; response data ต้องมาจาก API. ทดสอบ UI อย่างเดียวไม่พิสูจน์ migration หรือ idempotent seed

2.17 สรุปบทและฝึกตอบก่อนสอบ

แนวคิดที่ต้องอธิบายได้: แยกระดับความต้องการออกจากชนิด FR/NFR/BR ถามสิ่งที่กำกวมก่อนตั้งข้อตกลง และเชื่อมทุกข้อไปยังวิธีทดสอบ

สิ่งที่ต้องสร้างได้: ตาราง Requirement, Acceptance Criteria, Proposal และ Traceability Matrix

ตัวอย่างพร้อมวิธีคิด | คำถามสั้นพร้อมเหตุผล

“สมัครได้เร็ว และไม่เกิน 3 รายการต่อภาคการศึกษา” ต้องถามหรือแก้อะไร?
แนวตอบ: เร็วเป็น NFR ต้องตกลงเวลา วิธีวัด และภาระงาน ส่วนขีดจำกัด 3 เป็น BR ต้องถามวิธีนับรายการยกเลิก แล้วทดสอบขอบ 2→3 และการปฏิเสธรายการที่ 4

เชื่อมแนวสอบ: Part 1: Requirements และ Proposal

ตรวจความพร้อม: ลองอธิบายโดยไม่เปิดนิยาม แล้วสร้างชิ้นงานจากโจทย์ใหม่ ระบุสมมติฐานและวิธีตรวจผลให้ครบ

บทที่ 3

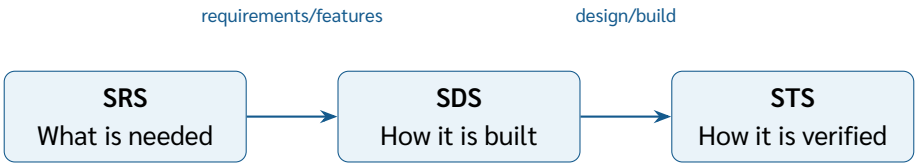
Specifications, SDD, Testing และ TDD

แกนบท: specification ลดช่องว่างระหว่าง need กับ implementation; tests เป็นหลักฐานที่ตอบกลับไปยัง requirement. การทำงานจริงย้อนกลับและทำ parallel ได้ ไม่ใช่การส่งเอกสารข้ามกำแพงครั้งเดียว

3.1 งานของ engineer ไม่ได้จบที่ requirements

Lecture ระบุผลลัพธ์ที่วิศวกรควรทำได้: Requirements, Specifications, Design Contracts, Acceptance Criteria, Test Plan และ Implementation/Coding Plan. AI ช่วยร่างและ refine ได้ แต่ไม่รับผิดชอบความถูกต้องแทนทีม. SDLC ทำงานเป็นวงจร: บาง activity เรียงลำดับ บางส่วนทำคู่ขนาน และผลจาก test อาจพากลับไปแก้ requirement/design

3.2 SRS, SDS, STS: What / How / Verify



Artifact	ตอบคำถาม	สิ่งที่ควรปรากฏ
SRS	ระบบต้องทำอะไร/มีข้อจำกัดอะไร	FR, BR, NFR, constraints, scope, assumptions, acceptance links
SDS	ระบบจะถูกออกแบบและประกอบอย่างไร	architecture, components, interfaces, data, UI conventions, errors, security decisions
STS	ใช้อะไรตรวจความต้องการ/การออกแบบ	test levels, scenarios, test data, expected result, environment, evidence

3.3 SDD workflow ที่ trace ได้

1. วิเคราะห์ stakeholder need → FR/BR/NFR ใน SRS.
2. ทำ complete feature inventory; ตรวจว่า scope ไม่ตกหล่น/ซ้ำ.
3. สร้าง requirements-to-features traceability.
4. กำหนด system-level SDS: architecture, stack, UI/API conventions.
5. สร้าง feature-level SDS: purpose, workflow/ state, rules, data/ API/ UI, permissions, validation, AC และ test considerations.
6. แตก implementation tasks และ tests; review consistency ก่อน build.

System-level vs feature-level SDS

System-level ตัดสินสิ่งร่วมทั้งแอป เช่น architecture, technology stack, data model, auth/authz, UI design system, API/error conventions และ cross-cutting NFR. Feature-level อธิบาย scope และ behavior ของ feature หนึ่ง. อย่าฝัง endpoint ทุกตัวหรือ schema ละเอียดลงใน system-level หากยังเป็นงานออกแบบระดับ feature.

ชิ้นงาน | SDS outline ที่นำไปใช้ได้

System: architecture/ components; tech stack; data domains; identity/ permission approach; API/ error convention; shared UI; logging/security/performance.
Feature: purpose/scope → linked SRS IDs → workflow/state → business rules → data change → interfaces → UI behavior → permission → validation/error → acceptance → test considerations.

3.4 Requirement, specification และ design contract

Requirement อาจสรุป need; specification ทำให้ behavior/condition ชัดขึ้น; design contract กำหนดขอบเขตระหว่าง component ว่า input, output, error และ responsibility คืออะไร. แยกสามระดับเพื่อป้องกัน solution detail ไปกลบความต้องการ แต่ต้องเชื่อมด้วย traceability

3.5 Testing levels และชนิด

ระดับ	Scope	เป้าหมายหลัก
Unit	function/method/class เล็ก ๆ	ตรวจ behavior หน่วยแยก
Integration	หลาย units/components/modules	ตรวจ data flow และ interface
System / E2E	assembled system ใน flow จริง	ตรวจ FR และ NFR ทั้งระบบ
UAT / acceptance	business users กับ real scenarios	ยืนยัน business need และให้ sign-off

Regression ไม่ใช่ระดับที่สูงขึ้น แต่เป็นการรันทดสอบเดิมซ้ำหลังแก้/เพิ่มงานเพื่อดูว่า behavior ที่เคยผ่านยังไม่พัง. Functional testing ตรวจ “ทำอะไร”; non-functional testing ตรวจ “ทำได้ดีแค่ไหน” เช่น performance, security, availability. ระดับและชนิดสามารถประกอบกันได้: เช่น integration + functional หรือ system + performance

3.6 จาก requirement ไปยัง test case

1. เลือก requirement ID และ acceptance condition.
2. กำหนด precondition, input, action, expected result.
3. เลือก level ที่เหมาะกับ boundary ที่ต้องตรวจ.
4. ระบุ normal, boundary และ failure path ที่ source รองรับ.
5. เก็บ actual result, environment และ evidence.

ชิ้นงาน | Test case mini-template

ID / linked Req: TC-REG-04 / BR-REG-01
Precondition: student มี 3 registrations ใน semester S
Action/Input: request registration ที่ 4 ใน S
Expected: registration ถูกปฏิเสธและไม่มี record ใหม่
Evidence: test result + record count ก่อน/หลัง
Assumption to resolve: cancellation นับรวมใน limit หรือไม่

3.7 TDD และ feedback loop

Test-Driven Development เป็นวงจรสั้น: เขียน test ที่ยังไม่ผ่าน → เขียน implementation ให้น้อยที่สุดเพื่อให้ผ่าน → refactor โดยรักษา tests. ประโยชน์คือทำให้ behavior ถูกระบุเร็วและลดการ refactor ที่ไม่มี safety net. TDD ไม่ได้ทดแทน integration, E2E, UAT หรือ NFR testing เพราะ unit test ไม่พิสูจน์ว่าระบบประกอบและตรง business need

ข้อผิดพลาดที่พบบ่อย | นับจำนวน test เป็นหลักฐานคุณภาพ

test จำนวนมากอาจ assert ผิดสิ่ง, ไม่เชื่อม requirement หรือ mock จนไม่ตรวจ integration. ประเมิน coverage เชิง requirement และ boundary/failure behavior รวมกับคุณภาพของ assertion/evidence.

สรุปบท 3

SRS = need/constraints; SDS = design; STS = verification. ทำ feature inventory และ traceability ก่อนแตกงาน. เลือก test level ตาม boundary; regression เป็นการรันทดสอบซ้ำ; TDD เป็น feedback loop ระดับ implementation.

3.8 จาก requirement ถึง engineering contract ที่ใช้สร้างระบบได้

Lecture Content | Core-SE / Exam-Support | L3 หน้า 3–7,33–43. วิศวกรต้องสร้าง Requirements, Specifications, Design Contracts, Acceptance Criteria, Test Plan และ Implementation Plan ได้แม้ไม่มี LLM. Requirement บอก need/constraint; specification ทำให้ input/state/result/error ชัด; design เลือกโครงสร้างที่จะทำตามนั้น; STS ระบุวิธีตรวจ. คำย่อ **SDD** ในหัวข้อ Spec-Driven Development เป็นวิธีทำงาน; **SDS** คือ Software Design Specification เป็นเอกสาร ห้ามใช้แทนกัน

ชิ้นงาน | SRS vs System SDS vs Feature SDS vs STS

Artifact	คำถาม	เนื้อหา	Reviewer
SRS	ต้องการอะไร	FR/BR/NFR + purpose/actors/scope/assumptions/constraints	stakeholder + engineer
System SDS	กฎร่วมและ architecture	stack/modules/data/auth/UI/API/error/NFR conventions	architect/security/tech lead
Feature SDS	feature ทำงานอย่างไร	workflow/state/rules/data/API/UI/permission/validation/AC	implementer + reviewer
STS	จะพิสูจน์อย่างไร	cases/data/setup/action/expected/evidence; test levels	QA + developer + PO

เหตุผล: เอกสารใหญ่ที่มีแค่ architecture ยังไม่พอให้ implement ทุก screen. ในทางกลับกัน screen spec ที่ไม่อิง convention ร่วมจะทำ auth/error/theme ต่างกันทุก feature

3.9 Recommended design process: ปิด coverage ก่อน code

Lecture Content | L3 หน้า 34–52; L4 หน้า 89,131–137. ขั้นครบ: SRS → complete feature inventory → requirement-to-feature trace → system SDS → select sprint vertical slice → per-feature contract (SDS+UI/API/Data+AC+STS) → bounded Issue implementation → tests/human review → PR ไป staging → sprint integration/regression/UAT → release main. หาก sprint test fail ให้สร้าง fix branch บน staging และแก้ SDS/STS ที่ผิดก่อนวนทดสอบใหม่

ตัวอย่างพร้อมวิธีคิด | Requirement หนึ่งข้อไม่เท่ากับ feature หนึ่งชิ้นเสมอ

L3 แยก capability upload attachments (FR) จาก max 5 MB, max five active และ removed attachments no longer downloadable (BR). Feature Attachment ครอบหลาย requirements; authorization และ data model อาจใช้ร่วมกับ Ticket Detail. ถ้าแตก BR ทุก field เป็น feature จะทำ inventory สับสน; ถ้ารวม FR ใหญ่จน review/test แยกไม่ได้ traceability จะอ่อน

3.10 Worked feature: Create Ticket contract ครบ 11 ช่อง

Explanation / Intuition based on L3 หน้า 35–42,58; L4 หน้า 100,133. ID ด้านล่างใช้เพื่อสอน pattern ไม่อ้างว่าเป็นเลขใน Lab sheet ที่ไม่ได้แนบ. ค่า input และข้อความที่ source ไม่กำหนดต้อง approve ก่อน baseline

Contract section	Draft artifact พร้อมเหตุผล
1 Purpose/scope	requester สร้าง ticket และกลับดู ticket ตนเอง; staff actions เป็น later slice
2 Related requirements	FR-EX-CREATE; BR-EX-INITIAL; NFR-EX-OWNERSHIP; link SRS version
3 Workflow/state	authenticated requester → form → validate → save → New → detail; invalid กลับ form ไม่สร้าง record
4 Business rules	new ticket starts New; required fields/ valid category ต้อง ตกลง; limit attachment ใช้ 5 MB/five ตาม example เมื่อ scope เลือกใช้
5 Data model	Ticket identity/requester/category/description/status/time; Attachment metadata link; server กำหนด owner
6 API	proposed POST /api/tickets; body category/description; response saved ID/ status; unauthenticated/invalid/error paths
7 UI	loading submit, invalid field message, success detail, failure retry; preserve input เมื่อแก้ได้
8 Permissions	requester create own; client requester Id ไม่ใช้เป็น identity; endpoint ตรวจทุกครั้ง
9 Validation/errors	type/required/size checks backend; save transaction fail ต้องไม่อ้าง success; safe error ไม่เผย credential
10 AC	valid create shows stored ID and New; invalid creates no ticket; other requester cannot read
11 Test considerations	unit validation, API real persistence, UI states, E2E create → My Tickets → Detail; unauthorized direct API

ข้อผิดพลาดที่พบบ่อย | ปิด gap ก่อน baseline

New ใน TokTickIT attachment example ไม่ใช่ Requested ใน ABC Service Ticket. ทั้งสองระบบใช้ vocabulary ต่างกัน. ห้ามนำ state ข้ามระบบเพราะมีคำว่า ticket เหมือนกัน. Contract draft ไม่ใช่ approved lab solution จนกว่าจะเทียบ SRS ที่ใช้จริง

3.10.1 API/data/UI contract ต้องเห็น invariants เดียวกัน

ชิ้นงาน | Input → validation → transaction → output

Precondition: active authenticated requester. **Input:** selected category และ description. **Validation:** category exists, fields valid, client cannot impersonate owner. **Persistence:** insert Ticket with req.user identity/status/time; atomic rollback on failure. **Output:** saved ticket identifier and detail. **Error:** invalid field tells user what to correct; infrastructure error uses correlation ID without secrets. **Invariant:** response success implies record exists under that requester.

เหตุผล: หาก UI แสดง success ก่อน backend durable save เสร็จ อาจเกิดงาน “เสร็จ” ที่ My Tickets หาไม่เจอ. AC ควรตรวจ persistence หรือ subsequent read ไม่ใช่ตรวจ toast อย่างเดียว

3.11 STS และ Test Plan: มี setup/action/oracle/evidence

Lecture Content | L3 หน้า 53–56; L4 หน้า 136–139. Test scenario เป็นเรื่องที่จะตรวจ; test case ระบุ precondition/ data/ actions/ expected result. Test Plan รวม scope/ levels/ environment/ execution/ criteria; STS ทำ testable contract สำหรับ feature. Expected result คือ oracle ที่ใช้ตัดสิน ไม่ใช่ “works correctly”

Case/level	Arrange	Act	Expected / evidence
TC01 unit	valid/ invalid description fixtures	validate input	valid accepted; invalid reason; report
TC02 API	test DB + requester A	POST ticket, then GET it	ID persisted; owner A; state New
TC03 API	requester B; A’s ticket ID	direct GET A ticket	deny safely; no content/ attachment leak
TC04 UI	API delayed	click submit	loading; prevent accidental repeat; wait for result
TC05 UI	API validation failure	submit invalid field	field error; input preserved; no success
TC06 E2E	running app/API/DB	create → list → detail	same ID/data in all views
TC07 regression	existing Lab 1 health/ category	run suite after feature	old behavior remains correct
TC08 UAT	PO + agreed realistic task	create ticket and inspect output	business acceptance recorded

Assumptions: duplicate-submit policy/status codes/exact validation are teaching additions and must be specified. Test execution status ที่ยังไม่รันคือ Pending ไม่ใช่ Pass; book examples เป็น test designs ไม่ใช่ผลรันของ production application

3.12 เลือก testing level โดยมองขอบเขตและผู้รับรอง

Unit ตรวจเล็กและ isolated ใช้ mocks/stubs ของ external dependencies. Integration ตรวจ implemented parts ทำงานร่วมกัน รวม DB/API/interfaces จริงตาม scope. System ตรวจ sprint increment ในแอปทั้งระบบเทียบ SRS/NFR. E2E เดิน user workflow ผ่าน UI/backend/DB/external boundary. UAT ให้ customer/PO ตรวจ business need และ sign-off. L3 รวม System/E2E ในตารางภาพรวม; L4 แยกเพื่ออธิบาย scope ชัดขึ้น

ชิ้นงาน | Specialized checks รอบ system/release

Regression รัน ของ เดิม หลัง change; smoke ตรวจ basic alive หลัง setup/ release; performance/ load ตรวจ volume+response target; security ตรวจ misuse; compatibility ตรวจ intended browser/device; recovery/failover ตรวจ crash แล้วกลับมาพร้อม state ที่ถูก. ไม่ใช่ test stage เพิ่มที่ต้องเรียงทุกชนิดเหมือนกันทุก project

3.13 TDD: ความหมายของ Red, Green และ Refactor

Lecture Content | L1 หน้า 30; L7 หน้า 36. Red คือเห็น test fail ด้วยเหตุผลที่ตั้งใจ ก่อน implementation; Green เขียนพอผ่าน behavior; Refactor ปรับ design โดยรักษา suite green. Test ที่ผ่านตั้งแต่ไม่มี implementation อาจไม่ตรวจ target behavior. Spec-Driven และ TDD ใช้ร่วมกันได้: spec บอก behavior/constraint; test บอก check ที่ executable

ตัวอย่างพร้อมวิธีคิด | TDD สำหรับ discount validation

Applied Example + Explanation. Session POS BR-009 จำกัด discount amount ระหว่าง 0 กับ subtotal.

1. Red: subtotal=100, discount=101 ต้อง rejected; รันเพื่อเห็น fail หาก validator ยัง accept ทุกค่า.
2. Green: implement lower/upper bound; ตรวจ 0,100 accept; -1,101 reject.
3. Refactor: แยก money validation ที่ใช้ร่วม โดย test เดิมยัง pass; อย่าเปลี่ยน expected ให้ตรง bug.

4. Trace: BR-009 → feature G → validator/API case → test output; UI validation อย่างเดียวไม่ enforce server.

3.14 Architecture evolution และการส่งมอบ

Lecture Content | L3 หน้า 28–30,51,57. Presentation เป็นหน้า interaction ของ human; Service layer headless ให้หลาย presentation channels เรียก business operation; Data layer เก็บและ query. POS desktop ที่มี logic อยู่เครื่องเดียว ต่างจาก web+services และ integrations: ยิ่งแบ่ง boundary ยิ่งต้องระบุ contracts/failure modes. CI ช่วย integrate/test บ่อย; CD ช่วย package/release ให้ feedback ไม่ติด deployment. Maintenance แก้ข้อผิดพลาด เปลี่ยนที่ UI ปกติไม่ได้ และปรับ requirements/environment โดยต้องเก็บ traceability

3.15 สรุปบทและฝึกตอบก่อนสอบ

แนวคิดที่ต้องอธิบายได้: SRS บอกสิ่งที่ต้องการ SDS บอกวิธีออกแบบ STS บอกวิธีตรวจ SDD เริ่มจากข้อตกลง ส่วน TDD ใช้รอบ Red–Green–Refactor

สิ่งที่ต้องสร้างได้: รายการ Feature, ข้อตกลง API/UI/Data, แผนทดสอบ และ Test Cases

ตัวอย่างพร้อมวิธีคิด | คำถามสั้นพร้อมเหตุผล

UI ของ Create Ticket แสดง success แต่เรียก GET แล้วไม่พบรายการ ถือว่า Done หรือไม่?
แนวตอบ: ยังไม่ Done เพราะข้อมูลไม่ได้บันทึกถาวรตามเงื่อนไขรับงาน ตรวจ Transaction และจังหวะตอบ success แล้วเพิ่ม API/Regression Test ที่อ่านข้อมูลกลับจริง

เชื่อมแนวสอบ: สนับสนุน Part 1 และ Part 3: Contracts และ Testing

ตรวจความพร้อม: ลองอธิบายโดยไม่เปิดนิยาม แล้วสร้างชิ้นงานจากโจทย์ใหม่ ระบุสมมติฐานและวิธีตรวจผลให้ครบ

บทที่ 4

UML และการจัดทำ SDS / STS

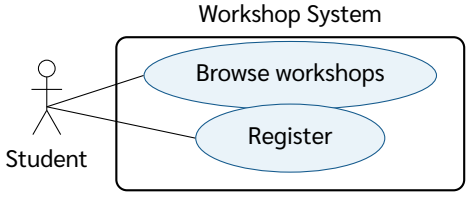
แกนบท: เลือก diagram ตามคำถามที่ต้องตอบก่อนวาด; รักษา semantics และทำให้ label/relationship อ่านได้. Diagram เป็น specification artifact ที่ใช้สื่อ behavior/structure ไม่ใช่ของตกแต่ง

4.1 เลือก UML จากโจทย์

สิ่งที่ต้องสื่อ	Diagram	เหตุผล
ใคร ใช้ ระบบ และ ทำ use case ใด	Use Case	ขอบเขต actor กับ service goal
workflow ทั้ง เส้น และ branch/loop	Activity	แสดง control flow, decision, partition
scenario หนึ่งตามเวลา	Sequence	แสดง lifeline และ message order
สถานะ ของ object และ การเปลี่ยน	State	แสดง lifecycle, trigger/guard/action
entity/ attribute/ relationship คงที่	Class	แสดง static structure และ multiplicity
หน้า จอ, field, action, validation state	UI prototype/list-form	เป็น specification ของ interface ไม่ใช่ exercise แต่งภาพ

4.2 Use Case Diagram: actor, boundary, relation

Actor คือ external role/system ที่โต้ตอบกับระบบ; วาด system boundary เพื่อบอก scope; use case ตั้งชื่อเป็น goal/action. Actor association เป็น solid line ไม่มี arrow. <<include>> คือ sub-behavior ที่ถูกเรียกเป็นส่วนบังคับ; <<extend>> คือ behavior เพิ่มแบบมีเงื่อนไข/ทางเลือก; generalization คือ specialization/inheritance. ตรวจทิศลูกศรของ include/ extend ตาม notation ที่ lecture ใช้ ไม่ใช้ความคุ้นชินแทนกฎรูป



อ่านภาพ: Student เป็น external actor; use cases อยู่ใน boundary. เส้น actor association ไม่มี arrow. ตัวอย่างนี้ยังไม่ระบุ include/extend เพราะ source ไม่ได้กำหนด subflow

แก้ diagram ที่ผิด: ตรวจตามลำดับ

Wrong: actor links มากเกินจน actor ดูเหมือนเรียก internal step ทุกตัว; ใช้ extend กับ mandatory behavior; duplicate use cases หลายชื่อทั้งที่เป็น goal เดียว.
Rule: actor เชื่อมกับ use case ที่มี interaction จริง; include = mandatory reused behavior; extend = optional/conditional extension; generalization = specialization.
Corrected: รวม use case ที่ซ้ำ, ลบ association ที่ไม่มี interaction, เปลี่ยน relation ตามเงื่อนไขจริง.
Why: ความหมายของ relation ทำให้ผู้อ่านอนุมาน behavior ได้; สัญลักษณ์ผิดคือ specification ผิด.

4.3 Activity Diagram: workflow และ control flow

ใช้ initial/final node, action, decision/merge, fork/join เมื่อมี parallel flow และ swimlane/partition เมื่อต้องแยกผู้รับผิดชอบ. Decision ต้องมี guard ชัด; loop แสดงการกลับไปทำ action; alternative flow ไม่เหมือน exception flow (error/failure). ตัวอย่าง POS: Start → add items → show total → [discount?] → choose payment → [payment success?] → receipt/inventory → End; payment decline ต้องมี path retry/switch/cancel ตาม requirement.

4.4 Sequence Diagram: หนึ่ง scenario ตามเวลา

Lifeline แทน participant; message arrow แสดง interaction ตามเวลา จากบนลงล่าง. เลือกเฉพาะ participants ที่มี interaction ใน scenario ไม่ยัดทุก component ของระบบ. ต้องรักษาลำดับ failure/retry หากเกิดใน path. อ่าน verb ใน scenario เป็น message; ตรวจ direction, reply และเงื่อนไข

ตัวอย่างพร้อมวิธีคิด | KU Registration: เก็บความล้มเหลวและ retry

Scenario source: Student login → resume draft ที่มี 2 courses → add course → submit แล้วพบ section เต็ม → เปลี่ยน section → submit ใหม่ → สำเร็จ → bank transfer → registration complete. Participants ที่โจทย์ระบุ ได้แก่ Student, KU Registration, KU Single Sign-On, KU Student Info, BBL Payment Gateway, KU Email.
Derivation: extract scenario → select lifelines that interact → translate verbs to messages → preserve first failed submit + retry → check order against scenario.
Assumption / unresolved: อย่าเพิ่ม hidden service/lifeline หรือ failure behavior ที่โจทย์ไม่ได้ระบุ; หาก diagram exam ต้องย่อ ให้ยังคง event ที่ทำให้ scenario ต่างจาก happy path.

4.5 State Diagram: lifecycle ไม่ใช่รายการ action

เริ่มจากคำที่เป็น status/state ของ object แล้วหา event/guard ที่เปลี่ยนสถานะ. ตัวอย่าง Service Ticket: Requested → Approved → In Progress → Closed. “approve”, “start work”, “close” เป็น transition/action ไม่ใช่ state. transition label ควรมี trigger และ actor/guard เมื่อจำเป็น เช่น approve [CCE]. เมื่อ Closed แล้ว source ระบุว่าไม่มี further action ให้ไม่มี outgoing transition ที่แต่งขึ้น

4.6 Class Diagram: static structure

Class แสดง class name, attributes พร้อม type, operations เมื่อมีประโยชน์, associations/multiplicities, composition/aggregation และ generalization. เลือก class จาก domain concepts/requirements ไม่ใช่สร้าง class ตามทุก noun ใน paragraph.

- Multiplicity บอกจำนวนความสัมพันธ์ เช่น 1, 0..1, 1..*.
- Composition ใช้ whole-part ที่ lifecycle ผูกแน่น.
- Aggregation ใช้ whole-part แบบ ownership อ่อนกว่า.

- Generalization ใช้ specialization เช่น CashPayment เป็น Payment.

4.7 UI prototype เป็น specification

derive fields, source of value, editability, requiredness, actions, validation และ state. ตัวอย่าง exam ระบุ required field ใช้ *, editable เป็น textbox, non-editable เป็น striped textbox, semester ใช้ current semester และ student info โหลดจาก logged-in user. อย่าตกแต่งจนกลบข้อมูลที่ต้องยืนยัน

ชิ้นงาน | UI field inventory

Field	Source	Editable?	Required?	Validation/state
Student ID	Logged-in user	No	Yes	แสดงค่าที่ระบบโหลด
Semester	Current semester	No	Yes	fixed ตามโจทย์
Course section	Student selection	Yes	Yes	ตรวจ seat/availability
Payment method	Student selection	Yes	Yes	branch ตาม payment option

4.8 SDS และ STS เชื่อม UML อย่างไร

SDS ระบุ architecture และ feature-level behavior; diagram ให้มุมมองหนึ่งของการออกแบบ. STS ระบุวิธีตรวจ behavior ที่ออกแบบ. UML แต่ละภาพควรมี reading guide และ semantic QA: actor/boundary, flow/guard, lifeline/message direction, state/transition, multiplicity; ตรวจคำไม่ชนเส้นและความละเอียดพออ่าน

Exam Insight: Lecture 10 เลือก Sequence, Activity, UI Prototype, Use Case, State สำหรับ UML/UI portion; Class diagram เป็นทักษะจาก lecture/applied example แต่ห้ามอ้างว่ามีคะแนนแน่นอนเมื่อ blueprint ไม่ระบุ.

4.9 Modeling method: อ่าน requirement แล้วเลือกมุมมอง

Lecture Content | Exam-High | L4 หน้า 3–75. UML ไม่ได้มีไว้ใส่ symbol ให้ครบ แต่ใช้ยืนยันว่า requirement ที่อ่านแปลงเป็น structure/behavior ที่สอดคล้องกัน. เริ่มจาก actor/role, entity/data, state, action, external system, condition และ side effect. จากนั้นเลือกภาพตามคำถาม: Use Case ใครทำอะไร; Activity workflow ที่มี branches; Sequence interaction ของ scenario; State lifecycle ของ object; Class data/relationships. UI Prototype อธิบายสิ่งที่ user เห็น/แก้/กด

ชิ้นงาน | Extraction worksheet ก่อนวาด

Extract	คำถาม	Example: ABC Service Ticket
Actor	ใครนอก system ใช้ capability	Customer, CCE, Programmer, IT Admin
Entity	ข้อมูลมี identity อะไร	Ticket, Action Record, User, Company, Software
State	ชีวิต object เปลี่ยนค่าใด	Requested, Approved, In Progress, Closed
Action	role ไหน trigger อะไร	CCE approve/close; programmer start/progress/close
Condition	เมื่อใดอนุญาต	CCE เฉพาะ Requested; closed รับ action ไม่ได้
Side effect	ต้องเกิดอะไรตามมา	latest action updates Last Status/Last Due Date

Input/output discipline: ทุกลูกศร/field ต้องย้อนหา requirement ได้. Added assumption แยกชื่อชัด; ห้ามเปลี่ยน Requested เป็น New, หรือเติม user role ที่ source ไม่กำหนดโดยเงียบ ๆ

4.10 Use Case: notation, direction และระดับรายละเอียด

Lecture Content | L4 หน้า 5–10,32–35,49–52. Actor อยู่ข้างนอก boundary; use case อยู่ข้างในและตั้งชื่อ action/goal. Association actor-to-use-case เป็น solid line ไม่มี arrow ตาม lecture. Include เป็น required subprocess: base → included. Extend เป็น conditional/optional behavior: extension → base. Generalization ใช้ unfilled triangle ชี้ parent; role ลูก inherit behavior ที่ source ให้ ไม่ใช่สร้าง permission inheritance ในโจทย์ WorkshopHub ที่ห้าม explicitly

ชิ้นงาน | Reasoning table สำหรับ link

Requirement meaning	Link	Why
ทุก checkout ต้อง process payment	Checkout include Payment	required part of completing checkout
ผู้ใช้เลือก apply discount เมื่อมีเงื่อนไข	Discount extend Checkout	base workflow ทำได้โดยไม่มี discount
CashPayment/ CardPayment เป็น variants ของ Payment	generalization → Payment	specialized forms ของ parent; ไม่ใช่ ordering
Cashier เริ่ม checkout	association	actor participates; ไม่ใช่ control-flow arrow

ข้อผิดพลาดที่พบบ่อย | Use Case ไม่ใช่ flowchart

Include/extend ไม่ใช้แทน “ทำอันนี้แล้วไปอันนั้น”. หากคำถามต้องการ sequence เวลา ให้ใช้ Activity/Sequence. ใน Service Ticket corrected example อาจารย์ให้ detail/create/edit เป็น extensions ของ View Tickets ตาม navigation ของตัวอย่างนี้; อย่านำ rule navigation นี้ไปอ้างว่า universal UML ทุกระบบต้องเหมือนกัน

4.11 Activity: branch, merge, loop และเจ้าของ action

Lecture Content | L4 หน้า 13–20,24–26,38–40,55–57. Workflow ต้องแสดง main/alternative/exception paths. Initial เป็น black dot; action เป็น rounded rectangle; decision เป็น diamond มี guards ที่อ่านแล้วแยกทางได้; merge รวม alternatives ก่อน action ถัดไป. Swimlane/partition บอกผู้รับผิดชอบ action ไม่ใช่ที่เก็บข้อมูล. เส้นข้าม lane หมายถึง flow เปลี่ยนผู้ทำ ไม่เท่ากับ API message รายละเอียด

ตัวอย่างพร้อมวิธีคิด | แตก POS checkout เป็น Activity ก่อนวาด

Lecture example. Customer/Cashier/POS/Payment Service/Printer/Inventory เป็น participants ตามระดับภาพ. Start → add items → show total → optional discount → choose cash/card → payment outcome. Cash รับเงินและทอน; card decline ต้อง retry/cash/cancel; success จึง finalization/receipt/inventory. Customer cancel ก่อน payment ต้องมี branch จบโดยไม่ทำ sale success side effects.
Decisions: payment method, card approved?, retry/switch/cancel? **Loops:** add more items หรือ retry payment.
Quality: ทุก guard ออกทางที่ชัด, ทุก path มี continuation/end; denied payment ไม่ลง successful receipt.
Assumption: E-wallet ที่ปรากฏใน workflow outline เป็น illustration; minimalist requirement ของ POS ที่กำหนด cash/card ไม่ได้อนุญาตเพิ่ม E-wallet โดยอัตโนมัติ

ผิด → rule → แก้: ถ้าภาพมี action “Customer pays” อยู่ใน POS lane ให้ย้ายการเลือก/ส่งเงินไป actor lane และคงการ validate/record ใน system lane. ถ้าสองทางเข้ากล่องเดียวจนอ่านไม่ชัด ให้เพิ่ม merge; ถ้าโจทย์ไม่ระบุ parallelism ไม่ใส่ fork เพื่อความสวย

4.12 Sequence: scenario หนึ่งเส้นทางต้องเดินได้ตั้งแต่ต้นจนจบ

Lecture Content | L4 หน้า 21–23,41–43,58–60. Participants เป็นแนวตั้งเรียงตาม responsibility; lifeline อ่านจากบนลงล่าง. Message มี sender/receiver/operation; return เมื่อแสดงควรเชื่อมกับ call. Scenario cash success ไม่ต้องรวมทุก possible choice ของ workflow; scenario card-declined-switch-cash ต้องแสดง failure แล้ว switch ตามโจทย์จริง

ชิ้นงาน | Scenario worksheet: CCE rejects Requested ticket

Precondition: CCE authenticated. **Participants:** CCE, UI, Ticket Service, DB.
1 Open list → query Requested → return list. 2 Open first ticket → fetch ticket+actions. 3 CCE chooses Close by CCE. 4 Service checks role/state and writes append-only Action Record. 5 Update Ticket Last Status/Last Due Date from latest action. 6 Commit and return saved state. 7 UI shows closed status/history.
Why: การแก้ ticket status อย่างเดียวขาด required action history; screenshot ของ closed text อย่างเดียวไม่พิสูจน์ latest-action consistency

Synchronous vs asynchronous | L4 หน้า 3: sync caller waits response; async caller continues แล้วรับ callback/notification ต่อมา. แสดง message และเวลาสอดคล้องชนิดงาน. Browser async function ไม่ได้แปลว่า business operation เป็น reliable asynchronous messaging. Kaching inventory ใช้ durable queue/outbox เพราะต้องรักษางานหลัง process failure

ข้อผิดพลาดที่พบบ่อย | Lifeline กับ swimlane ไม่ใช่คำเดียวกัน

โจทย์เรียก sequence “with swimlanes” ให้ใช้ provided participant columns เป็น lifelines และวาด messages ตาม scenario; Activity partitions มี actions/control flow. อย่าลาก control-flow ของ activity ไปเป็น sequence โดยไม่มี sender/receiver

4.13 State: model หนึ่ง object และ legal transitions

Lecture Content | L4 หน้า 45–46,61–62. State เป็นสภาพคงอยู่ เช่น Requested; action เป็น event เช่น approve. Guard บอกเงื่อนไขอนุญาต. ในภาพ lecture label bracket ใช้ actor เช่น approve [CCE]; อย่าเรียก actor bracket นั้นว่า boolean guard โดยไม่อธิบาย convention. Closed by CCE กับ Closed by Programmer เป็นคนละ state ที่ source ระบุ; “Closed” เดียวสูญเสียเหตุผลและผู้ปิด

ชิ้นงาน | Transition table ที่ใช้ตรวจ State และ authorization

From	Event / actor	To	Condition / side effect
Initial	save / System	Requested	initial Action Record; Last Due Date from Initial Due Date
Requested	approve / CCE	Approved	append action; update latest fields
Requested	close / CCE	Closed by CCE	append action; no new action after closure
Approved	start Work / Programmer	In Progress	notes/new due date in action
In Progress	add Progress / Programmer	In Progress	self-transition; update due date/history
In Progress	close / Programmer	Closed by Programmer	completion date; terminal

Source nuance: FR กล่าวว่า programmers add actions on Approved or In Progress; drawn status-change example shows start Work before close. หากโจทย์ถาม direct close from Approved ต้อง clarify หรือระบุ assumption; ไม่เติม arrow จนขัดโจทย์ status-only ที่ให้มา

4.14 Class: identity, multiplicity และ lifetime ownership

Lecture Content | Potential / Lecture-covered UML skill | L4 หน้า 11–12,36–37,53–54. Class มีชื่อ/attributes/types; association มี multiplicity ที่อ่านจากสองด้าน. Sale หนึ่งมี LineItem หลายรายการและเป็น composition เพราะ line อยู่กับ sale; Product ถูกอ้าง ไม่ถูกทำลายเมื่อ sale หาย จึงไม่ composition. Generalization triangle ชี้ parent: CashPayment/CardPayment → Payment, Cashier/Manager → User. Aggregation hollow diamond เป็น weak whole-part; lecture บอกไม่จำเป็นต้องใช้ในโจทย์ POS นี้

ตัวอย่างพร้อมวิธีคิด | อ่าน multiplicity โดยไม่ท่อง symbol

Explanation / Intuition. Registration header หนึ่งมี lines หลายรายการ; Draft เริ่มว่างจึง 0..* สมเหตุผลสำหรับ lifecycle รวม. ถ้าแสดง submitted-only view อาจ require 1..*. Line อ้าง course offering หนึ่งรายการ; offering มีหลาย registration lines จากนักศึกษาต่างคน. Diamond อยู่ด้าน owner header ไม่อยู่ที่ line. โจทย์ KU จำกัด immediate first-level links และไม่แสดง methods: ตอบครบหนึ่ง hop ดีกว่าใส่ domain ทั้งมหาวิทยาลัย

4.15 UI Prototype: List และ Form เป็นคนละการตัดสินใจ

Lecture Content | Exam-High | L3 หน้า 16–27; L4 หน้า 30–31,47–48,66–67,103–130. List ช่วยค้น/กรอง/เลือก record; Form ช่วยดูหรือแก้ record หนึ่ง พร้อม header และ line-item grid. Prototype ไม่ใช่แค่ลิสต์ชื่อ fields ต้องแสด navigation/actions/state/read-only/required/validation.

ชิ้นงาน | UI specification worksheet

Dimension	สิ่งที่ต้องเขียนและทดสอบ
Field source	login/profile/active semester/user edit/computed; ห้ามแก้ค่า identity ผ่าน form
Mode	view/edit/new; shading editable vs readonly; save/cancel controls
Validation	required leading red *; header errors separate row; line errors tooltip-style per lecture
LoV	reuse list for FK selection; sort/filter/date range/paging; selection returns key
Grid	named tab, row order, add above/below/delete, tooltips, numeric alignment
Responsive	header fields rearrange; line grid horizontal scroll; labels remain readable
State	loading/empty/success/error; dirty-bit warning before discard
Security	hide disallowed action for UX and enforce on backend; role/status context shown

เหตุผล: Hotel booking form ต้องเลือกหลาย rooms ต่อ room type; KU form ต้องหลาย course offerings และคำนวณ credits/fees. ใช้รูป generic form เดียวโดยไม่มี line items ทำ requirement สำคัญหาย

4.16 Agentic engineering, UI audit และ context handoff

Lecture Content | L4 หน้า 76–83,141–160. Vibe coding อาศัย chat fixes; agentic engineering มี living specs/tests, bounded work และ automated verification. Architect/coder/logic tester/UI auditor แยกหน้าที่ในตัวอย่าง; human supervisor review manifest และรับผิดชอบ blueprint. การเปลี่ยน Zen Green เป็น Business Blue พบ hard-coded alternating row color เป็นตัวอย่าง documentation/style-contract drift: แก้เป็น theme token ร่วมทั้ง list และ grid แล้วเก็บ evidence before/after

ชิ้นงาน | Context files ทำหน้าที่อะไร

File	Contract role ตาม L4 หน้า 156
AGENTS.md	process entrypoint/lifecycle/work norms/context handoff
docs/spec-core.md	system SDS/shared database/list/grid rules
docs/style-contract.md	theme/button/dialog/validation style gate
docs/testing-contract.md	STS catalog/test-authoring standards
feature template/spec	columns/lookups/rules per feature
construction and QA playbooks	bounded implementation/review steps
visual reference	exact grid/UI target to inspect

Gate: new session reads specs and current repo; end session updates context files. A remembered plan ไม่ใช่หลักฐานว่าไฟล์และ tests ปัจจุบันตรงกัน. Lab 2 staging integrated/UAT ผ่านก่อน main ตาม release path; implementation ที่พร้อมเฉพาะ feature branch ยังไม่เท่ากับ accepted increment

4.17 สรุปบทและฝึกตอบก่อนสอบ

แนวคิดที่ต้องอธิบายได้: เลือกภาพตามคำถาม: Use Case คือผู้ใช้กับเป้าหมาย Activity คือ Workflow Sequence คือ Scenario State คือ Lifecycle และ Class คือโครงสร้าง

สิ่งที่ต้องสร้างได้: UML, UI Prototype และข้อตกลงใน SDS/STS

ตัวอย่างพร้อมวิธีคิด | คำถามสั้นพร้อมเหตุผล

CCE ปิด Ticket ที่อยู่ Requested ต้องแสดงอะไรใน Sequence และ State?
แนวตอบ: Sequence ต้องมีการเปิดรายการ/รายละเอียด ตรวจ Role และ State บันทึก Action แล้วอัปเดต Ticket จาก Action ล่าสุด State เปลี่ยน Requested→Closed by CCE และไม่เปิดรับ Action ใหม่หลังปิด

เชื่อมแนวสอบ: Part 3: UML/UI ที่เลือก 30 คะแนน; Class ยังเป็นเนื้อหา Lecture

ตรวจความพร้อม: ลองอธิบายโดยไม่เปิดนิยาม แล้วสร้างชิ้นงานจากโจทย์ใหม่ ระบุสมมติฐานและวิธีตรวจผลให้ครบ

บทที่ 5

Security in Software Engineering

แกนบท: Security เป็นทั้งคุณภาพของ product และคุณภาพของวิธีสร้างระบบ. ออกแบบ trust boundary, identity, policy และ safe failure ตั้งแต่ requirement/design ไม่ใช่เพิ่มปุ่มซ่อนหลังเขียนเสร็จ

5.1 เริ่มจาก landscape

ก่อนเลือก control ให้ระบุ **asset** (ข้อมูล/บัญชี/บริการที่ต้องคุ้มครอง), **entrypoint** (login, API, upload, search, admin screen), **trust boundary** (browser → server, server → database) และผู้โจมตี/ความเสียหายที่เป็นไปได้. การคิดเป็น landscape ช่วยเชื่อม vulnerability กับตำแหน่งที่ต้องป้องกัน

5.2 Authentication กับ Authorization

	Authentication	Authorization
คำถาม	คุณเป็นใคร?	identity นี้ทำ action กับ object นี้ได้หรือไม่?
จุดทำงาน	login/session/token validation	policy check ณ request
Failure ตัวอย่าง	password ผิด, account inactive	Requester อ่าน ticket ของคนอื่น
หลักฐาน/ทดสอบ	login/session lifecycle, expiry/logout	role/object/action decision matrix

5.3 Identity lifecycle และ credential

บัญชีมี state เช่น Created → initial password → password changed → active use → inactive. Model ต้องคำนึงถึง setup, reset, disable และ recovery. Password ควรถูกเก็บเป็น proof ด้วย salted password hashing ที่เหมาะกับ password verification ไม่ใช่ plaintext หรือ reversible encryption. Encoding เปลี่ยน representation แต่ไม่ให้ protection; encryption ใช้ key และถอดกลับได้; hashing เป็น one-way transform.

ข้อผิดพลาดที่พบบ่อย | สับสน hashing กับ encryption

หากระบบต้องตรวจ password ไม่จำเป็นต้องถอด password กลับมา. ใช้ password hashing; encryption เหมาะกับข้อมูลที่ต้องกู้คืนโดยผู้มี key; encoding ไม่ใช่ security control.

5.4 Authentication methods และ session

Lecture เปรียบเทียบ Basic Auth, Digest, API keys, session-based login และ token patterns. Basic Auth ส่ง credential ทุก request จึงต้องใช้ HTTPS และมักไม่เหมาะกับ browser app สมัยใหม่; API key เหมือน secret ต้อง scope/เก็บให้ปลอดภัย; session login ให้ server จัดการ session และ browser ส่ง cookie. Token-based design ต้องระบุ expiry, renewal, revocation และ storage. การมี access/refresh token แยกหน้าที่ได้ แต่เพิ่ม lifecycle และ revoke complexity.

Safe failure message ไม่ควรบอกว่า email มีอยู่แต่ password ผิดจนช่วย attacker enumerate account. Session lifecycle ต้องระบุ login, use, expire, logout, blocked และสิ่งที่ server invalidate จริง.

5.5 Authorization model: RBAC / ABAC / ACL

Model	ตัดสินจากอะไร	ตัวอย่าง
RBAC	role ของ subject	Requester, IT Staff, Administrator
ABAC	attributes ของ user/resource/action/context	allow เมื่อ IT Staff และถูก assign ให้ ticket นี้
ACL	permission list บน resource	Ticket 123: Alex read/update, Bob read-only

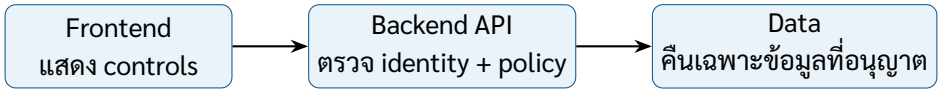
ชิ้นงาน | RBAC matrix เป็น design และ test input

Operation	Requester	IT Staff	Administrator
Create own ticket	Allow	Maybe	Maybe
View all ticket queue	Deny	Allow	Depends
Write internal note	Deny	Allow	Maybe
Create user account	Deny	Deny	Allow
Deactivate user	Deny	Deny	Allow + safety rules

คำว่า Maybe/Depends ใน source มีความหมาย: ต้องให้ stakeholder/policy ตัดสินก่อน implement. ห้ามแปลงเป็น Allow/deny เอง. ตรวจ workload และ permission combinations; role matrix ช่วยลดการตัดสินซ้ำ แต่การดู object ownership อาจต้องใช้ ABAC/object-level condition ร่วมด้วย.

5.6 Enforce policy ที่ backend

Frontend ซ่อน button เพื่อ usability ได้ แต่ไม่ enforce security เพราะ client แก้ได้และ request ปลอมได้. ทุก protected API ต้องตรวจ authentication และ authorization; test ต้องเรียก API โดยตรงด้วย role ที่ไม่ควรมีสิทธิ์. สำหรับ ownership อย่าเชื่อ requesterId จาก client; ใช้ authenticated identity และ filter record ตาม user ปัจจุบัน



อ่านภาพ: security boundary อยู่ที่ backend; UI เป็น affordance ไม่ใช่ gate. **OAuth 2.0** เน้น delegated authorization; **OIDC** เพิ่ม identity layer/id token; **SSO** ให้ sign in ครั้งเดียวเข้าหลาย application ตามระบบที่เชื่อมกัน.

5.7 Common vulnerability และ safe operations

ปัญหา	ความหมาย/ตัวอย่าง	แนวป้องกันหลักในขอบเขต lecture
Broken access control	อ่าน ticket ผู้อื่น/เรียก admin API	backend policy ทุก endpoint, object ownership test
Injection / SQLi	input เปลี่ยนความหมาย query/command	parameterized queries/ORM; allow-list structural choices
XSS	untrusted text ถูก execute เป็น script	encode output, sanitize rich text, avoid unsafe HTML
CSRF	browser แนบ session cookie กับ request ปลอม	CSRF token, SameSite cookies, origin checks
SSRF	server fetch URL ที่โจมตีชี้ไป internal resource	constrain destinations; validate trust boundary
Misconfiguration	debug exposure, weak cookies, broad CORS, open admin	safe defaults, configuration review
Vulnerable components	outdated dependency/runtime	inventory, update, review known risk

Input validation มีทั้ง business correctness (required, valid role, unique email) และ security boundary (type/length/size/allowed values). ทำ frontend validation เพื่อ UX และ backend validation เพื่อ security. สำหรับ SQL injection parameterization คือ primary defense; validation อย่างเดียวไม่พอ

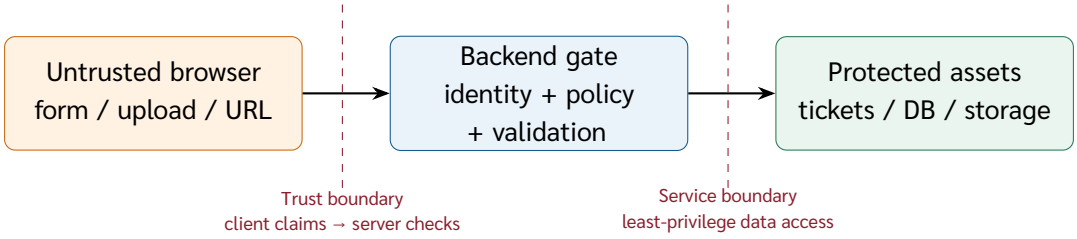
5.8 Security analysis pattern

1. ระบุ asset, actor, action, object และ trust boundary.
2. เขียน policy เป็น allow/deny พร้อม condition; ระบุ unresolved.
3. บังคับ policy ที่ server ทุกเส้นทาง.
4. เลือก test: unauthenticated, wrong role, wrong owner, invalid input, repeated request.
5. บันทึก evidence โดยไม่เปิดเผยข้อมูลลับใน error/log.

Exam Insight: Part 1 ให้สวมบท requirement/security analyst: แสดง role/policy และอธิบายเหตุผล รวม assumptions; “ปุ่มหาย” ไม่ใช่หลักฐาน authorization.

5.9 Security quality ตลอด SDLC: trust เปลี่ยนตรงไหน

Lecture Content | Exam-High | L5 หน้า 5–8,40. Confidentiality ป้องกันการอ่านที่ไม่ได้สิทธิ์; Integrity ป้องกันการแก้ที่ไม่ได้สิทธิ์; Availability ให้ legitimate users ใช้ได้เมื่อจำเป็น. Product quality ถามผู้ใช้เชื่อระบบได้หรือไม่; engineering quality ถามวิธีสร้างมีกฎ/ขอบเขต/test/safe default ที่เชื่อได้หรือไม่. Shortcut ที่ไม่เก็บเป็น risk/ติดตามจะกลายเป็น security debt



อ่านภาพ: browser data เป็น claim ไม่ใช่ proof; authenticated identity มาจาก session ที่ validate แล้ว. Backend database credential ก็ต้อง least privilege ไม่ใช่ถือว่าอยู่หลัง server แล้วปลอดภัยเสมอ. Actors รวม normal users/inactive users/insiders/buggy clients; assets รวม internal notes และ attachments นอกเหนือจาก passwords

5.10 Authentication: account และ session lifecycle

Lecture Content | L5 หน้า 9–17. Accounts มี states: created/initial password/password changed/active/inactive. Inactive ห้าม login แม้ password ถูก; change-password ต้อง verify current temporary password และ validate new password. Salt ต่อ password และ slow password hashing ลดผลจาก database leak; Base64 เป็น encoding ไม่ใช่ encryption; encryption recover ด้วย key; hashing one-way ใช้ verification

ชิ้นงาน | Methods ที่ lecture ใช้และสิ่งที่ต้องระวัง

Method	Mechanism	Security check
Basic Auth	Base64 credentials ทุก request	HTTPS; Base64 ไม่ปกป้อง password
Digest	nonce challenge/calculated response	legacy; configuration/HTTPS ยังต้องดู
API key	long secret สำหรับ client/service	protect/scope/rotate; ไม่ commit
Session	backend creates session; cookie attached	expiration/logout invalidation; secure cookies
Bearer token / JWT	code แนบ token header; server verify	token holder ใช้ได้จนหมดอายุ; expiry/revocation ต้องกำหนด
Access + refresh	short access; longer refresh renews	refresh protection/rotation/revocation ตาม design

Terminology note: lecture บางสไลด์ตั้งชื่อหัวข้อ authorization methods ขณะอธิบาย credential/session mechanism. ใช้ distinction หน้า 8/19 เป็นหลัก: พิสูจน์ identity คือ authentication; runtime policy ว่าทำอะไรได้คือ authorization. JWT ไม่ได้แปลว่าไม่มี server state ทุกสถานการณ์ โดยเฉพาะ revocation/refresh lifecycle

ตัวอย่างพร้อมวิธีคิด | Login และ logout test plan

Explanation / Intuition based on L5. ตรวจ valid active account, invalid credentials, inactive account, first-login password change, expiry และ direct protected request หลัง logout. Current-user endpoint คืน safe profile/access state ไม่คืน hash/token/secret. Generic failure ลด enumeration; internal log เก็บ failure type/correlation โดยไม่ log password. Repeated failure ใช้ rate-limit/slowdown ที่ specification กำหนด

5.11 Authorization: role ยังไม่ตอบ object และ workflow

Lecture Content | L5 หน้า 19–24. ตัดสินจาก Subject+Action+Object+Context. RBAC role permissions; ABAC subject/resource/context attributes; ACL permission list ต่อ resource. Role IT Staff มี update ticket ไม่ได้บอกว่าจะ update ticket ของใครได้และสถานะใด. Ownership/assignment/status/business rule ต้องตรวจเพิ่มเติม

ชิ้นงาน | Decision record: Requester A wants Ticket 202 of B

Subject: authenticated Requester A. **Action:** read. **Object:** Ticket 202 owner B. **Context:** ordinary requester session. **Rule:** requester reads own tickets. **Decision:** Deny. **Why:** role allows read capability แต่ object owner ไม่ตรง. **Verification:** direct API request denied without B's content/attachment or protected existence detail. **Implementation principle:** current identity from req.user; don't accept requester Id in body as authority

ตัวอย่างพร้อมวิธีคิด | WorkshopHub exact RBAC policy

Exam Blueprint | L10 หน้า 9. Student: view published, submit registration. Coordinator: view published, create workshops, edit workshop details, export participant lists. Exactly one role, ไม่มีการสืบทอดสิทธิ์, deny any ungranted operation. Therefore coordinator submit registration = Deny; student export = Deny. ไม่เติม admin role หรือ coordinator registration privilege เพราะดู "น่าจะได้". Cancellation จาก requirement exercise ไม่อยู่ใน policy exercise นี้: หากนำสอง exercises มารวมต้อง resolve policy gap ไม่ silently grant

OAuth/OIDC/SSO | L5 หน้า 24: OAuth2 delegated authorization เช่น grant Drive access; OIDC identity layer+ID token เช่น Sign in with Google; SSO sign in once ใช้หลาย apps. ไม่ถือ access token ทุกชนิดเป็น identity proof โดยอัตโนมัติ

5.12 Vulnerability walkthrough: mechanism → defense → negative test

Lecture Content | L5 หน้า 25–38. ชื่อ OWASP เป็น awareness framework ใน lecture ไม่ใช่ permission policy ของระบบ. ตารางนี้เก็บเฉพาะ vulnerabilities ที่ source สอน; test เป็น proposed verification ไม่ใช่ผลรันจริง

Vulnerability	Mechanism / asset	Defense	Verification
Broken access control	เปลี่ยน ID แล้วอ่าน ticket B; confidentiality	backend role+owner+state per endpoint	requester A directly calls B ID; deny no leak
SQLi	input becomes SQL syntax; records integrity/C	prepared values/ORM; allow-list structural columns	quote/ SQL-like input handled as data; no unintended rows
XSS	untrusted text rendered as executable HTML; session/ UI	output encoding; sanitize rich text; avoid unsafe HTML	save script-like note; display inert text per design
CSRF	browser auto-attaches valid cookie to forged action	CSRF token/SameSite/origin validation	request without valid proof rejected despite cookie
SSRF	server fetches attacker URL/ internal network; secrets	destination allow-list; block internal access	internal/ loopback URL rejected; no secret response
Misconfiguration	debug/ CORS/ cookie/ open services expose boundary	environment separation/ safe defaults	production error no stack; restricted origins/access
Vulnerable components	known dependency/runtime flaws	inventory/patch/review	check locked versions and update evidence

5.12.1 SQL injection: ทำไม validation อย่างเดียวไม่พอ

Unsafe SQL concatenation ใส่ user text ใน query grammar; input ที่ถูก escape ไม่ครบยังเปลี่ยนคำสั่ง. Parameterized query แยก plan+values จึงไม่ให้ value กลายเป็น syntax. Prisma query API เช่น findUnique where email ปลอดภัยในขอบเขตนั้น แต่ raw concatenated SQL ยังอันตราย. Sorting column names เป็น query structure จึงใช้ allow-list ไม่ใช่ bind arbitrary column string เป็น value

5.12.2 XSS, CSRF, SSRF ต่างกันที่ใครทำสิ่งใด

XSS ให้ browser execute attacker code เสมือนเว็บ trusted; CSRF ให้ browser ส่ง action ที่ใช้ session เดิม; SSRF ให้ server ส่ง request ตาม attacker URL และใช้ server network privilege. “validate input” ประโยคเดียวไม่แยก defense ได้: XSS ต้องดู output context; CSRF ดู request origin/proof; SSRF ดู destinations/network boundary

5.13 Upload scenario: defense ต้องปิดทั้งสามช่อง

Exam Blueprint | L10 หน้า 10. Student bypass browser file check, disguised script stored in executable folder, URL triggers execution and reads reports. Actor คือ student ที่ misuse; assets คือ reports/storage/runtime access; attack surface คือ upload endpoint และ served URL. Trust changes ที่ browser → server, uploaded data → server execution และ process → report storage

ตัวอย่างพร้อมวิธีคิด | คำตอบพร้อมเหตุผลและ verification

1. **Confidentiality affected:** source ระบุอ่าน reports ผู้อื่น. Integrity/Availability เป็น possible risk ให้ระบุเป็น inference ไม่ใช่เหตุการณ์ที่โจทย์ยืนยัน.
2. **Backend validation:** ตรวจประเภท/ขนาด/เนื้อหา ตาม allowed PDF policy ไม่พึ่ง browser extension.
3. **Storage/ execution:** keep uploaded data non-executable, controlled names/ paths และ least privilege; endpoint serving file ไม่ execute content.

4. **Authorization:** download checks owner/role; ไม่เปิด public raw folder URL ให้ทุกคน.
5. **Tests:** direct upload bypass UI, mismatched type, script content, other-user file ID และ execution attempt. Assert reject/no execution/no read-other-reports; log safe security event.

5.14 Safe operations: config/logs/secrets/recovery

Lecture Content | L5 หน้า 34–40. 400 invalid input, 401 unauthenticated, 403 forbidden, 404 missing/safe concealment, 409 conflict, 500 unexpected server error ต้องใช้ consistent contract. User error ให้แก้ได้แต่ไม่เผย stack/DB/secret/protected object. Developer log มี correlation/category/context; security signal เห็น repeats/forbidden/invalid tokens. Separate dev/test/production: test ไม่มี real secrets, production HTTPS/secure cookies/strict CORS/no stack/least privilege

ชิ้นงาน | Secure-by-default review

- Database/network/storage ไม่เปิดเกินจำเป็น; protected backups; app ไม่ใช้ DB superuser.
- .env อยู่นอก Git; .env.example ใช้ placeholders; leaked secret ต้อง rotate ไม่ใช่แค่ลบไฟล์ล่าสุด.
- Dependencies/runtime patched; rollback/incident notes; logs exclude passwords/tokens.
- Defense in depth, fail safe, simplicity, segmentation, usability และ minimized attack surface เป็น design choices ที่ต้องตรวจได้.

5.15 สรุปบทและฝึกตอบก่อนสอบ

แนวคิดที่ต้องอธิบายได้: Authentication ยืนยันตัวตน Authorization ตรวจสิทธิ์ต่อการกระทำและข้อมูล ต้องบังคับใช้ที่ Backend และรักษา Confidentiality, Integrity, Availability

สิ่งที่ต้องสร้างได้: ตาราง RBAC, การวิเคราะห์ภัยคุกคาม และกรณีทดสอบสิทธิ์

ตัวอย่างพร้อมวิธีคิด | คำถามสั้นพร้อมเหตุผล

Requester A เข้าสู่ระบบถูกต้อง แต่เปลี่ยน ID เพื่ออ่าน Ticket ของ B ผิดที่ใด?

แนวตอบ: Authentication ผ่าน แต่ Authorization ระดับข้อมูลผิด API ต้องตรวจเจ้าของด้วยตัวตนที่ยืนยันแล้ว ทดสอบทั้ง Ticket และไฟล์แนบของ B ว่าถูกปฏิเสธและไม่มีข้อมูลรั่ว

เชื่อมแนวสอบ: Part 1: Security; ใช้ประกอบการออกแบบทุกบท

ตรวจความพร้อม: ลองอธิบายโดยไม่เปิดนิยาม แล้วสร้างชิ้นงานจากโจทย์ใหม่ ระบุสมมติฐานและวิธีตรวจผลให้ครบ

บทที่ 6

Project Management: เริ่มโครงการ วางแผน และประมาณการ

แกนบท: Project Life Cycle บริหารโครงการ (Initiate → Plan → Execute → Close); SDLC ทำให้ software เปลี่ยนจาก need เป็น product. สองวงจรเกี่ยวกันแต่ไม่ใช่คำเดียวกัน

6.1 เริ่มด้วย value และ charter

Initiation ตอบว่าควรทำหรือไม่ ก่อนวางแผนละเอียด. Cost-benefit analysis แยก tangible gains/costs ที่คำนวณได้ออกจาก intangible ที่ต้องพิจารณาแต่ไม่ฝืนใส่ตัวเลข. ROI ใน lecture ใช้ $ROI = (G - C)/C$, โดย G คือ gain และ C คือ cost ในช่วงเวลาที่เทียบกัน

ตัวอย่างพร้อมวิธีคิด | ROI จากตัวอย่าง lecture

ต้นทุนตั้งต้น $6,000 และ $25 ต่อเดือน × 12 = $300; รวม cost $6,300. รายได้คาดการณ์ $10,000.

$$ROI = \frac{10{,}000 - 6{,}300}{6{,}300} = 0.5873 \approx 58.7\%.$$

Interpretation: gain สุทธิต่อ cost ประมาณ 58.7% ตามสมมติฐานตัวเลขในตัวอย่าง. ไม่ใช่กำไรสุทธิเท่ากับ ROI และไม่รวม intangible benefits เข้าเศษส่วนโดยไม่มีวิธีวัด.

Project charter รวมองค์ประกอบ initiation เช่น goals, scope, deliverables, success criteria, schedule, risk/ROI และการอนุมัติ. เป้าหมายที่ดีระบุ what, by how much, how และ deadline/ผู้รับประโยชน์ตามบริบท; SMART ช่วยตรวจ specific, measurable, attainable, relevant, time-bound.

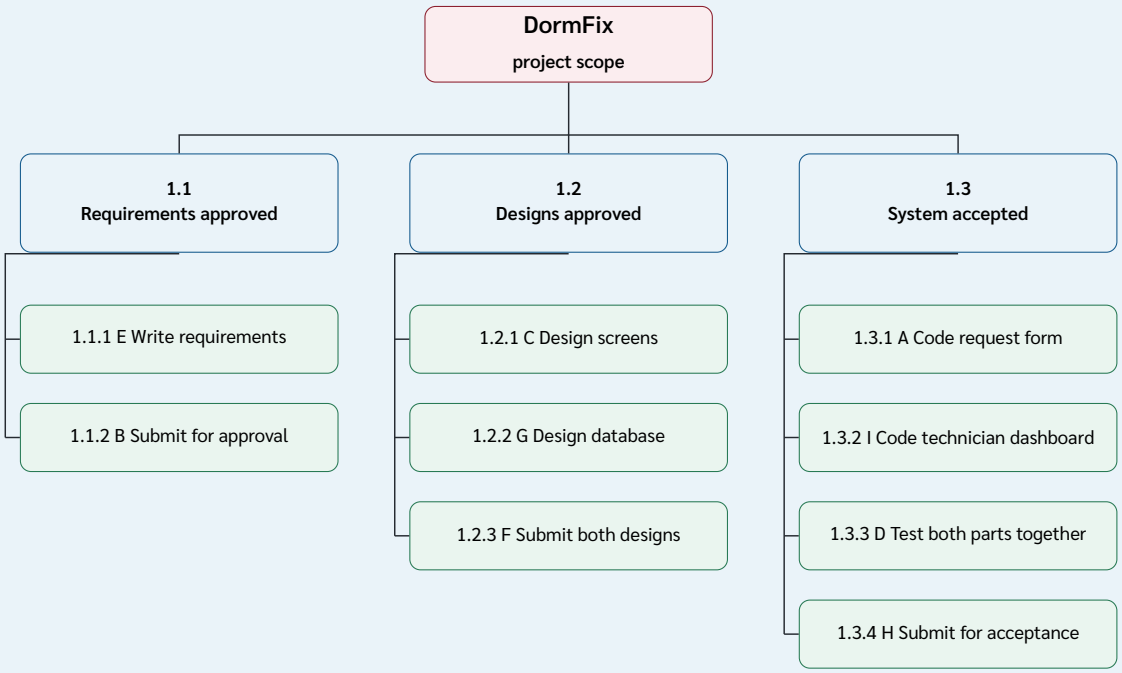
6.2 Goal, deliverable, milestone, task

คำ	หมายถึง	ตรวจว่าไม่สับสน
Goal	ผลลัพธ์/การเปลี่ยนแปลงที่ต้องการ	บอก impact หรือ measurable outcome
Deliverable	สิ่งที่ส่งมอบให้ลูกค้าหรือ project	เป็น output ที่มองเห็น/ตรวจรับได้
Milestone	จุดเหตุการณ์สำคัญ/approval/จบ phase	เป็น marker ไม่ใช่งานที่กิน working time
Task	activity ที่ต้องทำเพื่อสร้าง output	มี owner, effort/duration และ dependency

6.3 WBS: แตก scope เป็น hierarchy

เริ่มจาก deliverable ที่ต้องตรวจรับ แล้วแตกลงเป็น milestone และ task. ภาพนี้ใช้ลำดับชั้นเดียวกับตัวอย่าง DormFix ใน Lecture 10: task letter ทุกตัวอยู่ใต้ milestone ที่มันช่วยให้สำเร็จ.

ชิ้นงาน | WBS tree: DormFix



เส้นเชื่อมแสดง parent–child ในโครงสร้างงาน; ภาพนี้ยังไม่ใช่กำหนดการ. ลำดับ C/G และ A/I จัดสลับกันได้เมื่อเป็นงานขนาน. ใช้ WBS ID เดิมเมื่อต่อไปทำ RACI และ Gantt เพื่อให้ตรวจย้อนกลับได้. ช่อง milestone 1.4 ในแบบฝึกของอาจารย์เว้นว่าง เพราะมีงานครบในสาม milestone แล้ว.

6.4 Effort, duration, dependency และ critical path

Effort คือปริมาณงาน; duration คือเวลาปฏิทินที่ผ่านไป. คนสองคนอาจลด duration ได้ในบาง task แต่ไม่ทำให้ effort หาย. Dependency ระบุ predecessor; parallelism ทำได้เมื่อเงื่อนไขก่อนหน้าอนุญาต. **Critical path** คือ path ที่กำหนดระยะเวลารวมต่ำสุดของโครงการภายใต้ dependency/duration; delay บน path เลื่อน finish โดยตรง. **Float** คือเวลาที่กิจกรรมเลื่อนได้ก่อนกระทบ milestone/finish ตามนิยาม schedule.

6.5 Three-point / PERT estimation

ให้ O optimistic, M most likely, P pessimistic. Estimate expected effort และความไม่แน่นอน:

$$E = \frac{O + 4M + P}{6}, \qquad SD = \frac{P - O}{6}.$$

ตัวอย่างพร้อมวิธีคิด | TokTickIT: estimate แล้วสื่อเป็น range

Task	O	M	P	E
Role guards/ownership	2	3	6	3.33
Queue API search/filter/sort/page	3	5	10	5.50
Claim/reassign/priority/status	4	6	11	6.50
Comments and internal notes	4	7	13	7.50
Admin users	3	5	12	5.83
Authorization/regression/E2E tests	4	6	14	7.00
Total	20	32	66	35.67

รายงานเป็นช่วง: **35.7 h ± 3.2 h**. การเขียน tests อาจลด estimate; integration และ debugging อาจเพิ่ม estimate.

6.6 RACI: responsibility without guessing

Role	หน้าที่	คำถามตรวจ
R - Responsible	ทำงาน	ใครลงมือ?
A - Accountable	ทำให้แน่ใจว่างานเสร็จ/รับผิดชอบ outcome	ใครรับผิดชอบสุดท้าย?
C - Consulted	ให้ feedback ก่อนงานเสร็จ	ใครต้องให้ข้อมูลสองทาง?
I - Informed	รับทราบหลังเสร็จ	ใครต้องได้รับ update?

ใช้ RACI เมื่อ coordination ซับซ้อนพอ; ดู R distribution ว่ามีคน overload หรือไม่. ถ้า prompt มีเพียงรายชื่อทีมแต่ไม่มี meeting notes/role evidence ให้สอนกฎและทำเครื่องหมาย unknown; ห้าม invent assignment เพื่อให้ตารางเต็ม.

6.7 Risk และ buffers

Risk ระบุ event/condition, likelihood/impact และ response ตามข้อมูลที่มี. Project buffer คือ pool ท้ายโครงการ; task buffer เหมาะเมื่อ delay อยู่นอกการควบคุมทีม เช่น approval จาก stakeholder. อย่าใส่ buffer โดยไม่อธิบายเหตุผล และอย่าซ่อน buffer ใน task duration จนอ่านไม่ออก

6.8 Gantt: คำนวณก่อนวาด

Workflow: parse schedule wording → dependency → parallelism → justified buffers → dates → chart → explanation. Gantt แสดง task span, dependency, buffer และ milestone. Milestone เป็น point บนวันสุดท้ายที่ mark ไม่ใช่ working task. ถ้าโจทย์ให้ weekdays only, start date และ buffer rules ต้องใช้ให้ครบ

Exam Blueprint

DormFix schedule facts: start Mon 4 Jan 2027; weekdays only; successor เริ่มหลัง dependency + applicable buffer; task buffer 1 วันเมื่อจำเป็น; project buffer 2 วัน; milestone เป็นจุดบนวันสุดท้าย. worked result วาง E Jan 4–6, B Jan 7 + external buffer Jan 8, C/G parallel Jan 11, A/I parallel Jan 18, project buffer Jan 28–29, final milestone Jan 29. ตรวจ calendar และ dependency กับ source ก่อนใช้งาน.

อธิบาย Gantt ทุกครั้ง

Why dependency? อะไรต้องได้รับอนุมัติก่อน? **Why buffer?** delay อยู่ใน/นอกการควบคุมทีม?
Assumption? วันหยุด, inclusive date, parallel capacity หรือ duration ที่โจทย์ไม่ระบุคืออะไร?
Chart อย่างเดียวไม่แสดงเหตุผลของ schedule; คำอธิบายเป็นส่วนของคำตอบ.

6.9 Initiation: ทำไมโครงการนี้จึงควรถูกอนุมัติ

Lecture Content | Core-SE | L6 หน้า 2–33. Project Life Cycle คือ Initiate/ Plan/ Execute/ Close; SDLC คือ กิจกรรม requirements/design/code/test/release/maintain ภายในงาน software. Initiation ต้องตกลง Goals, Scope, Deliverables, Success Criteria, Stakeholders, Resources ก่อน charter approval แล้ววางแผนละเอียด. Sponsor อนุมัติ/สนับสนุน, PM ถือแผน, team ทำงาน, customers ซื้อ/รับประโยชน์ และ users ใช้งานจริง อาจเป็นคนละกลุ่ม

ชิ้นงาน | SMART กับ OKR ใช้ต่างกันอย่างไร

Framework	Structure	ตัวอย่างจาก lecture
SMART	specific/measurable/attainable/ relevant/time-bound	ลด median first-response 20% this semester ด้วย automatic routing
OKR	objective + measurable key results; stretch goals	queue manageable; 90% routed, median under 4h, no ticket older 10d

SMART ตรวจ target practical; OKR key results deliberately ambitious ตาม source ไม่ได้หมายความว่า forecast ทุกอย่างต้องใช้ stretch estimate. Organisation/department/project OKRs align ขึ้นด้านบน. Vanity metric เช่น hits มาก ไม่บอกว่าจะปรับอะไร; metric ที่ดีตอบ goal และมี benchmark. Attainable ต้องมีเหตุผลว่าทีมทำได้ ไม่ใช่แค่เขียนคำว่า achievable

ตัวอย่างพร้อมวิธีคิด | Goal → deliverable → success criteria

Lecture example + Explanation. Goal ลด response time; deliverable routing rules/assignment engine/queue dashboard. Success criteria ระบุผลและมาตรฐานที่ตกลงไว้ก่อนส่งมอบ. Baseline ใช้ median hours จากช่วงเดิม; target 20% ต้องระบุช่วงเปรียบเทียบและ measurement method. ส่ง dashboard ได้เป็น deliverable แต่ไม่ได้พิสูจน์ response time ลดแล้ว: launch ส่งระบบ, landing วัด impact

6.10 Cost-benefit และ charter ต้องเขียน assumptions ของตัวเลข

Lecture Content | L6 หน้า 7–9,59–65. Tangible gains/costs คำนวณ ROI; intangible retention/morale/brand ใช้ประกอบ decision ไม่บังคับใส่เลข. Cost รวม people/one-time/ongoing/long-term ในช่วงเดียวกับ gains. Example ROI 58.7% เป็นกรณีตัวเลขใน lecture ไม่ใช่ universal approval threshold; reported “strong” ใน example ยังต้องเทียบ risk/feasibility/organisation policy

ชิ้นงาน | Charter outline ที่ตอบสี่คำถาม

Why? summary/goals/objectives/business case/benefit-cost.
What? scope/deliverables/success criteria/OKRs.
What will it take? team/budget/schedule/milestones/resources.
What could go wrong? constraints/assumptions/risks/approvals.

Small project ใช้ short charter; complex project link analysis appendix. เป็น living document ที่ update โดยยังเก็บ approved baseline/decision

ตัวอย่างพร้อมวิธีคิด | TokTickIT Lab 3 miniature charter

Explanation / Intuition anchored L6 หน้า 13,40,80,115. Goal server-enforced role+ownership และ IT Staff shared queue. In scope role guards, search/filter/sort/page, claim/reassign/priority/status, public comments/internal notes, admin users, verification. Out scope email/SSO/self-registration/Actions Taken/dashboard ใน sprint นี้. Deliverable accepted staging → main vertical slice พร้อม specification/tests/UI/API/reviewer/AI-use docs. Risk integration/env/user permissions. Success evidence direct unauthorized API denied และ E2E staff queue task completed; deadline/resources ต้องใช้ค่าที่ charter จริงอนุมัติ ไม่ invent

6.11 Scope creep, fixed corner และ stakeholder grid

Lecture Content | L6 หน้า 34–58. Scope เป็น agreed understanding รวม in/ out, audience, complexity และ timeline/budget/resources. Internal creep มาจากทีมเพิ่ม improvement; external creep มาจาก customer/business/ technology. Change request ไม่ใช่คำสั่งอัตโนมัติ: record indirect costs, schedule/resource impact และทางเลือก. ถ้า time fixed ให้ scope/cost trade-off; ถ้า behavior fixed ต้อง time/cost trade-off ที่ผู้มีอำนาจยอมรับ

ชิ้นงาน | Power-interest grid → engagement plan

Influence / interest	Strategy	Evidence/communication
High / High	manage closely	regular decisions/reviews; requirements approved
High / Low	meet needs	periodic briefings; approval evidence concise
Low / High	show consideration	progress/risk/update; meaningful user feedback
Low / Low	monitor	minimal engagement until affected

Stakeholder analysis: list affected/contributors/invested → interest/influence → ability to participate/how involve. Higher influence ไม่ใช่เหตุผลลบ user needs; ใช้จัด communication และ decision authority. TokTickIT worked grid ให้ instructor/IT Staff manage closely, requester/peer reviewer receive progress/review evidence ตามบทบาท

6.12 Organisation ส่งผลต่อ design และ delivery

Lecture Content | L6 หน้า 67–71,114. Traditional reporting single chain อาจทำ PM ต้องยืม authority; matrix มีหลาย chains/autonomy/shared resources และ divided loyalty. Conway's Law อธิบาย organisation structure สะท้อน system boundaries; business-area end-to-end teams ลดการรอข้าม layers. Wall of Confusion คือ Dev วัด change แต่ Ops วัด stability: Scrum ด้านเดียวไม่แก้ release bottleneck. Brooks quotation เตือน manpower เพิ่มใน late project อาจเพิ่ม coordination/onboarding แทนลดเวลา

6.13 WBS / RACI / Gantt: ต่อกันด้วย IDs ไม่ใช่รูปเหมือนกัน

Lecture Content | Exam-High | L6 หน้า 71–85,95–96. WBS hierarchy milestone/tasks; RACI responsibility; Gantt dates/duration/dependency. Break top-down หรือ roll-up bottom-up ได้ แต่ทุก task ต้องอยู่ใน scope และเข้าถึง milestone. Milestone เป็น point ไม่มี workload; task เป็น activity มี duration/owner. ใช้ stakeholder-review test ช่วยเลือก significant milestone ไม่ตั้งทุก coding task เป็น milestone

ตัวอย่างพร้อมวิธีคิด | RACI มีคำบอกบทบาทแล้วจึงใส่อักษร

Explanation / Intuition; ไม่ใช่เฉลย DormFix RACI ที่ไม่มี meeting notes. สมมติ meeting notes เขียน "BA drafts requirements, PM is accountable for completion, sponsor provides feedback before submission, designer receives approved document." Draft task row: BA=R, PM=A, sponsor=C, designer=I. ถ้าไม่มีบทบาทเขียน –; อ่าน workload Rs ของแต่ละคนเพื่อดู overload. A ไม่เท่ากับคนที่ลงมือและ C ไม่เท่ากับผู้รับข่าวหลังเสร็จ

6.14 Critical path: คำนวณ forward/backward แล้วอธิบาย float

Explanation / Intuition to expand L6 หน้า 81–85. ตัวอย่างใหม่ใช้หน่วย working days, task start-after predecessor finishes, ไม่มี resource conflict. A=2; B=3 หลัง A; C=2 หลัง A; D=2 หลัง B,C. กำหนด ES=0 และ EF=ES+duration เพื่อหลีกเลี่ยง inclusive date confusion

ชิ้นงาน | Worked CPM table

Task	Predecessor	ES	EF	LS	LF	Float
A (2d)	–	0	2	0	2	0
B (3d)	A	2	5	2	5	0
C (2d)	A	2	4	3	5	1
D (2d)	B,C	5	7	5	7	0

Forward: max predecessor EF ให้ D starts5. Backward จาก finish7: D LS5; B LF5/LS2; C LF5/LS3; A LF=min(2,3)=2. Float=LS-ES. Critical A → B → D=7d; C มี float1 ไม่ใช่ task ที่ละได้. CPM longest path ไม่ใช่เลือก task duration ยาวสุดเพียงชิ้นเดียว. ถ้าคนเดียวต้องทำ B และ C พร้อมกัน schedule ต้อง resource-level เพิ่ม ไม่ใช้ dependency-only answer เป็น final

6.15 PERT uncertainty: รวม variance ไม่รวม SD ตรง ๆ

Lecture Content numeric data | L6 หน้า 90–92; Explanation for derivation. Expected total จากหก tasks =35.6667h. SD แต่ละ task: 0.667,1.167,1.167,1.5,1.5,1.667h. ภายใต้ assumption estimates independent:

$$\sigma_{total} = \sqrt{\sum_i \left(\frac{P_i - O_i}{6}\right)^2} = \sqrt{10.4444} \approx 3.232h.$$

ดังนั้นหนึ่ง sigma 35.7±3.2h คือ 32.5–38.9h; สอง sigma upper $35.6667 + 2(3.2318) = 42.13h$ จึง source รายงาน plan around42h. ไม่ใช่ guaranteed completion probability เมื่อ independence/distribution ไม่รองรับ. Effort hours ไม่ใช่ calendar duration; parallel capacity/dependencies/working days ยังต้องใช้คำนวณ schedule

ข้อผิดพลาดที่พบบ่อย | ใช้ AI แล้วไม่ลบเวลาตรวจ

L6 หน้า 93: typing/tests ลด แต่ diff review/integration/debugging เพิ่ม. อย่า estimate จาก typing time อย่างเดียว. Planning fallacy นึกแต่ happy path; buffer ใช้รับ risk ที่ระบุ ไม่ใช่ซ่อน task duration

6.16 DormFix schedule แบบครบ: dependency → dates → Gantt

Exam Blueprint | L10 หน้า 14–21. E → B → (C,G) → F → (A,I) → D → H. C/G และ A/I parallel เพราะ prerequisites พร้อมและ owners ต่างกัน. F รอทั้ง designs; D รอทั้ง coded parts และ Dao ผู้ทำ I พร้อมทดสอบ. B/F/H เป็น external approval จึง task buffer1d; team tasks ใช้ pooled project buffer2d

ชิ้นงาน | ทุก task พร้อม owner/duration/date/buffer

Task	Owner	Days	Work dates (Jan)	Buffer	Due marked
E	Aom	3	4–6	–	6
B	PM	1	7	8	8
C	Bank	3	11–13	–	13
G	Chai	2	11–12	–	12
F	PM	1	14	15	15
A	Chai	4	18–21	–	21
I	Dao	3	18–20	–	20
D	Dao	2	22,25	–	25
H	PM	1	26	27	27
Project buffer	–	2	–	28–29	29

Calendar: weekends 9–10,16–17,23–24 ไม่เป็น work days. Marked Due รวม task buffer ตาม exam template; ไม่แปลว่า approval task ทำงานสองวัน. Milestones Requirements approved=8, Designs approved=15, System accepted=29

ตาม source rule last project buffer. คำอธิบายต้องบอก external checks อาจช้า และ milestones ใช้ * ไม่ใช้ working tick. Full drawn chart อยู่ visual atlas บท 9.

6.17 Risk/resources/procurement/communication/kick-off

Lecture Content | L6 หน้า 98–113. Budget planning แตก tasks → cost estimates → sum → contingency/tax → approval. Procurement source/quote/choose/negotiate/deadlines/evaluate/pay; vendor risk ไม่จบเมื่อ hire ต้อง NDA/RFP(TOR)/SOW/contract และ check-ins ตาม scope. Agile procurement collaborative living contract; traditional fixed terms/extensive docs. Lecture ระบุ budget/communication/procurement ไม่ใช่ส่วนลงลึกของ course แต่ยังต้องเข้าใจ purpose

ชิ้นงาน | Risk register pattern ที่มี owner และการติดตาม

Event	Likelihood/ impact	Owner/response	Monitor
Staging environment breaks	Medium/High	Nun; tested Docker Compose per source	weekly setup run
Teammate unavailable	Low/High	Ploy; pair on T2 per source	coverage/availability review
Integration late	high-risk pattern	agree early staging integration + tests	backlog/dependency/PR age

Risk process identify → analyse → prioritise → owner → respond → monitor. Mitigation ต้องลด likelihood/impact จริง; “be careful” ไม่ใช่ response. Kick-off ชวน team/stakeholders/sponsor ให้ vision/scope/roles/expectations ตรงกัน; agenda introductions, purpose, goals/scope, roles, tools/process, next actions, questions. Designated notetaker เก็บ decision/action owner แล้วส่ง summary; meeting จบโดยไม่มี follow-up ไม่ปิด coordination risk

6.18 สรุปบทและฝึกตอบก่อนสอบ

แนวคิดที่ต้องอธิบายได้: อนุมัติคุณค่าและขอบเขตก่อนแตกงาน WBS แสดงลำดับชั้น RACI แสดงความรับผิดชอบ Gantt แสดงวันและ Dependency ส่วน PERT แสดงค่าประมาณกับความไม่แน่นอน

สิ่งที่ต้องสร้างได้: Charter, WBS, RACI, Gantt และทะเบียนความเสี่ยง

ตัวอย่างพร้อมวิธีคิด | คำถามสั้นพร้อมเหตุผล

งาน D ของ DormFix เริ่ม 21 มกราคมได้หรือไม่?
แนวตอบ: ไม่ได้ เพราะ A ยังทำงานในวันที่ 21 และ D ต้องรอทั้ง A กับ I เสร็จ D จึงเริ่มวันทำงานถัดไป คือ 22 มกราคม ทดสอบสองวันเป็น 22 และ 25 โดยข้ามวันหยุด

เชื่อมแนวสอบ: Part 2: WBS 3, RACI 2, Gantt 5 คะแนน

ตรวจความพร้อม: ลองอธิบายโดยไม่เปิดนิยาม แล้วสร้างชิ้นงานจากโจทย์ใหม่ ระบุสมมติฐานและวิธีตรวจผลให้ครบ

บทที่ 7

Project Execution: backlog, Scrum, delivery และ Git workflow

แกนบท: execution เปลี่ยนแผนเป็น increment พร้อมติดตาม deviation, feedback และ evidence. Backlog เป็นลำดับงานที่ยังเปลี่ยนได้; Sprint Backlog เป็นงานที่ทีมรับเข้ารอบ; Done increment คือผลที่เสร็จตามเกณฑ์

7.1 Requirements gathering และ priority

Lecture 7 แสดงขั้น: identify stakeholders → establish goals/objectives → elicit → document → analyze/confirm → prioritize. Elicit ไม่ใช่ประชุมอย่างเดียว; ใช้ถาม สังเกต และอ่านเอกสารเดิม. Document ควรรักษาคำของ user ก่อนแปลเป็น system language. MoSCoW ช่วยจัด Must/Should/Could/Won't (this time); priority ต้องผูกกับ goal/constraint ไม่ใช่ความดังของผู้ขอ

7.2 URS, SysRS, SRS และ design

Document	หน้าที่	ตัวอย่าง
URS	business need และ expectation ของ end user; อาจเป็น user stories/use cases; validate ด้วย UAT	"Requester ต้องสร้างและดู ticket ของตนเอง"
SysRS	system requirements, scope, constraints, assumptions/dependencies	server ต้องอยู่ใน Thailand (constraint)
SRS	agreed software behavior/requirements ที่ใช้เป็นฐานออกแบบ/test	FR/NFR/BR รายละเอียดที่ implement ได้
SDS	architecture, module boundary, interface	component/API contract

Lecture เตือนว่า server location เป็น system constraint จึงอยู่ SysRS ไม่ใช่ functional SRS behavior. Technical lead แตก requirement เป็น modules ที่มี boundary ชัดก่อนพัฒนา parallel.

7.3 Testing, deployment และ maintenance

Unit → integration → system/E2E → acceptance เป็นระดับที่ต่อยอดกัน. System test ครอบคลุมระบบประกอบตาม SRS และ NFR; acceptance ใช้ผู้ขอ/ผู้ใช้ยืนยัน business need. Deploy ทีละ environment ด้วย gates: tested code → UAT → production. Maintenance รับงานจาก user, environment, defect และ change; เอกสาร product มี requirements, design, technical/code, QA/test และ user documents. SOP ช่วยงานซับซ้อนซ้ำได้ด้วยขั้นตอน/flowchart

7.4 Tracking plan และ roadmap

Tracking เปรียบเทียบ plan กับ actual: schedule, milestone progress, cost, decisions, changes, dependencies, risks. Deviation อาจเป็นบวกหรือลบ จึงต้องบันทึกผลและปรับแผน. Gantt เหมาะแสดง task/date/dependency; roadmap ระดับสูงสื่อ goals/milestones ให้ stakeholders แต่ไม่กำหนดว่าใครทำอะไรในวันใด. Burndown แสดง story points remaining เทียบเวลาใน sprint.

7.5 Product backlog และ workflow

Board จาก lecture: New Issues → Icebox → Product Backlog → Sprint Backlog → In Progress → Review/QA → Done. Card โดยปกติเดินไปขวา ยกเว้นกลับไป backlog เมื่อยังไม่ทำ/ต้องปรับ. Product backlog เรียง importance/business value; ด้านบนละเอียดกว่า ด้านล่างอาจเป็น idea บรรทัดเดียว.

7.6 Backlog refinement / grooming

เป้าหมาย: rank, split, clarify และทำให้ story พร้อมเข้า sprint. Lecture แนะนำ refine ทุก sprint, มีอย่างน้อยสอง sprint ที่ groomed และยิ่ง refine ดี sprint planning ยิ่งง่าย.

ชิ้นงาน | User story 3 blocks

Story: As a [role], I need [capability], so that [value].
Assumptions / Details: facts, rules, dependency, constraint; unknown แยกให้เห็น.
Acceptance Criteria: Given [context], When [action], Then [observable result].
Metadata: Assignee / Label / Milestone / Estimate; ช่องที่ไม่มีข้อมูลให้เขียน “-” ตาม exam sample.

ตัวอย่างพร้อมวิธีคิด | Badminton booking: raw request → story

Story: As a student, I need to book a badminton court for one hour, so that I do not have to queue at the counter.
Assumptions and Details: Courts/opening hours are already stored; one booking per day.
AC: Given Court 2 is free at 17:00; When I book it; Then Court 2 at 17:00 is shown as “Booked”.
Label: enhancement.
เหตุผล: story ระบุ role/capability/value; details เก็บ rule; AC บอก state ที่ตรวจได้. ห้ามย้าย assumption ไปปนเป็น source fact.

7.7 Scrum roles, artifacts, events

องค์ประกอบ	หน้าที่	ผลที่ต้องจำ
Product Owner	จัดอันดับ backlog, สื่อ sprint goal และ value	ตัดสินลำดับ product
Scrum Master	เอื้อ meeting/process และช่วยทีมทำงานได้	ไม่ใช่ project manager ที่สั่ง task
Development Team	ประเมิน capacity/เลือกวิธีส่งมอบ	autonomy ต่อวิธีทำงาน
Product Backlog	stories ทั้งหมดที่ยังไม่อยู่ sprint	ranked; refinement ต่อเนื่อง
Sprint Backlog	stories ที่เลือกสำหรับ sprint + how	commitment/forecast ของรอบ
Done increment	ของที่เสร็จตาม Done	inspect ได้

Events ตาม lecture: backlog refinement ก่อน sprint, sprint planning ตอนเริ่ม, daily standup ระหว่าง sprint, sprint review เพื่อ demo increment, retrospective หลัง review. Sprint planning กำหนด “what can be delivered” and “how”; outcome คือ Sprint Backlog. PO นำ goal/ranked backlog; Scrum Master facilitate; team ตัดสินว่าอะไร fit.

7.8 Sprint Planning: goal ก่อน capacity

ขั้นตาม lecture: ดึง stories จากบน backlog → assign story points/labels → ตรวจว่า story มีข้อมูลเริ่มทำได้ → หยุดเพิ่มเมื่อถึง velocity. Velocity คือ points ที่ทีมทำเสร็จได้ใน sprint; เปลี่ยนได้และ unique ต่อทีม เพราะการให้ points ต่างกัน

ตัวอย่างพร้อมวิธีคิด | ShuttleNow: goal + capacity

Goal: ให้ นักศึกษา ติดตาม shuttle และ รู้ เวลา ถึง. Velocity 16; anchor story 3 points.

Story	Points	Select?	Reason
Live position	8	Yes	supports goal
QR fare	3	No	outside goal
Arrival at my stop	5	Yes	supports goal
Alert before arrival	3	Yes	supports goal
Arrival all stops	5	No	would exceed capacity after relevant items
Mobile route-list bug	2	No	not selected for this goal

รวม 8+5+3=16. QR fare fit point budget แต่ไม่สนับสนุน goal จึงไม่เลือก. Capacity ไม่ใช่ criterion เดียว; goal fit และ readiness ก็สำคัญ.

7.9 End-of-sprint: unfinished work ตาม lecture

Course-specific procedure ของ lecture ให้ credit งานที่ทำแล้วและแยก remaining work เพื่อให้ velocity ถูกต้อง:

1. ปรับ description/points ของ story ที่ unfinished, ใส่ label unfinished และย้าย story ส่วนนั้นไป Done เพื่อให้เครดิตงานที่เสร็จ.
2. สร้าง story ใหม่สำหรับ remaining work.
3. assign remaining points แล้วนำ story ใหม่ไป sprint ถัดไป.

Untouched stories กลับขึ้นบน Product Backlog และ unassign จาก sprint milestone; อย่าย้ายทั้ง story ที่ยังไม่ทำไป sprint ใหม่. ปิด stories/milestone เดิมและสร้าง sprint milestone ใหม่.

Course-specific Scrum rule

ทำตามขั้นตอนที่ lecture สอน แม้จะต่างจากคำอธิบาย Scrum ทั่วไปจากที่อื่น. ในข้อสอบยึด primary course source.

7.10 Git: อ่าน repository graph ก่อนพิมพ์ command

Commit คือ node/snapshot ที่เชื่อม parent; branch คือ pointer ที่ขยับตาม commit; HEAD ชี้บริบท checkout ปัจจุบัน (มักชี้ branch). การอธิบาย graph ทุกครั้ง: Before graph → interpret HEAD/branch → command/transaction → pointer/commit effect → After graph → Why.

Detailed practice: ดูบท Git Worked Transactions สำหรับภาพกราฟจริง แยก commit nodes, branch refs, HEAD และ remote tracking พร้อม Before/Command/After/Why.

สรุปบท 7

Execution ต้องติดตาม deviation และ evidence. Refinement ทำ story sprint-ready; Sprint Planning เริ่มจาก goal, readiness และ velocity. Course lecture มี end-of-sprint procedure เฉพาะ. Git ตอบด้วย graph ก่อน/หลังและเหตุผลของ pointer.

7.11 Execution คุม feedback, documentation และ actual progress

Lecture Content | L7 หน้า 3–19. Gathering requirements วนหกขั้นและ priority ใช้ MoSCoW; URS เน้น business/end-user intent, SysRS system constraints, SRS software behavior. Design document/module boundary ทำ parallel development เป็นไปได้. Testing levels/type เป็นสองแกน: unit/integration/system/acceptance คือ scope; functional/NFR/regression คือคำถามที่ตรวจ. Deploy tested code → UAT → production; maintenance มี bugs/enhancements/requirements/UI issues

ชิ้นงาน | Document map พร้อมตัวอย่าง use

Category	เนื้อหา	ใช้เมื่อ
Requirements	URS/SRS/SysRS	scope/acceptance/impact analysis
Design	architecture/module boundaries	implementation/integration decisions
Technical	code comments/working papers	maintenance/debug/explain how
QA	test plans/data/scenarios/cases/strategy/trace	execute/verify/regression
User + SOP	FAQ/install/help/tutorial/manual; repeatable complex tasks	operate/support/recovery

Tracking เปรียบ schedule/action/task/milestone/cost/decisions/changes/dependencies/risks กับ plan; deviation บวกหรือลบได้. Roadmap สื่อ goals/milestones แบบ stakeholder view; Gantt บอก who/when; burndown บอก remaining points. อย่าใช้ roadmap high-level แทน schedule ที่โจทย์ถาม dependency/date

7.12 Agile practices และองค์กรที่ทำให้ delivery ติด

Lecture Content | Core-SE | L7 หน้า 21–58. Agile iterative/collaborative/cross-functional/adaptive/evolutionary/early delivery/continual improvement. *Extreme Programming (XP)* เป็นแนวทางเชิงปฏิสัมพันธ์ที่มุ่งคุณภาพและการตอบสนองต่อความต้องการที่เปลี่ยน โดยใช้ feedback loops หลายช่วงเวลา. Five practices ใน lecture คือ small batches, MVP, BDD, TDD และ pair programming. Small batches ลด risk; MVP cheapest/easiest usable experiment ที่ทดสอบ value hypothesis; build → measure → learn แล้ว pivot/persevere. MVP ไม่ใช่ database ไม่มี screen หรือหน้าจอไม่มี data. BDD ใช้ story/scenario แบบ outside-in; TDD ใช้ failing test ขับการออกแบบ function แบบ inside-out; pair ใช้ driver ลงมือและ navigator ตรวจแนวทาง แล้วสลับบทบาทบ่อย.

ชิ้นงาน | BDD vs TDD vs Spec-Driven

Approach	Starting point	Concrete check
BDD	behavior/value from stakeholder outside-in	Given count2, When service restarted, Then count2
TDD	failing test for unit inside-out	counter increment/validation function Red → Green → Refactor
Spec-Driven	living approved contract	SRS/SDS/STS, scoped agent issue, trace evidence

Organisational anti-pattern: renaming PM → SM/product manager → PO/developers → Scrum team ไม่สร้าง authority/flow ใหม่. Cross-functional team ต้องมีทุก skill เพื่อส่ง end-to-end. Loosely coupled/tightly aligned มี business mission และ end-to-end ownership. Water-Scrum-Fall มี approvals ก่อน/central QA-Ops หลังทำ delivery date ถูกกำหนดนอก sprint; DevOps culture/loose coupling/automated pipeline ลด bottleneck ทั้งสาย

7.13 Scrum roles/artifacts/events พร้อม input/output

Lecture Content | Exam-High | L7 หน้า 39–50,94–96,114–153. Source ใช้ PO, Scrum Master, Scrum team/ development team. PO owns vision/ order/ requirements acceptance/ ship/ continue; SM facilitates/ coaches/ shields/ unblocks/ time boxes/ empirical data และไม่มี management authority assign work; team self-assigns/ negotiates/how. Event descriptions รักษา lecture ไม่เอากฎทั่วไป overwrite

Event	Input / participants	Output / why
Refinement	PO/ SM/ team; backlog/ new issues	ranked/split/clarified sprint-ready stories; clear New Issues
Planning	PO goal/ranked backlog; SM facilitates; team capacity	Sprint Backlog + goal + forecast; select ready work within velocity
Daily stand-up	team; SM blocker help; PO optional	replan day; 15min; not manager status report
Review	increment/ AC; team demo; PO/ stakeholder/ customer feedback	acceptance/rejection; new backlog stories from feedback
Retrospective	team process experience; SM facilitates	keep/stop/start changes for next sprint; action actually applied

Artifacts: Product Backlog unimplemented stories not yet in sprint, ordered by value; Sprint Backlog selected forecast/work plan; Done Increment tested usable result. Sprint usually2 weeks in worked course examples; health check mentions2–4w or fewer. Do not confuse story-specific acceptance definition with team-level DoD from L1

7.14 User stories: value + details + criteria + metadata

Lecture Content | L7 หน้า 60–72,102–111. Story persona เป็น role ไม่ใช่ชื่อคน. As a role / I need capability / So that benefit. Assumptions/details ทำให้ไม่ต้องเดา; AC เป็น measurable Given/When/Then. INVEST: Independent, Negotiable, Valuable, Estimable, Small, Testable. Epic ใหญ่เกิน sprint/estimate honest ต้อง split เป็น useful stories ไม่ใช่ชิ้น layer

ตัวอย่างพร้อมวิธีคิด | Counter requirements แตกเป็นสี่ stories ตาม lecture

User: single counter เพื่อ track times; multiple counters เพื่อหลาย counts. **Service Provider:** persist last known count เพื่อไม่เสียหลัง restart. **System Administrator:** reset เพื่อเริ่มนับใหม่. เรื่องเดียวใช้ persona ต่างกันตาม need ไม่ใช่ใช้ Developer ทุก story.
Persistence details: Redis/ name-value ตาม worked source; AC Given count2, When restart, Then return2. **Reset details:** POST /counters/{name}/reset ตาม source; Given count5, When reset and get, Then0. **Cloud story:** managed platform/container; technical debt label ในตัวอย่าง; URL available AC. Choice Redis/cloud เป็น course example ไม่ใช่ policy ทุกระบบ

ชิ้นงาน | Grooming check ก่อนบอกว่า Ready

- User value ที่อธิบายได้; scope พอดี sprint; dependencies และ unknowns เปิดเผย.
- AC positive/negative/boundary ตาม risks; expected outcome observable.
- Details สอดคล้อง SRS/SDS; label bug/enhancement/technical debt ไม่สลับ.
- Assignee/milestone/estimate ยังไม่มี ให้ – ตาม L10 grooming example; อย่า invent ที่มาจาก planning.

Triage destinations: soon → ranked Product Backlog; good idea not now → Icebox; irrelevant → reject/close with reason. Refinement ให้ New Issues empty และอย่างน้อยสอง sprints groomed ตาม source tip; ทุก item ไม่ต้องละเอียดเท่ากัน ด้านบนต้อง ready

7.15 Story points และ sprint capacity: อย่าแปลงเป็นชั่วโมง

Lecture Content | L7 หน้า 67–72,117–126. Points relative overall effort/complexity/uncertainty; scale 1, 2, 3, 5, 8, 13, 21 หรือ T-shirt. Anchor เป็น story ที่ทุกคนเข้าใจ. One step larger คือ next scale ไม่ใช่บวก 1. Velocity uniquely team และเปลี่ยนได้; compare teams ด้วย points ไม่ได้

ตัวอย่างพร้อมวิธีคิด | Single Counter sprint ตาม lecture

Goal: single counter working/deployed. Selected single counter8 + persistence5 + cloud5=18. Multiple counters/reset/remove ยัง backlog/icebox ตาม priority. Cloud/persistence ไม่ใช่ extra feature ที่ผู้ใช้กด แต่ทำให้ goal เป็น working increment ที่ใช้ได้และไม่เสีย state. Milestone short title+goal+two-week dates; self-assignment เกิดเมื่อ developer takes card ไม่ใช่ SM แจก

ShuttleNow: anchor3: same3, step bigger5, two steps8, step smaller2. Select live8+own-stop5+alert3=16; QR3 outside goal; all-stops5 would exceed; phonebug2 source ไม่เลือก. ถ้าข้อมูลใหม่ทำให้ phonebug blocks goal ต้อง revisit dependency ไม่ใช้ generic knapsack แทน rationale

7.16 Daily execution, WIP, burndown และ feedback

Lecture Content | L7 หน้า 129–159. Take next highest unassigned priority → self-assign → In Progress. WIP1 ต่อคน เว้น blocked item ให้ทำ next story. Finished → Review/QA+PR; merged → Done. Daily questions previous day/today/blockers; discussions หลัง 15min เฉพาะคนเกี่ยวข้อง. SM เอา blockers ออก ไม่ assign implementation

ตัวอย่างพร้อมวิธีคิด | อ่าน burndown ของ 18 points

Explanation / Intuition. ถ้า story8 merged เหลือ 10; story5 merged เหลือ 5; อีก 5 merged เหลือ 0. Ideal line ลดสม่ำเสมอเป็น reference แต่ actual step-down ตาม completed work. Flatline อาจมาจาก large item/blocker/review wait ไม่สรุปทันทีว่าทีมไม่ทำงาน. ดู card/PR/evidence ประกอบ; source weekend ไม่ทำ burn จากงานที่ยังไม่เสร็จ

Review inspect product กับ AC; retrospective inspect process. Feedback เป็น new Issue แล้ว triage/rank ไม่บังคับ next sprint ทุกคำขอ. Metrics baseline+goals: lecture ตัวอย่าง reduce deployment10h → 2h, test defects25% → 50%. Four actionable metrics: Mean Lead Time (idea → production), Release Frequency, Change Failure Rate, MTTR. อย่าแทน definition lecture ด้วยคำว่า commit lead time โดยไม่ label extra interpretation

7.17 Unfinished / untouched / rejected: แยกสามกรณีตาม source

Lecture Content | course-specific | L7 หน้า 149,161–165.

ชิ้นงาน | End-of-sprint handling

Situation	Action	Why
Untouched	top Product Backlog; remove sprint milestone	priority ต้อง evaluate ใหม่; ไม่ carry โดยอัตโนมัติ
Rejected at review	label+close; Story ใหม่/new AC	acceptance failure เห็นชัด; ไม่ count เดิมซ้ำ
Unfinished with completed part	adjust old description/points; label unfinished → Done; new remaining story	source ให้ credit ส่วนเสร็จและ velocity accurate

ตัวอย่างพร้อมวิธีคิด | 8-point story ทำได้ 5 เหลือ 3

ปรับ Story เดิม อธิบายเฉพาะ completed work และ 5 points,label unfinished,Done ตาม lecture. Create new story remaining work+new AC+3points และ assign next sprint ตาม ขั้นตอนเฉพาะของวิชานี้. รวม 5+3=8 ยัง trace ได้; ไม่ย้าย Story เดิม 8 points ทั้งก้อนไป next sprint หรือ นับว่าเสร็จครบ 8 points. เป็นกฎเฉพาะ source วิชานี้; ไม่อ้างเป็น universal Scrum rule

Closeout: done → closed, close current milestone, new milestone, adjust unfinished; check stories assigned old sprint closed และ remaining reassigned ตาม source. Health check structure(accountabilities/order/sprints/backlog),events(goal/ forecast/ daily replan/ Done),feedback(review → backlog,retro → process change). Anti-pattern no empowered PO,too large/ not dedicated/ distributed/ siloed/ not self managing ต้อง diagnose ด้วย evidence ไม่ใช่ตำแหน่งใน orgchart

7.18 สรุปบทและฝึกตอบก่อนสอบ

แนวคิดที่ต้องอธิบายได้: PO จัดลำดับตามคุณค่า ทีมประเมินกำลังทำงานและเลือกวิธีส่งมอบ SM ช่วยกระบวนการ การเลือก Sprint ต้องดูเป้าหมาย ความพร้อม และ Velocity ร่วมกัน

สิ่งที่ต้องสร้างได้: User Story, Product/Sprint Backlog, Board และ Burndown

ตัวอย่างพร้อมวิธีคิด | คำถามสั้นพร้อมเหตุผล

Story 8 points ทำเสร็จส่วนหนึ่ง 5 และเหลือ 3 ต้องจัดการอย่างไรตาม Lecture?
แนวตอบ: ปรับ Story เดิมให้เหลือขอบเขตที่ทำเสร็จและ 5 points ติดป้าย unfinished แล้วย้าย Done สร้าง Story ใหม่พร้อม AC สำหรับงานที่เหลือ 3 points กฎนี้เป็นขั้นตอนเฉพาะของวิชาตาม Lecture

เชื่อมแนวสอบ: Part 2: Backlog Grooming 10 และ Sprint Planning 5 คะแนน

ตรวจความพร้อม: ลองอธิบายโดยไม่เปิดนิยาม แล้วสร้างชิ้นงานจากโจทย์ใหม่ ระบุสมมติฐานและวิธีตรวจผลให้ครบ

บทที่ 8

Applied Example: POS clarification และ Kaching SDS

Applied Example | Exam-Support. บทนี้ใช้ Lecture 3 ChatGPT Sessions 28 หน้า และ Kaching System Level SDS v0.1 22 หน้า เพื่ออ่านการเปลี่ยน vague need เป็น artifact จริง. เป็น sample draft ไม่ใช่ academic definition หรือ approved production implementation. POS minimalist requirements ใน L3/L4 มี returns/refunds; หลัง clarification Kaching ตัดออก จึงห้ามผสม scope สองเวอร์ชัน

8.1 Clarification ที่เลือกถามเรื่องมีผลต่อ product

Session หน้า 1–6. เริ่มจากคำถาม 40 ข้อแล้วลดเป็น stakeholder decisions ที่ developer ไม่ควรเดา: VAT inclusive/ exclusive, discount policy, split payment, outages, return eligibility/authorization, deployment scope/reporting. Routine recommendations ยังต้องเปิดเป็น assumptions ไม่ใช่แอบสร้าง source requirement

Ambiguity	Confirmed decision	Impact downstream
VAT / currency	tax-inclusive Thai VAT, baht/ satang	arithmetic/UI/receipt/test rounding
Stock availability	physical item at counter; no synchronous stock check	remove block-check integration; async decrement
Discount	single order-level; percent and amount fields	percent computes rounded amount; manual amount clears percent
Split payment	not allowed; cash or card	one payment method; UI/ state/ test reject split
PSP unavailable	cash only if no request sent	distinguish unavailable-before-send from unknown-after-send
Returns/refunds	Customer Service outside POS	remove return/refund/receipt/restock features here
Inventory	asynchronously process sale messages	durable outbox/retry/dedup required
Reporting	assigned store for Manager; any/ consolidated for Admin/ Head Office	store scope authorization and report filters
Deployment	multiple terminals/ multiple stores	unique IDs/context/audit and capacity

เหตุผล: คำถาม “ขายเสร็จตอนไหน” สำคัญกว่าเลือกรูปไอคอน เพราะเปลี่ยน persistence/receipt/inventory behavior. Clarification มีรอบถัดไปได้เมื่อคำตอบเผย gap ใหม่

8.2 Worked arithmetic: discount field state และ precision

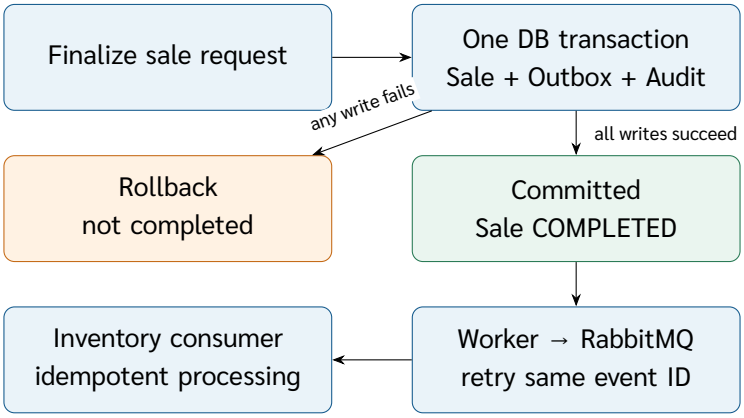
Applied Example source rules BR-004–010; numeric input below is Explanation / Intuition. สมมติ subtotal1,235.50 บาทและ percent7.5:

$$D = 1235.50 \times 7.5/100 = 92.6625 \rightarrow 92.66, \quad Total = 1235.50 - 92.66 = 1142.84.$$

ถ้า user เปลี่ยน amount เป็น 100.00 ต้อง percent=null และ total1,135.50. ถ้า percent active แล้ว cart เปลี่ยนต้อง recalculate; manual amount ไม่ย้อนสร้าง percent โดยพลการ. Test0%,100%,negative,over-subtotal,rounding-half และ cart change แยกกัน. Monetary persisted calculations ใช้ Decimal/NUMERIC ตาม SDS ไม่ใช้ binary float. VAT rate ไม่เติมเลขจากความจำ;ใช้ approved tax configuration ตาม source

8.3 Gap ที่เปลี่ยนจาก “retry” เป็น atomic contract

Session หน้า 10–13. FR เดิมบอก retain/retry message แต่ถ้าเขียน message ไม่ได้ completed sale อาจไม่มี inventory update. Revision ระบุ save sale+pending outbox เป็น transaction เดียว; either both or neither. ถ้า delivery ไม่ได้หลัง durable write แล้ว sale ยัง completed และ worker retry ได้. นี่เป็นความต่างระหว่าง local recording failure กับ remote delivery failure



อ่านภาพ: transaction success เป็น completion boundary; broker/consumer ไม่ได้อยู่ใน DB transaction. Stable event ID ป้องกัน deduction ซ้ำเมื่อ at-least-once redelivery. Worker crash หลัง publish ก่อน mark sent อาจ publish อีกครั้ง จึง broker confirm อย่างเดียวไม่ทำ exactly-once end-to-end

ชิ้นงาน | Failure cases กับ required result

Failure	Expected behavior
DB/outbox write fails for cash	no completed sale; cash sale open; cashier informed
Already-approved card, finalization fails	recover original attempt; never blindly charge again
RabbitMQ down after commit	completed sale unchanged; pending outbox retry
Consumer down	durable message waits/redelivers; stock eventually updates
Printer fails	completed sale remains; authorized reprint+audit

8.4 Unknown card result ไม่ใช่ decline

Session FR-009–022; Kaching หน้า 12–14. ก่อน send สร้าง durable unique PaymentAttempt; ถ้า create ไม่ได้ไม่ send. Decline ยืนยันแล้ว retry/cash/cancel ได้. Timeout หลัง send ไม่ทราบว่า charged หรือไม่ ต้อง block new payment/cash/cancel และ query original reference. Approved แต่ finalize sale ไม่ได้ต้อง recover โดยไม่ charge ใหม่. PSP cannot determine ยัง recovery pending; operator ดูได้แต่ห้าม manual force approved

ตัวอย่างพร้อมวิธีคิด | ทดสอบกรณี timeout ที่อาจเกิดเงินจริง

Proposed verification. Arrange PSP approved แต่ response หาย; Act cashier attempts new card/cash; Expected denied until reconciliation. Query original reference → approved → finalize sale+outbox once. Check one charge/one completed sale/one event identity. การแสดง generic “failed, retry” ที่ไม่แยก unknown มี double-charge risk

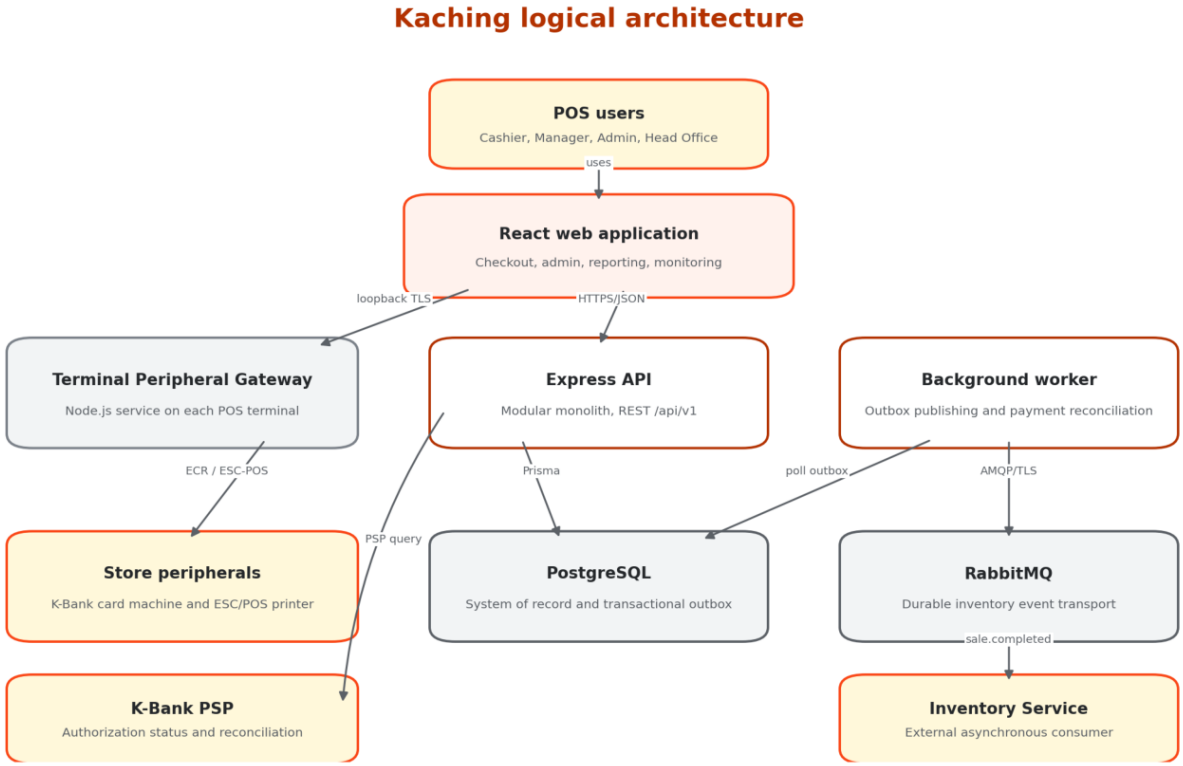
8.5 อ่าน System-Level SDS ทั้ง 18 ส่วน

Kaching SDS v0.1 | source หน้า 1–22.

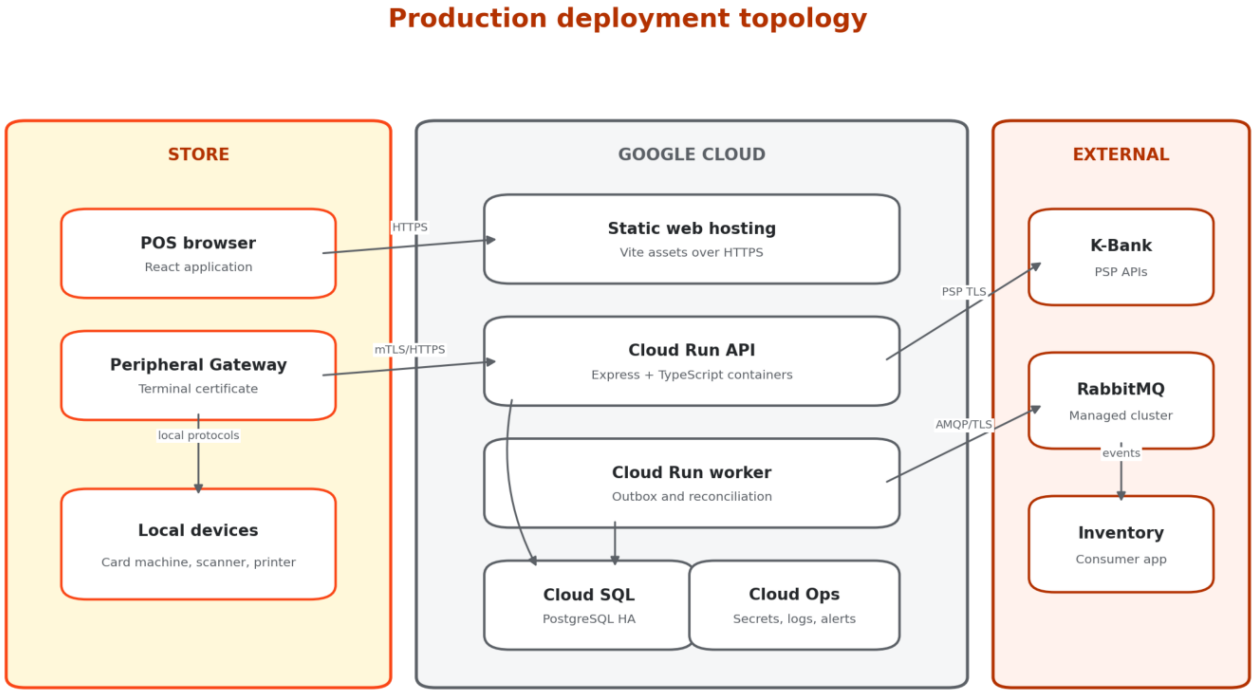
Section / pages	Engineering meaning และสิ่งที่ต้องตรวจ
Introduction (3)	purpose/scope/exclusions/audience/terms; not screen/class/endpoint full spec
Architectural Drivers (4–5)	prevent duplicate charge/durable sale outbox/multi store; fixed stack and measurable targets
System Context (5–6)	Kaching owns sale/payment status; inventory owns stock; PSP owns sensitive payment
Architecture Overview (6–7)	modular monolith API+worker+gateway; PostgreSQL source of truth; interfaces/modules isolated
Deployment (8–9)	static frontend/CDN, Cloud Run API/worker, Cloud SQL HA, RabbitMQ quorum, terminal certificate; environments separated
Application Structure (9–10)	frontend/ server state/ forms/ errors; API v1/ camelCase/ UTC/ money strings/problem-detail/idempotency/version/OpenAPI
Data Architecture (11–12)	entity IDs/money snapshots/sale states; transaction boundaries
Integration (13–14)	bank adapters/gateway/reprint/outbox delivery/failure matrix; exact bank protocol unresolved
Security (14–15)	hash/session/CSRF/RBAC/store scope/disabled states/secrets/audit/ card boundary
UI Standards (16)	tokens/accessibility, cart/totals layout, scanner/search, recovery state
Observability (17)	structured logs/correlation; recovery/outbox/DLQ/DB/gateway alerts
Performance/ Recovery (17–18)	server revalidate; indexes/pools; backup/PITR; RPO/RTO drill
Development/Delivery (18–19)	monorepo/ shared contracts/ migrations/ CI/ security gates/ versioned containers/promotion/rollback
System Test Strategy (19)	unit/ component/ integration/ contract/ E2E/ fault/ load/ security/ recovery/accessibility
Feature Allocation (20)	features A–R plus administration S allocated to runtime components
Decisions/Risks (20–21)	ADR1–5; provider/ gateway/ delivery/ backend outage risks and treatments
SRS Amendments (21)	FR039–043/ BR021/ NFR012–016 draft additions; update before baseline
References (22)	external supporting refs; sample choices not universal policy

Baseline check: version0.1 says Sample Draft for Editing and prepared by placeholder. These fields alone warn not approved production contract. L3 หน้า 51/L4 หน้า 132 ask to close upstream amendments/protocols/printer/ hosting/region/message ack rules/stakeholder approval before v1.0

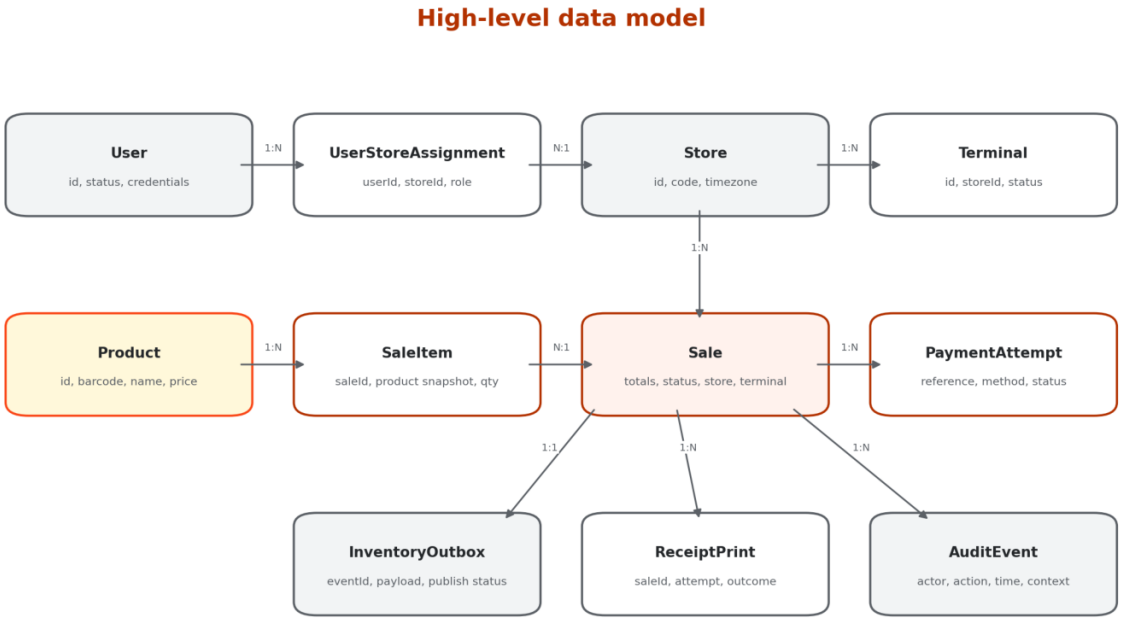
8.6 Architecture, deployment และ data: อ่านสามภาพคนละคำถาม



Source figure | Kaching หน้า 5. Logical architecture บอก components และ interactions: web → API/gateway, API → DB/PSP, worker → outbox/RabbitMQ, RabbitMQ → Inventory. Modular monolith เก็บ business modules ใน deployment หลักแต่ worker แยกเพื่อไม่แย่ง checkout capacity; adapters isolate vendor protocol. Domain services ไม่ dependon Express/RabbitMQ ตรง ๆ ตาม layering rule



Source figure | Kaching หน้า 8. Deployment บอก where/how components run และ network security. Dev/staging ใช้ synthetic/masked data; terminal gateway certificate เฉพาะ terminal และ ขาออก only. Backend outage หยุด checkout เพราะ no offline scope; PSP outage ก่อน send ยัง cash ได้เมื่อ backend healthy. อย่าอ่านลูกศร diagram เป็น“ทุก dependency down แล้ว sale ยัง continue”



Source figure | Kaching หน้า 11. Data model บอก entities/relationships. Sale Item snapshot SKU/name/unit price/ VAT/quantity รักษาประวัติเมื่อ catalog เปลี่ยน. User roles/store scope separate; paymentattempt/outbox/audit มี identity เพื่อ recovery/trace. Report business date Asia/Bangkok ต่างจาก UTC timestamp storage

8.7 Measurable quality targets และ verification ที่ตรงหน่วย

ชิ้นงาน | Kaching targets ไม่ใช่ค่า default ทุก app

Target	Meaning	Proposed check
20stores×10terminals/200users	deployment/concurrency capacity	load profile ครบ 200; notjust200registered accounts
20completed sales/sec	end-to-end throughput รวม outbox	count successful durable completions
p95≤2s non-external actions	response percentile excluding payment/printing	log defined operation latency under load
99.5%monthly backend availability	measurement window/definition ต้อง specified	uptime/error budget monitor
RPO15min/RTO4h	loss window/recovery elapsed time ต่างกัน	restore exercise+timestamp evidence
WCAG2.1AA/ keyboard/ min resolution	UI quality constraints in sample	contrast/keyboard/focus/ responsive checks

RPO vs RTO: RPO ยอมเสียข้อมูลย้อนหลังเท่าไร; RTO กลับบริการภายในเท่าไร. มี backup ไฟล์ไม่ได้พิสูจน์ restore within4h. Contract test ตรวจ API/event schema; fault injection ตรวจ rollback/worker crash/duplicate delivery; recovery test ตรวจ runbook และ payment/outbox backlog หลัง restore

8.8 Traceability: ครบ 69 ข้อใน session ก็ยังไม่จบงาน

Session หน้า 15–24. FR38+BR20+NFR11=69; positive68mapped, BR020scope boundary1, unmapped0 ตาม matrix source. นี้พิสูจน์ inventory coverage ของ draft นั้น ไม่พิสูจน์ทุก requirements approved/tested implemented. SDS เพิ่ม feature S และ proposed requirements จึงต้อง regenerate matrix. ตารางต่อไปย่อข้อความเพื่ออ่าน coverage; ใช้ IDs ของ**revised FR list**ไม่ใช่ draft ก่อน renumber

ID	Requirement meaning (ย่อ)	Feature
FR-001	Sign in / roles	A
FR-002	Start sale	A,D
FR-003	Add/change/remove items	E
FR-004	Recalculate totals	F,G
FR-005	Discount input fields	G
FR-006	Cancel before completion	D
FR-007	Cash or card selection	H
FR-008	Cash tender and change	I
FR-009	Durable payment reference before send	J
FR-010	No send if attempt not stored	J
FR-011	Send amount + reference	J
FR-012	Display approval/decline	J
FR-013	Decline retry/cash/cancel	J,H,D
FR-014	PSP unavailable before send	J,H
FR-015	Interrupted request: query original reference	K
FR-016	Block further payment while unresolved	K
FR-017	Approved but finalization failed	K,L
FR-018	Reconcile unresolved attempt	K
FR-019	Finalize approved without new charge	K,L
FR-020	Return unpaid if declined/not processed	K,H,D
FR-021	Recovery pending until resolved	K
FR-022	View recovery attempts	P,A
FR-023	Durable sale record	L,B
FR-024	Print successful receipt	M
FR-025	Reprint without duplicate/reversal	M
FR-026	Atomic sale + outbox	L,N

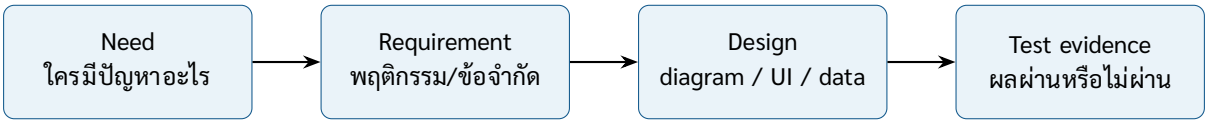
FR-027	Failure: cash open / approved-card recovery	L,I,K,N
FR-028	Inventory payload identifiers/items	N,B
FR-029	Async inventory transmission	O
FR-030	Retain/retry until acknowledgement	O
FR-031	Delivery failure does not reverse sale	O,L
FR-032	View pending/failed outbox	P,O
FR-033	Manager assigned-store report	Q,A
FR-034	Selected-store report	Q,A
FR-035	Consolidated report	Q,A
FR-036	Separate cash/card totals	Q
FR-037	Add product	C,A
FR-038	Update product/price within scope	C,A
BR-001	THB VAT-inclusive prices	C,F
BR-002	Subtotal = sum extended lines	E,F
BR-003	One order-level discount	G
BR-004	Percent-to-amount formula	G
BR-005	Round to nearest satang	G,F
BR-006	Manual amount clears percent	G
BR-007	Cart change recalculates active percent	G,E
BR-008	Percent 0..100	G
BR-009	Amount 0..subtotal	G
BR-010	Final total = subtotal - discount	F,G
BR-011	No split payment	H
BR-012	Cash tender sufficient	I
BR-013	Card approval required	J,L
BR-014	Completion payment + durable save; printer separate	L,M
BR-015	Cancelled sale no totals/message	D,Q,N
BR-016	No synchronous stock check	E,N
BR-017	Completed local-date/store totals	Q
BR-018	Manager only assigned store	A,Q
BR-019	Admin/Head Office selected/consolidated	A,Q
BR-020	No returns/refunds/refund receipts	Scope boundary
NFR-001	p95 checkout $\leq$2s excluding external payment/printing	D,E,F,G,H,I,L
NFR-002	Durable completed sale survives external failure	L,M,J,K,N,O
NFR-003	Durable retry-capable inventory messages	N,O
NFR-004	Unique transaction prevents duplicate stock deduction	N,O
NFR-005	PSP failure permits valid cash sale	H,I,J
NFR-006	Role-based access enforcement	A
NFR-007	No sensitive card data; approved secure integration	J,K
NFR-008	Actor/time/store/terminal audit	R
NFR-009	Multi-store/terminal IDs no collisions	B,L
NFR-010	THB 2 decimals/local reporting time	F,B,Q
NFR-011	Approved printer/card/PSP/messaging interoperability	M,J,O

บทที่ 9

แผนภาพอ่านเร็ว: จากโจทย์สู่ชิ้นงาน

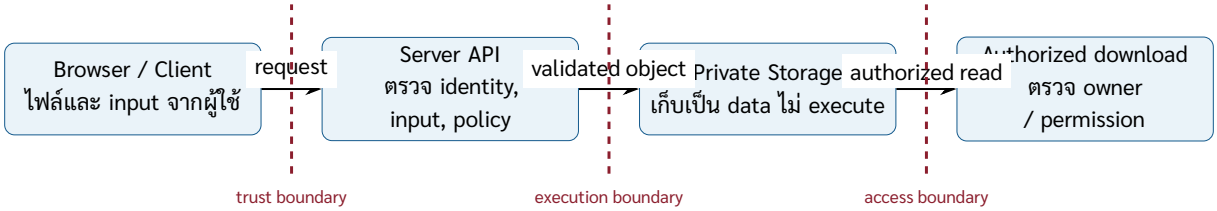
ภาพในบทนี้วาดใหม่เป็น vector เพื่อลดปัญหาตัวหนังสือพร่าหรือถูกตัดจากภาพสไลด์ ใช้อ่านความสัมพันธ์สำคัญ แล้วกลับไปดูคำอธิบายเต็มในบทต้นทาง

9.1 Traceability: requirement ต้องมีหลักฐาน



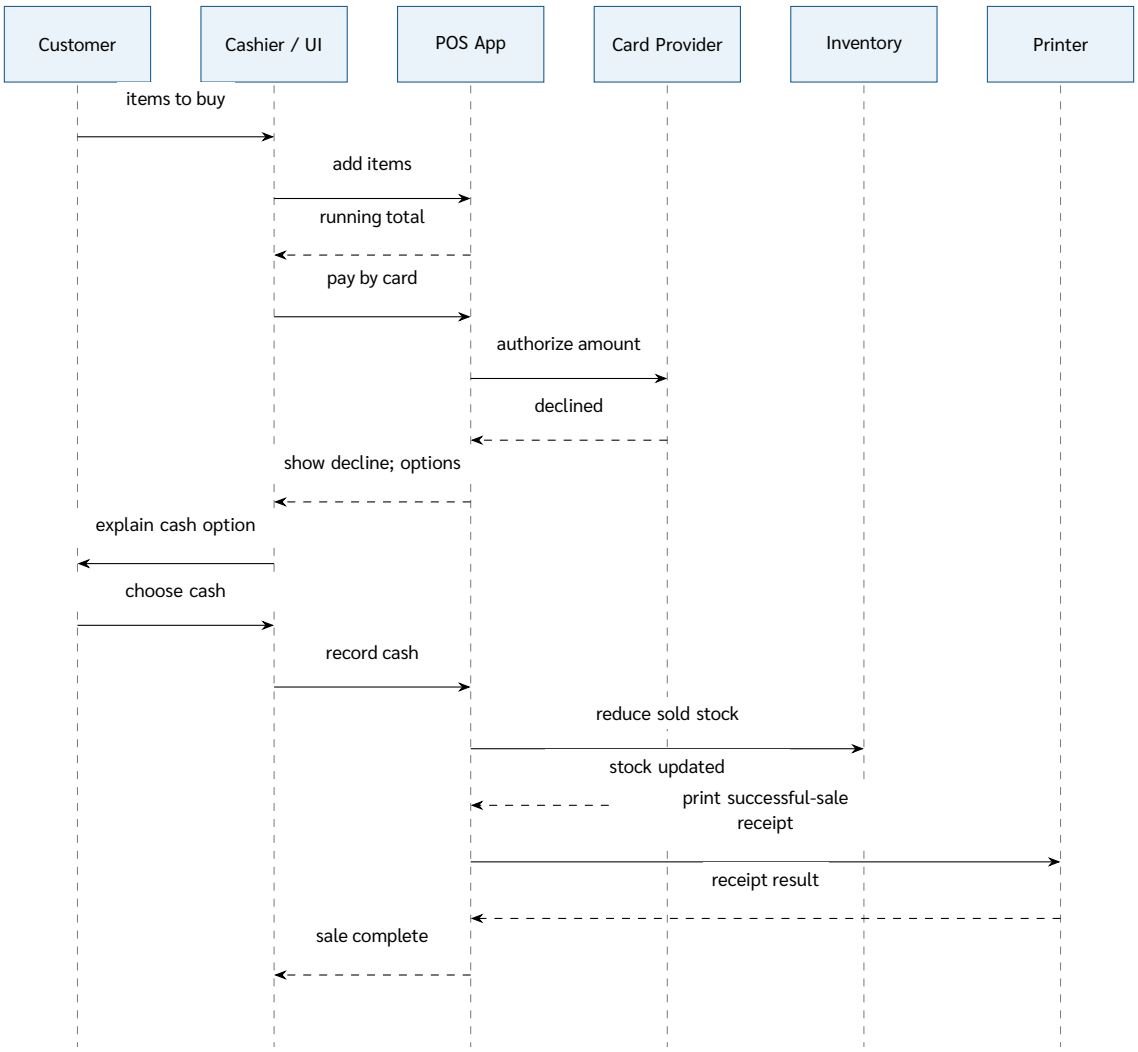
ทุกลูกศรต้องตอบได้ว่าเหตุใดชิ้นงานถัดไปจึงตามจากชิ้นก่อนหน้า หากไม่มีหลักฐานให้ทำเครื่องหมายคำถามที่ต้องถาม ห้ามเติมเป็น fact. ความต้องการกว้าง ๆ เช่น “ทำงานเร็ว” ต้องตกลง workload, จุดวัด และเกณฑ์ผ่านก่อนสร้าง performance test.

9.2 Security boundary: ข้อมูลจาก client ยังไม่น่าเชื่อถือ



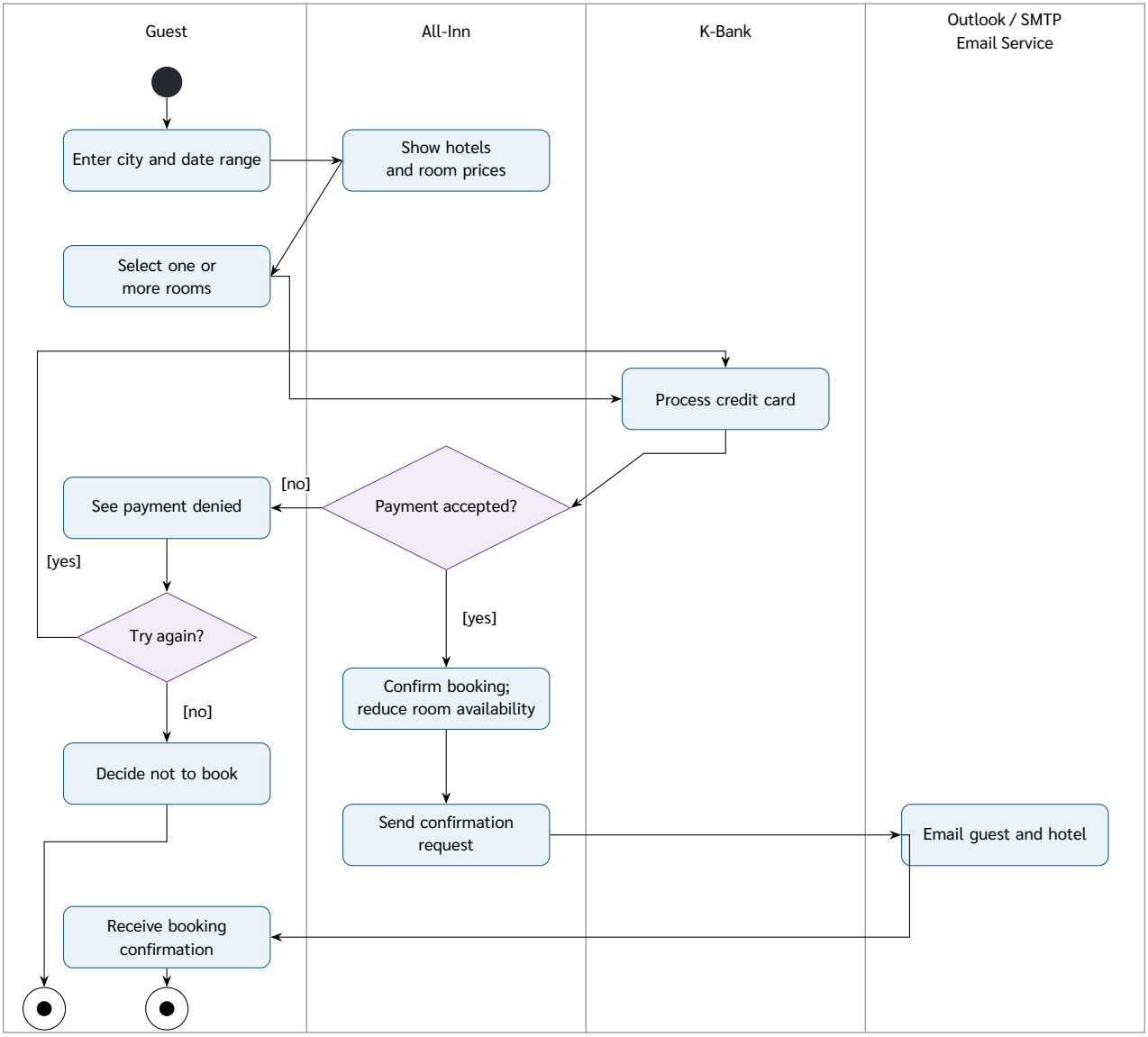
การซ่อนปุ่มที่ UI ไม่ใช่ authorization. ตรวจ permission ที่ server ทุก request; validate file ที่ server; เก็บ upload ในพื้นที่ execute ไม่ได้; การอ่านไฟล์ต้องตรวจสิทธิ์กับ object นั้น. แผนภาพคือคำอธิบายแนวคิด ไม่ใช่ architecture ที่โจทย์ระบบใดระบบหนึ่งยืนยัน.

9.3 Sequence: POS เมื่อบัตรถูกปฏิเสธ



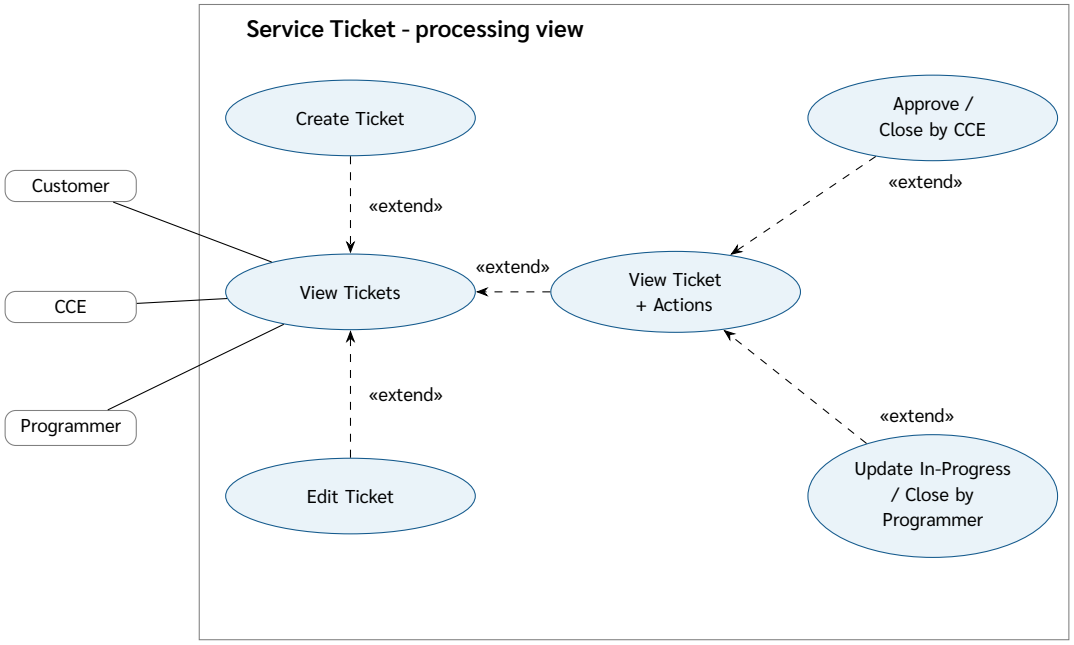
อ่านจากบนลงล่าง: provider ตอบ declined ก่อนเสนอทางเลือก ลูกค้าเลือกเงินสด จึงบันทึกยอดขาย แล้วค่อยปรับ stock และพิมพ์ใบเสร็จ. ลูกศรทึบเป็น request/action; เส้นประใช้กับ response ในตัวอย่าง. อย่าทำผลสำเร็จ เช่น stock reduction ก่อนมีการชำระสำเร็จ.

9.4 Activity: การจองและชำระเงิน



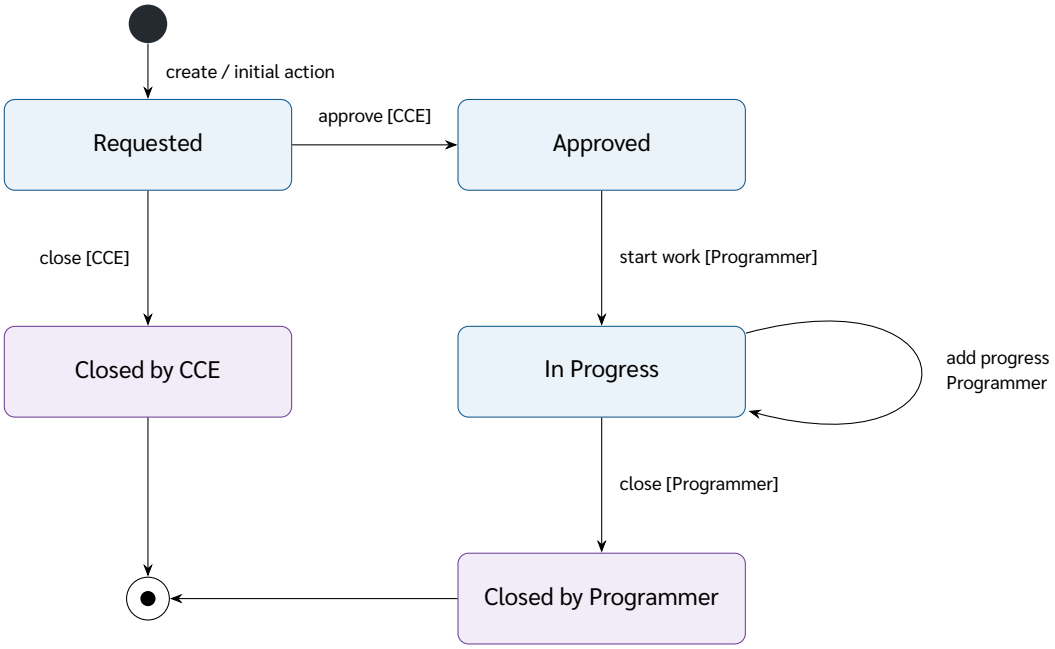
แบ่ง swimlane ตามผู้รับผิดชอบ; diamond คือ decision ที่ outgoing flow ต้องมี guard. ทาง declined แยก retry กับ cancel. เฉพาะ accepted path ทำให้ยืนยัน booking ลด availability และส่งอีเมล. ลูกศรของทาง retry เข้าสู่การชำระเงินโดยไม่ตัดผ่านข้อความ/กิจกรรมอื่น.

9.5 Use Case: แสดง actor และเป้าหมาย



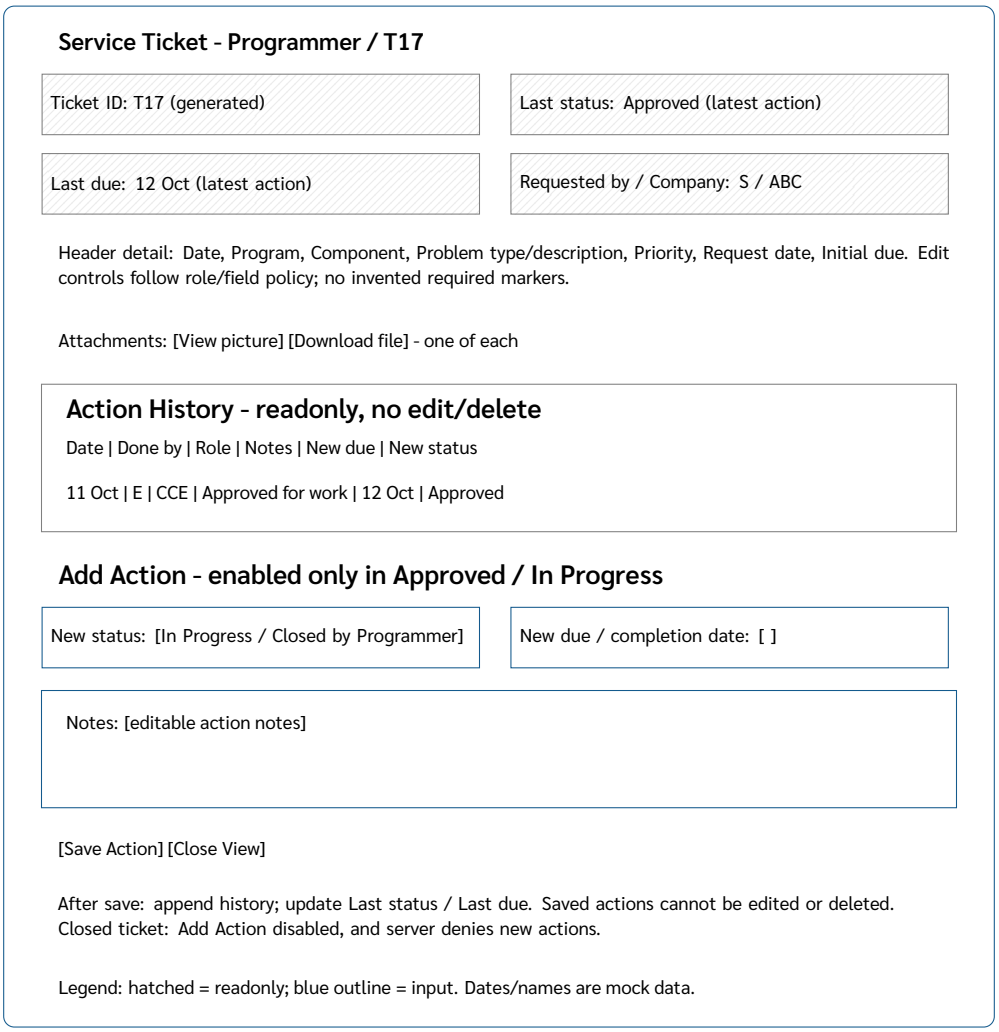
กรอบคือ system boundary; actor อยู่นอกระบบ; ellipse คือ goal ที่ระบบให้บริการ. ใช้เส้น association แสดง actor ใช้ capability. ใช้ include/extend เมื่อความสัมพันธ์มีความหมายตาม UML; อย่าใช้แทนลูกศรลำดับเวลา.

9.6 State: สถานะเป็นสภาพของ object



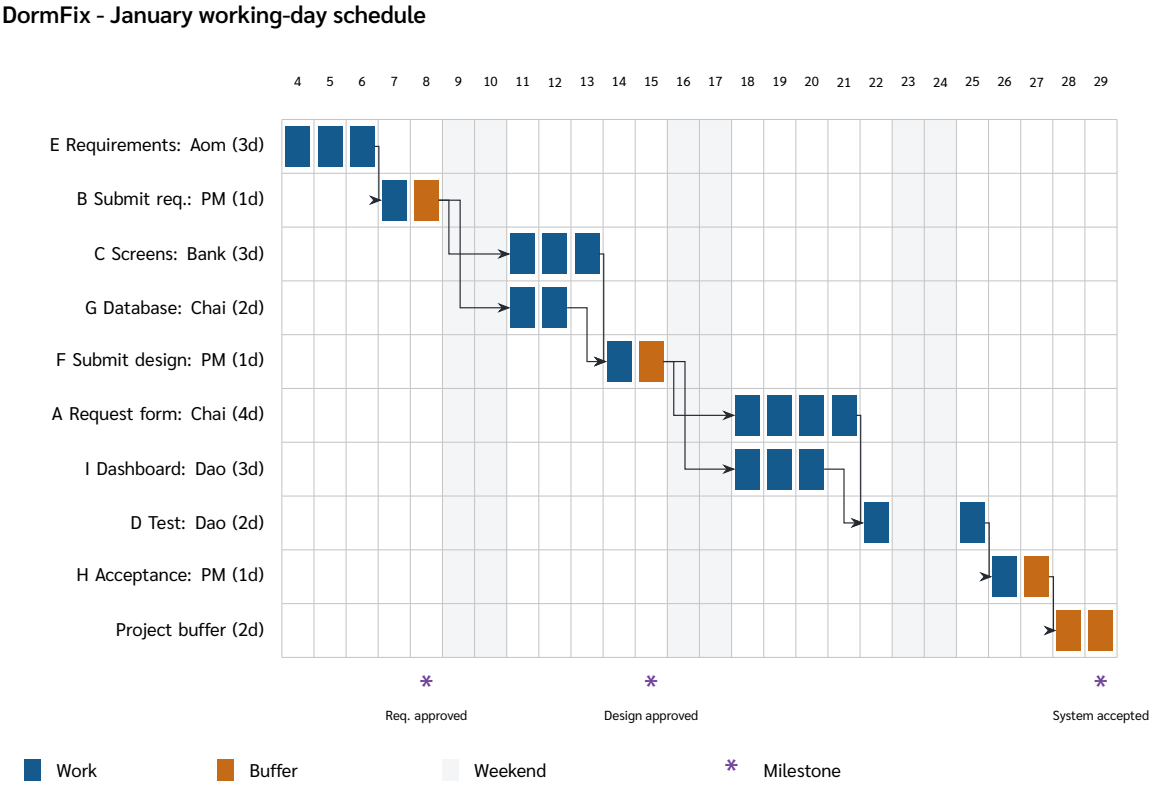
State diagram แสดง lifecycle ของ Ticket หนึ่งใบ. ชื่อสถานะคือสภาพที่คงอยู่; transition คือ event [guard] / effect. การปิดโดย CCE กับ Programmer แยกสถานะ เพราะผู้มีสิทธิ์และความหมายต่างกัน. ทดสอบทั้งทางอนุญาตและการเปลี่ยนสถานะที่ต้องปฏิเสธ.

9.7 UI: แยกข้อมูลอ่านอย่างเดียวกับ action



พื้นที่ hatch คือข้อมูล read-only; เส้นขอบสีน้ำเงินคือ input. แยก Action History ที่บันทึกแล้วจาก Add Action ที่กรอกใหม่ และแสดงเงื่อนไขที่ทำให้ปุ่มเปิด/ปิด. ค่า Last status/Last due คำนวณจาก action ล่าสุด จึงไม่ควรให้ผู้ใช้แก้ตรง ๆ.

9.8 Gantt: วันทำงาน dependency และ buffer



ตารางนี้เริ่มวันจันทร์ 4 มกราคม 2027 และนับเฉพาะวันทำงาน. สีน้ำเงินคืองาน สีส้มคือ buffer สีเทาคือวันหยุด; ดาวเป็น milestone ซึ่งไม่เพิ่มวันทำงาน. เมื่อตรวจแผน ให้เทียบทุกวันเริ่ม/จบกับ predecessor และระบุเหตุผลของ buffer.

บทที่ 10

Integrated Worked Casebook: จากโจทย์สู่หลักฐาน

บทนี้รวมวิธีคิดที่แยกอธิบายในบทก่อนให้เห็นเป็นงานต่อเนื่อง ตั้งแต่คำขอของ stakeholder ไปจนถึง specification, design, plan, test และ evidence. ป้ายแหล่งข้อมูลยังคงแยกชัด: กฎมาจาก Lecture Content; สถานการณ์โจทย์มาจาก Applied Example หรือ Exam Blueprint; เหตุผลประกอบเป็น Explanation / Intuition. ข้อมูลที่ source ไม่ระบุจะแสดงเป็นคำถามหรือ assumption.

10.1 Case A - WorkshopHub: แปลงคำขอเป็น requirement set

โจทย์ตั้งต้น: นักศึกษาที่ sign in ควรดู workshop ที่เปิดรับสมัครได้; ขั้นตอนสมัครควรรวดเร็ว; นโยบายมหาวิทยาลัยจำกัดสามรายการต่อ semester; นักศึกษายกเลิกการสมัครได้ และ coordinator ดูรายชื่อผู้สมัครได้. เป้าหมายไม่ใช่รีบเขียน feature list แต่ต้องแยก behavior, quality, policy และช่องว่างของข้อมูล.

ข้อความต้นทาง	ชนิด	เหตุผลและสิ่งที่ต้องทำต่อ
นักศึกษาที่ sign in ดู workshop ที่เปิดรับสมัครได้	FR	เป็น observable behavior; ถามว่าต้องแสดงข้อมูลใด เรียง/กรองอย่างไร และ “เปิดรับ” ใช้สถานะหรือเวลาใด
การสมัครควรรวดเร็ว	NFR	เป็น quality expectation; “เร็ว” ยังไม่ testable จนกว่า จะ กำหนด จุด วัด percentile, load และ threshold
ลง ทะเบียน ได้ไม่ เกิน 3 รายการ ต่อ semester	BR / domain rule	เป็น policy ที่ govern หลาย behavior; ต้อง มี boundary checks และกำหนดว่า cancellation นับอย่างไร
นักศึกษายกเลิก และ coordinator ดู รายชื่อ	FR สองข้อ	คนละ actor, permission และ acceptance check; split เพื่อให้ atomic และ trace ได้

10.1.1 Worked rewrite: อย่าเติมค่าที่ stakeholder ยังไม่ได้ให้

ชิ้นงาน | FR-WRK-01 - Workshop list

Rewrite: The system shall show workshops that are open for registration to a signed-in student.

Why this form: ระบุ system, action, object และเงื่อนไขผู้ใช้; ยังไม่ผูกหน้าจอหรือวิธี query กับ implementation.

Questions before baseline: “open” อิงวันเวลา, capacity หรือสถานะที่ coordinator กำหนด? ต้องแสดงจำนวนที่ว่างหรือไม่?

Verification: เตรียม workshop ที่ open/closed และตรวจผลที่ผู้ใช้ sign in; ทดสอบผู้ใช้ที่ยังไม่ sign in แยกตาม access

rule ที่ยืนยันแล้ว.

ชิ้นงาน | NFR-WRK-01 - Registration response time

Rewrite pattern: The registration operation shall complete within [T seconds] for [specified percentile] under [specified operating condition].

Required clarification: เริ่มจับเวลาที่ user action หรือ request ถึง server? จบเมื่อ response แสดงหรือเมื่อบันทึกสำเร็จ? ระบุ concurrent load, percentile และ threshold.

Status: Blocked / unresolved จนกว่าจะตกลง metric. ห้ามเปลี่ยน placeholder เป็นตัวเลขสมมติแล้วเรียกว่า source requirement.

ชิ้นงาน | BR-WRK-01 - Registration limit

Rule: นักศึกษาหนึ่งคนมี workshop registrations ได้สูงสุด 3 รายการต่อ semester.

AC: Given student S มีรายการที่นับใน semester K แล้ว 3 รายการ; When S ขอรายการที่ 4 ใน K; Then ระบบไม่สร้างรายการใหม่.

Boundary verification: ทดสอบก่อนถึงขีดจำกัด, เท่าขีดจำกัด และเกินขีดจำกัด; ตรวจว่า semester อื่นไม่ถูกนำมารวมโดยไม่ตั้งใจ.

Assumption / unresolved: แหล่งโจทย์ไม่ได้ระบุว่า cancelled registration ยังนับหรือไม่ ต้องถามหรือประกาศ assumption และใช้ให้สอดคล้องทั้ง AC, design และ test.

10.1.2 Atomicity: แยก requirement แล้วสร้าง acceptance แยก

ชิ้นงาน | Before → Transformation → Finished artifact

Before: “Students cancel a registration and coordinators see participant lists.”

เหตุผลที่ split: มีผู้ใช้สอง role, policy คนละชุด และ verification คนละแบบ; การเปลี่ยนสิทธิ์ coordinator ไม่ควรทำให้ student cancellation เปลี่ยนไปด้วย.

FR-WRK-04A: The system shall allow a student to cancel a registration that is eligible for cancellation.

FR-WRK-04B: The system shall allow an authorized coordinator to view the participant list for an assigned workshop.

AC-WRK-04A: Given registration R is eligible under the confirmed cancellation rule; When its student cancels R; Then R changes to the defined cancelled state and is no longer treated as an active registration.

AC-WRK-04B: Given coordinator C is authorized for workshop W; When C opens W’s participant list; Then only participants for W are shown.

Assumption / unresolved: “assigned workshop”, cancellation window และการคืนที่นั่งยังไม่อยู่ในข้อความตั้งต้น ต้องยืนยันก่อนถือเป็น requirement.

10.1.3 Proposal gap analysis และ traceability

ช่อง Proposal	หลักฐานใน WorkshopHub source	คำตอบเชิงวิศวกรรม
Problem	แบบฟอร์มแยกและ coordinator รวมข้อมูลด้วยมือ	Present; สื่อ pain ปัจจุบัน
Users	Students, coordinators	Present; ยังอาจต้องระบุผู้ดูแลระบบหากอยู่ในscope
Solution	WorkshopHub	Present
Expected value	ลด duplicate/error และงาน admin	Present as expected value; ยังไม่ใช่ผลที่วัดแล้ว
Scope	free workshops ใน release แรก; payment/certificates ภายหลัง	Present; ใช้เป็น release boundary
Risk	ไม่ระบุ	Missing; เสนอให้ทำ risk review แยก ห้ามอ้างว่าsource มีแล้ว

Effort	ต้องเสร็จก่อน semester หน้า	Incomplete; เป็น target date ไม่ใช่ effort estimate
Evidence	ไม่ระบุ	Missing; แนะนำ ให้ กำหนด acceptance/ UAT evidence เป็น recommendation

ชิ้นงาน | Traceability slice

Req ID	Source	AC	Planned artifact	Verification	Evidence status
FR-WRK-01	Coordinator	AC-WRK-01	Open workshop list	Functional test	Pending
NFR-WRK-01	Coordinator	Threshold unresolved	Performance condition	Timed measurement	Blocked: metric missing
BR-WRK-01	University policy	AC-BR-01	Registration rule	Boundary test	Pending
FR-WRK-04A	Student	AC-WRK-04A	Cancellation behavior	State/ functional test	Pending
FR-WRK-04B	Coordinator	AC-WRK-04B	Participant list	Authorization test	Pending

อ่าน matrix: “Pending” หมายถึงยังไม่มีหลักฐาน; “Blocked” หมายถึงยังออกแบบ verification ให้จบไม่ได้เพราะข้อมูลขาด. การตั้งชื่อ test หรือ artifact ล่วงหน้าไม่ได้แปลว่ามี implementation แล้ว.

10.2 Case B - KU Registration: requirements → UML → UI

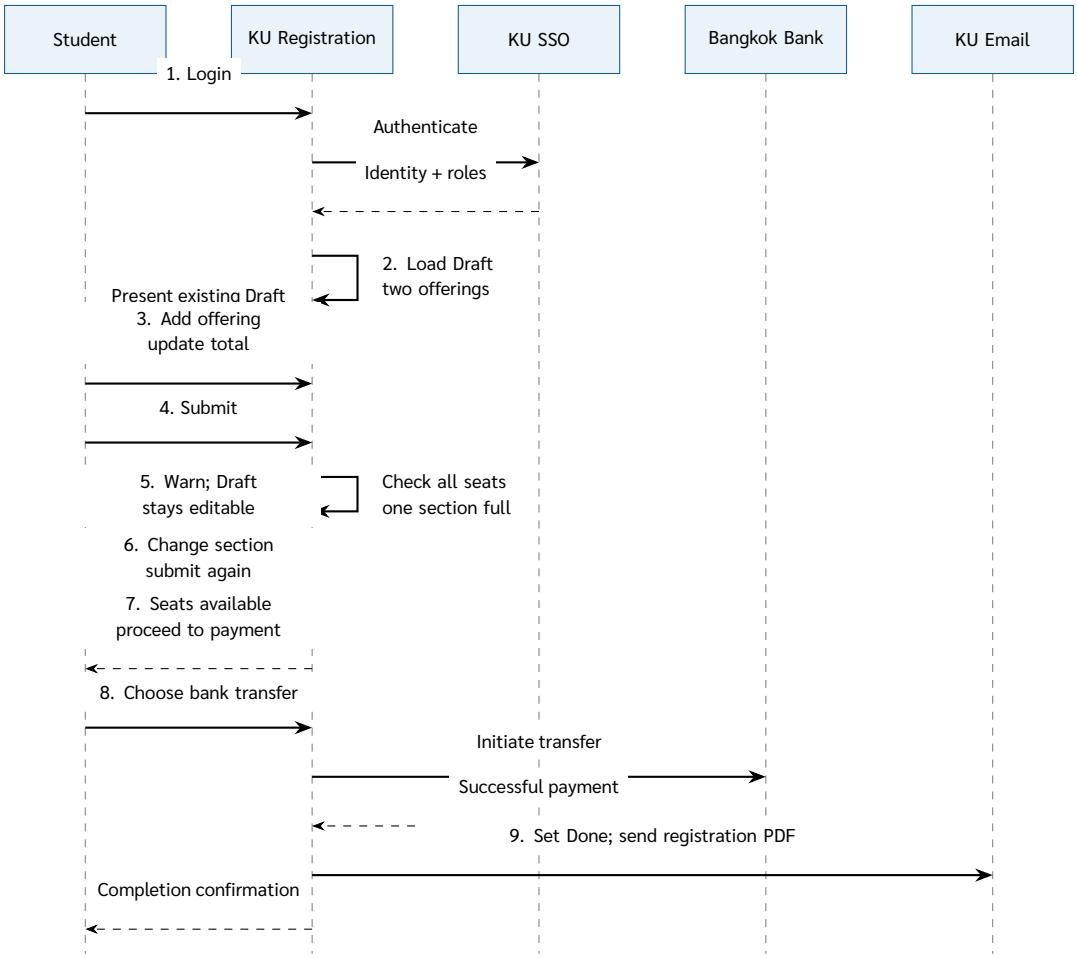
Applied Example | UML Example Exam Problem หน้า 1–9. ระบบนี้คือ Course Registration มี Draft/Done และ payment; ไม่ใช่ Grades Submission ที่มี Draft/Ready/Submitted. ก่อนวาด ให้แยก facts จาก assumption ดังนี้

ชิ้นงาน | Requirement extraction ที่ใช้ร่วมทุกภาพ

กลุ่ม	Facts จากโจทย์
Actors	Student, Department Admin, University Admin; roles มาจาก KU SSO
External	KU SSO, KU Student Info, KBank card gateway, Bangkok Bank transfer gateway, KU Email
Entities	Semester, Course, Course Offering, Registration Form Header, Line Item, Payment
Rules	active semester; draft เริ่มไม่มีวิชา; save/edit draft ได้; submit ตรวจที่นั่งทุก section; full ต้องแก้แล้ว submit ใหม่
Completion	payment สำเร็จ → Done; Done แก้ออนไลน์ไม่ได้; email ส่ง PDF registration

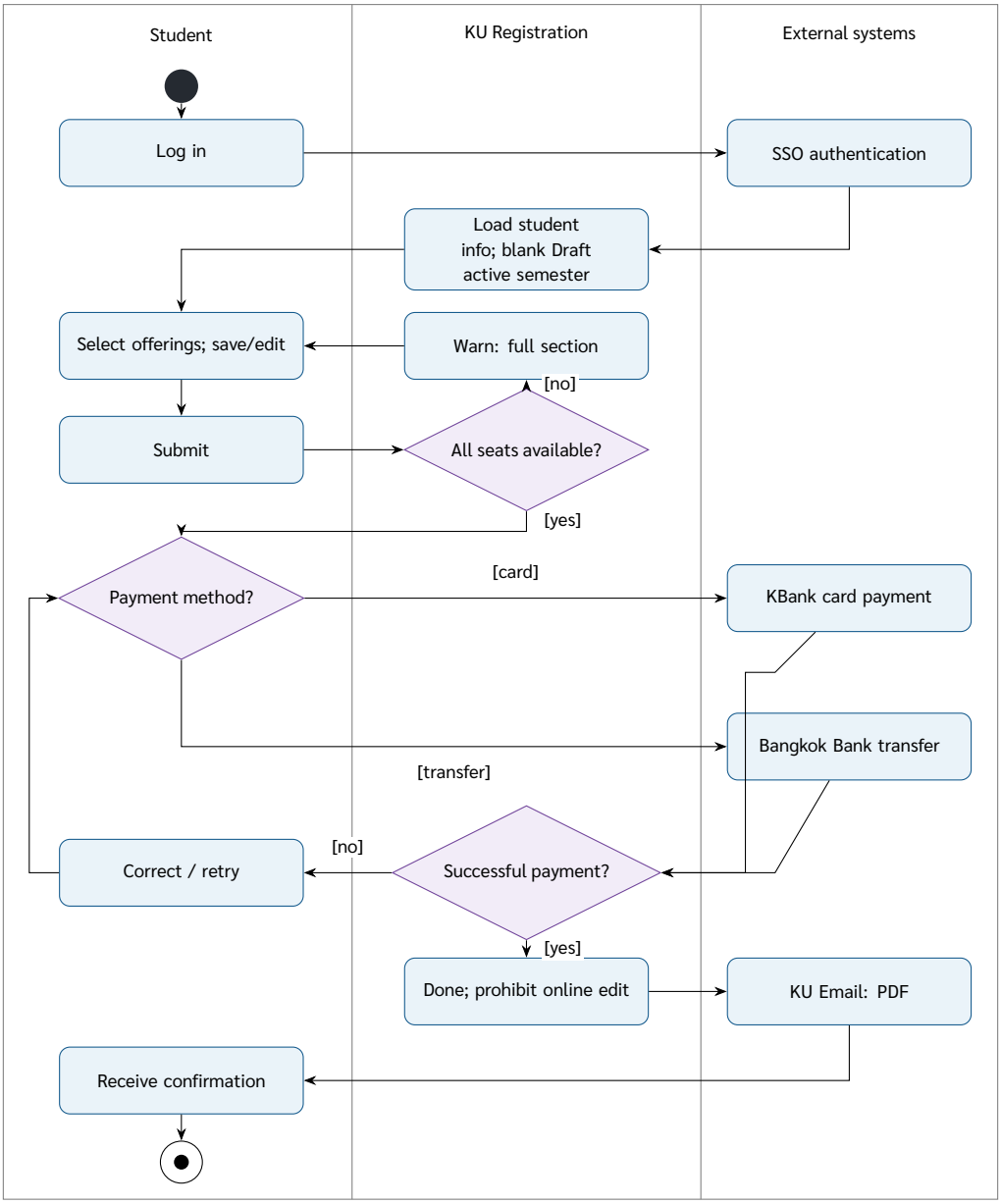
การตัดสินใจ: Sequence ใช้ scenario ที่กำหนด 9 ขั้น; Activity ใช้ workflow ครั้งแรกซึ่งเริ่ม blank form และครอบทั้งสองวิธีจ่าย. ทั้งสองภาพจึงมี input ต่างกัน แม้เป็นระบบเดียวกัน

10.2.1 Sequence: รักษาลำดับ scenario ทั้ง 9 ขั้น



Why / validation: full section ทำให้ submission fail ก่อน payment; retry ต้องตรวจทุก section ใหม่. Scenario กำหนด transfer จึงไม่มี card path ในภาพนี้. KU Student Info ใช้เมื่อเปิด blank form หรือ refresh identity; ไม่เติม call โดยอ้างว่าอยู่ใน 9-step scenario หากโจทย์ไม่ได้ระบุ. การ initiate/confirm bank transfer ในภาพเป็น abstraction ของ integration; source ไม่ระบุ protocol หรือ callback timing.

10.2.2 Activity: blank form, decision branches และ loops



อ่านภาพ: lane บอก responsibility; diamond มี mutually exclusive guards. Payment branches กลับมาที่การตรวจผลเดียวกัน; full-section loop กลับไปแก้วิชา ไม่ไปจ่ายเงิน. **Assumption / unresolved:** source ขอ loops จนสำเร็จแต่ไม่แจกแจงทุก bank error; retry payment ในภาพเป็น teaching abstraction. การสงวนที่นั่ง/หมดเวลาการจองและ race ระหว่าง submit กับ payment ต้อง clarify ก่อน implementation

10.2.3 UI prototype: List View และ Registration Form View

Applied Example + proposed prototype. ตัวเลขตัวอย่างช่วยให้เห็น field relationships; ไม่ใช่ข้อมูลผู้ใช้จริง. กล่องพื้นเทา/ลายเฉียงคือ readonly; ขอบน้ำเงินคือ editable; * คือ required. LoV เปิด Course Offering List เพื่อเลือก offering ID แล้ว populate field ที่เกี่ยวข้อง.

ชิ้นงาน | Course Offering List View

Current semester: 2025-1 (fixed) Filter: Course code/name [Search]

ID / Code	Course / section	Credits	Fee	Max/Reg	Left
113 / CPE334	Software Engineering / 1	3	8,000	40/37	3
119 / MTH233	Calculus II / 7	4	6,000	40/28	12

Detail columns: Semester, Room, Class time, Display notes (instructor/remark). [Add selected offering] [Close]. List ต้องมี course ID/code/name, credits, fee, semester, section, room, time, notes, max students, registered, seats remaining ครบตาม FR7; ตารางนี้ย่อบาง columns ลง detail row เพื่ออ่านในหน้าเดียว.

Student Registration Form **[Close] [Save Draft] [Clear]**

Registration No.: generated after save	*Date: [15/09/2025]
Username: from authenticated session	Student ID / Name: KU Student Info
Program / Email: KU Student Info	Semester: current active semester
Total credits / fee: calculated	Status: Draft [Submit]
Paid on: auto after successful payment	Paid by: selected payment outcome
*Course Offering ID [LoV]: 113	Code/name/section/credits/fee/seats

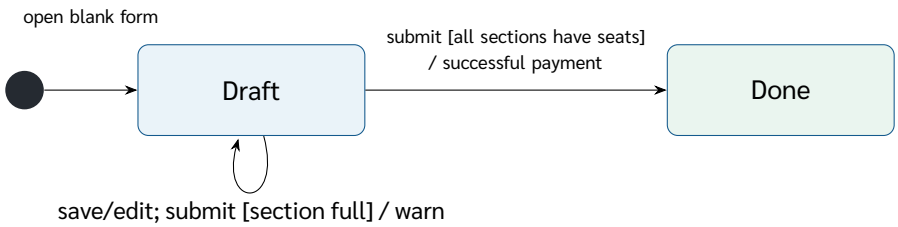
Line grid: add/remove rows; totals recalculate; full-section warning beside affected row

ชิ้นงาน | Field/action contract ที่ตรวจคู่กับภาพ

Field/action	Source / edit ability	Validation / state
Date / Offering ID	editable date and LoV per sample	required; offering must exist in active semester
Student / semester	session + external data / readonly	backend derives identity; ignore forged input
Totals / payment / status	calculated or transaction outcome / readonly	never editable to bypass payment
Save / Clear / Submit	student actions while Draft	verify ownership/state; Done disallows edit
Seats remaining	current offering data / readonly	display is advisory; recheck all at submit

Source caveat: sample UI has a heading about entering scores/grades despite being registration; correct to course selection. Source solution numbering swaps UI/activity labels; follow question text, not mislabeled solution captions.

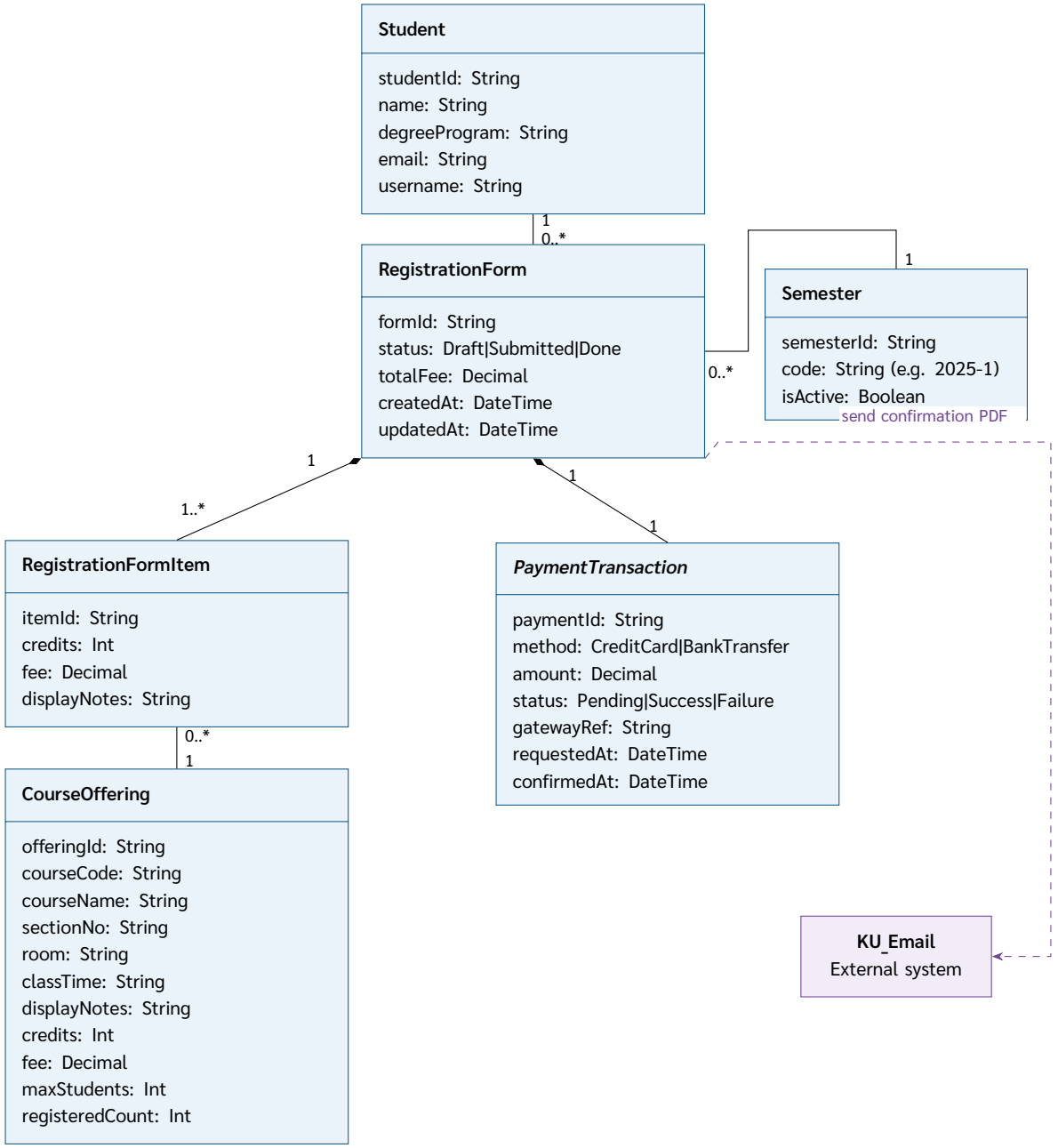
10.2.4 State และ Class: ไม่เติม lifecycle ที่โจทย์ไม่ให้



Why: Draft/Done เป็น status facts ของ FR8/11. “Awaiting Payment” อาจมีประโยชน์ต่อ design แต่ต้องติด assumption; อย่าอ้างว่า source ระบุ. Done ไม่มี edit transition ตาม FR11. State view นี้ต่างจาก Service Ticket ที่มีผู้ปิดสองแบบ และต่างจาก Grades Submission ที่มี Ready.

KU Class: ขยายภาพเพื่ออ่าน Attributes และ Multiplicity

Applied Example redraw: วาดใหม่จาก UML Example Exam Problem หน้า PDF 9 คง Attributes และความสัมพันธ์ของภาพต้นฉบับ ไม่แสดง Methods



Class scope ตามโจทย์: Header/Line Item และ links หนึ่ง hop; attributes/external calls/association/multiplicity/inheritance/composition; **ไม่ใส่ methods** และไม่แตก second-level links. ภาพต้นฉบับใส่ Submitted ใน Header แต่ FR หลักให้ Draft/Done จึงไม่ถือ Submitted เป็นสถานะที่โจทย์ยืนยัน ภาพเป็นตัวอย่างโครงสร้าง ต้องตรวจ Field Inventory ของ FR เพิ่ม เช่น Total Credits และกฎของ Form ภาพต้นฉบับใช้ 1..* line items และ 1:1 payment; draft เริ่ม blank และยังไม่จ่าย จึงต้องระบุว่าภาพย่อมอง populated form หรือเสนอ all-lifecycle model เป็น 0..* lines และ 0..1 completed payment ภายใต้ assumption. ไม่แก้ multiplicity โดยไม่อธิบาย scope.

ตัวอย่างพร้อมวิธีคิด | Mini problem + answer

นักศึกษาดู seats remaining=1 แล้วมีคนอื่นจ่ายก่อน เขาควร submit ผ่านจากค่าบนจอหรือไม่? **ตอบ:** ไม่ใช้ displayed count เป็น acceptance evidence; backend ต้อง recheck. ถ้าเต็ม แจ้ง row ที่ผิดและให้เปลี่ยน section/resubmit. การ lock/reserve seat เป็น design gap ที่ต้องตัดสินเพิ่ม ไม่เดาว่ามีอยู่แล้ว.

10.3 Case C - DormFix: ใช้หลักฐานเติม RACI

โครง WBS และ Gantt ของ DormFix อธิบายและวาดไว้ครั้งเดียวในบท 6 และบท 9 ตามลำดับ. ส่วนนี้ต่อจากโครงนั้นเพื่อฝึกตัดสินใจว่าอะไรตอบได้จาก meeting notes และอะไรต้องเว้นเป็น unknown.

10.3.1 RACI: เติมจาก evidence ไม่ใช่ความรู้สึก

Lecture 10 ระบุให้ใช้ meeting notes เป็นข้อมูล role แต่ notes สำหรับ DormFix ไม่ได้แจกแจง involvement ราย task ในข้อความที่ใช้ทำแบบฝึกนี้ จึงไม่สามารถอ้าง matrix ที่มีตัวอักษรครบทุกช่องได้อย่างมีหลักฐาน.

ชิ้นงาน | RACI decision sheet - ตัวอย่างวิธีกรอก

Task	คน/role 1	คน/role 2	Evidence from notes	Status
Write requirements	?	?	ยังไม่มี assignment ใน source ที่ใช้อยู่	Unresolved
Design screens	?	?	ต้องอ่าน meeting note	Unresolved
Test system	?	?	ต้องอ่าน meeting note	Unresolved

Rule เมื่อมี notes: R ทำงาน; A ทำให้แน่ใจว่างานสำเร็จ; C ให้ feedback ก่อนจบ; I รับทราบหลังจบ. ตรวจว่าแต่ละ task มี accountability ตามโจทย์ และอย่าอนุมานจาก job title อย่างเดียว.

10.3.2 Cross-reference: timeline and estimates

ใช้ผลคำนวณและ Gantt ชุดเดียวกับบท 6; ไม่คัดลอกตารางวันที่หรือสูตร PERT/ROI มาซ้ำ. กลับไปดูบทนั้นเมื่อจำเป็นต้องตรวจ dependency, uncertainty หรือเหตุผลของ buffer.

10.4 Case D - Court Booking และ ShuttleNow: backlog / Sprint Planning

10.4.1 Raw issue → story ที่พร้อม refinement

ชิ้นงาน | Backlog item: court-booking pattern จาก Lecture 10

User Story: As a student, I need to book a badminton court for one hour so that I do not have to queue at the counter.
Assumptions and details: Courts and opening hours already exist; one booking per day.
Acceptance Criterion: Given Court 2 is free at 17:00; When I book it; Then Court 2 at 17:00 is shown as “Booked”.
Metadata: Label = enhancement; ช่องที่ไม่มีค่าระบุ “-”.
Readiness check: story มีผู้ใช้/goal/value; assumption แยกชัด; AC ระบุ state observable. ยังต้องถามเรื่อง timezone, cancellation และ concurrent booking หากโจทย์ต้องการพฤติกรรมเหล่านั้น.

10.4.2 Backlog refinement ไม่ใช่การเขียน story ให้ยาว

Backlog Refinement ทำงานกับ order และความพร้อม: rank item ตาม product goal, split item ที่ใหญ่, clarify behavior/assumption, เติม acceptance criteria และทำให้ story sprint-ready. Template 3 blocks เป็น Story + Assumptions/Details + Acceptance Criteria. เมื่อ story ยังมีคำสำคัญกำกวม อย่าผลักความเสี่ยงให้ทีม sprint โดยไม่บันทึก.

10.4.3 ShuttleNow - เลือกด้วย Goal และ Capacity พร้อมกัน

Sprint goal คือให้นักศึกษาติดตาม shuttle และรู้เวลาถึง; velocity = 16; story ขนาด 3 points ใช้เป็น anchor เปรียบเทียบ. Worked selection จากโจทย์:

Story	Points	Sprint 6?	Decision rationale
Live position	8	Yes	สนับสนุน goal การติดตามรถโดยตรง
QR fare	3	No	points พอใส่ได้ แต่ไม่สนับสนุน sprint goal นี้
Arrival at my stop	5	Yes	ช่วยตอบเวลาถึงสำหรับผู้ใช้
Alert before arrival	3	Yes	ทำให้ผู้ใช้รู้ก่อนถึง
Arrival all stops	5	No	เพิ่มแล้วเกิน capacity ของชุดที่เลือก
Mobile route-list bug	2	No	ไม่ถูกเลือกสำหรับ goal นี้

เลือก $8 + 5 + 3 = 16$ points. จุดหยุดคือเมื่อทั้ง goal coverage และ capacity ถูกตอบ ไม่ใช่รับทุก story ที่ “พอใส่ได้”. Story QR fare ถูกปฏิเสธด้วยเหตุผล goal แม้ขนาด 3 points จะพอดีกับพื้นที่ที่เหลือ.

10.5 Case E - Git transactions

ดูบท Git Worked Transactions สำหรับกราฟ Before/After พร้อม commands, parents, HEAD และ remote-tracking refs.

10.6 Cross-artifact consistency audit

ก่อนส่งงาน ให้เดินตาม row ของ requirement ตัวเดียวแล้วตรวจว่าแต่ละ artifact พูดเรื่องเดียวกัน:

1. Need/Policy ระบุเจ้าของและขอบเขตเดียวกับ requirement.
2. Requirement สอดคล้องกับ acceptance criterion; threshold ที่ unresolved ยังไม่กลายเป็นตัวเลขใน test.
3. UML ใช้ actor, message, state และ decision ที่ไม่ขัดกับ requirement.
4. UI แสดงค่าจาก source ที่ถูก; read-only field ไม่มี affordance ให้แก้.
5. Backlog/WBS แตกงานจาก scope จริง; dependency สะท้อน prerequisite.
6. Security rule ถูก enforce ที่ boundary ที่เชื่อถือได้; negative test ครอบคลุม denied case.
7. Test/evidence ปิด traceability chain; pending/blocked ไม่ถูกระบุว่า pass.

ข้อผิดพลาดที่พบบ่อย | ความครบถ้วนปลอม

ตารางที่ทุกช่องเต็ม, diagram ที่มีทุก symbol, Gantt ที่ไม่มีช่องว่าง หรือ test status ที่เป็น green ทั้งหมดอาจดูเรียบร้อย แต่ไม่ได้แปลว่ามีข้อมูลจริง. ความซื่อสัตย์ต่อ source รวมถึงการแสดง unknown, assumption, blocked และ pending.

บทที่ 11

Lecture 10: Midterm Exam Blueprint

Lecture 10 ใช้บอกชนิดโจทย์, role, artifact และคะแนนที่ระบุ ไม่แทนเนื้อหาทั้งหมดใน Lecture 1–7 และไม่ยืนยันว่า sample ใดจะออกจริง

Exam Blueprint

Part	Points	Role	Output / source backbone
1 Requirements + Security	25	Requirement/ Security analyst	stakeholder requirements, lightweight proposal, RBAC, security landscape/ vulnerability; L2 + L5
2 Planning + Execution	25	PM / Product Owner / Scrum Master ตามโจทย์	WBS 3, RACI 2, Gantt 5, backlog grooming 10, sprint planning 5; L6 + L7
3 UML + Git	50	Software designer/ modeler + Git user	30 points เลือกจาก UML/UI 5 ชนิด; Git workflow 20 points เลือก task 3 แบบ; L1 + L4 และ Lecture 10

Instruction ที่กำกับคำตอบ

Part 1 ระบุ “Use your own notes. Explain your decisions and state any assumptions.” ดังนั้นคำตอบที่ถูกแต่ไม่มี rationale/assumption ยังไม่ครบ. Part 2 ระบุบทบาทแยก: PM ทำ WBS/RACI/Gantt; PO ทำ backlog grooming; Scrum Master ทำ sprint planning.

11.1 Part 3: UML/UI + Git ที่ระบุคะแนน

Skill	Points	Skill output
Sequence Diagram with swimlanes for scenario	10	participants, messages, order, response/failure path
Activity Diagram with partitions for workflow	10	actions, decision/loop, partitions
UI prototype of list + form view	10	field/source/editable/required/action/state
Use Case Diagram	10	actor, boundary, use case, relation semantics
State Diagram	10	states, triggers/transitions, guard/actor
Git: predict repository graph	10	graph after transactions
Git: explain given repository graph	10	interpret branch/HEAD/commit state
Git: commands + redraw graph	10	commands and pointer changes

UML/UI table states “30 points selected from below”; Git states 20 points selected from below. Class diagram appears in lecture/application material but not in this exact 30-point selection table: tag as lecture-covered/ potential skill, not guaranteed points.

11.2 Exam answer discipline

1. Read role and requested artifact; underline action verbs (classify, revise, draw, predict, explain).
2. Extract facts, source constraints, assumptions, and missing information.
3. Choose representation that matches question; make artifact legible.
4. Explain decision with a specific “because” linked to source/goal.
5. Verify semantic consistency and state evidence/unknowns.

บทที่ 12

วิธีทำข้อสอบ: ตอบให้เห็นเหตุผลและหลักฐาน

บทนี้เป็น checklist ไม่ทำซ้ำโจทย์เต็มและเฉลย ซึ่งอยู่ใน Mock Exam แยกเล่ม

12.1 Part 1: จากคำขอสู่ข้อกำหนดที่ตรวจได้

คำถาม	วิธีตรวจ
ชนิดของข้อความ	FR = พฤติกรรมระบบ; NFR = คุณภาพที่มีเงื่อนไขวัด; business rule = ข้อจำกัดของโดเมน; stakeholder goal = ผลที่ต้องการ
ความชัดเจน	ระบุผู้กระทำ เงื่อนไข ข้อมูล/ผลลัพธ์ และขอบเขต; หาคำกำกวม เช่น quick, easy, all
Atomicity	ถ้าคนละ actor หรือแยกตรวจรับได้ ให้แยก requirement และ acceptance criteria
Verification	เลือก test/inspection/analysis ให้เหมาะ ระบุ input, expected result และหลักฐาน
ข้อเสนอ	แยก facts ที่โจทย์ให้จาก recommendation; scope, risk, effort และ evidence ต้องแสดงช่องว่างหากยังไม่มี

ตัวอย่างพร้อมวิธีคิด | โครงคำตอบ NFR ที่ยังไม่มี threshold

ภายใต้ workload ที่ตกลงกัน 95% ของคำขอค้นหาการสมัครต้องเสร็จในไม่เกิน [threshold] วินาที. ก่อน baseline ต้องตกลง percentile, concurrent load, start/end point และ threshold; ห้ามแต่งค่าตัวเลขขึ้นเอง.

ข้อเสนอพร้อมตัดสินใจควรเชื่อมปัญหา → ผู้ใช้ → ทางเลือก → คุณค่า → ขอบเขต → ความเสี่ยง/effort → วิธีพิสูจน์ผล. วันส่งงานเป็น deadline ไม่ใช่ effort estimate.

12.2 Part 1: Security ตอบจากเส้นทางโจมตี

1. ระบุ actor, asset, entry point และสิ่งที่ attacker ควบคุมได้.
2. ทำเครื่องหมาย trust boundary: client → server, service → storage, user → protected object.
3. บอกผลกระทบที่โจทย์ยืนยัน แยกจากผลที่เป็นเพียงความเสี่ยง.
4. เสนอมาตรการที่หยุดกลไกโจมตี แล้วจับคู่กับ test และ expected result.

Threat	ตอบให้ตรงกับกลไก
สิทธิ์เกิน	ตรวจ authenticated identity + role/owner/resource/action ที่ server; ทดสอบ direct API และเปลี่ยน object ID
ไฟล์อันตราย	ตรวจเนื้อหาที่ server, เก็บนอก executable web root, download ผ่าน authorization endpoint; ทดลอง bypass client check
SQL injection	parameterized query; ทดสอบ input ที่เป็น syntax และยืนยันว่าเป็น data ไม่ใช่ query
XSS	context-aware output encoding; ทดสอบ input script ในทุกหน้าที่ render
CSRF / SSRF	ตรวจกลไก request origin/token หรือ allowlist/egress ตามชนิดความเสี่ยง และทดสอบปลายทางที่ห้าม

ข้อผิดพลาดที่พบบ่อย | คำตอบ security ที่ยังไม่พอ

“ใช้ HTTPS” ไม่แก้ authorization ผิด, upload ที่ execute ได้ หรือ query ที่ต่อ string. “ซ่อนปุ่ม” ไม่บังคับ policy. ระบุ boundary และ test ที่พิสูจน์ผลให้ชัด.

12.3 Part 2: วางแผนให้ทุก artifact สอดคล้องกัน

WBS: ตั้งชื่อ milestone เป็นผลที่ตรวจรับได้ → จัด task ใต้ milestone → ใช้ทุก task ครั้งเดียว → ตรวจ parent/child. WBS บอก hierarchy ไม่ได้บอก schedule.

RACI: ใช้ meeting note/role evidence เท่านั้น. R ลงมือ, A accountable ต่อผล, C ให้ข้อมูลสองทาง, I รับทราบ. ถ้าไม่มีหลักฐานให้ใส่ unresolved แทนเดา.

Gantt: ทำ dependency graph ก่อน → คำนวณวันทำงานและ parallel tasks → วาง external/task buffer ตามเงื่อนไข → เพิ่ม project buffer → ใส่ milestone → ตรวจวันหยุดและ predecessor.

Backlog: แปลง issue เป็น Role–Capability–Benefit, เพิ่ม Details/Assumptions และ Given–When–Then ที่สังเกตผลได้; เติม Label, Estimate, Milestone, Assignee เฉพาะที่โจทย์ให้หรือสมเหตุผลพร้อมระบุข้อเสนอ.

Sprint: เริ่มจาก Sprint Goal และ Definition of Done → เลือกงานที่จำเป็นต่อ goal → ตรวจ capacity/velocity และ dependency → อธิบายสิ่งที่ไม่เลือก.

จุดเช็กความสอดคล้อง

ใช้ WBS ID เดียวใน RACI/Gantt; task ที่ยังไม่อนุมัติต้องไม่เริ่มก่อน approval; estimate เป็นหน่วยเดียวกัน; milestone เป็นจุดเหตุการณ์ ไม่ใช่งานมี duration; Sprint points ไม่ใช่ชั่วโมง.

12.4 Part 3: เลือกภาพ/เครื่องมือให้ตรงคำถาม

ถ้าโจทย์ถาม	ใช้	ตรวจจุดสำคัญ
ใครเรียกใคร เมื่อไร	Sequence	participant, ข้อความมีทิศทาง, ลำดับเวลา, return/branch
workflow และทางเลือก	Activity	action, swimlane, decision guard, start/end, ทุก branch
ข้อมูลและความสัมพันธ์	Class	class/entity, attribute/type, association, multiplicity/ownership
เป้าหมายของ actor	Use Case	actor นอก boundary, system goal, association และ include/extend ที่มีความหมาย
วงจรชีวิต object	State	state, event [guard] / effect, initial/final, valid/invalid transitions
หน้าจอและการกรอกข้อมูล	UI List/Form	label, grouping, editable/readonly, default, action, validation/error

ก่อนส่ง diagram: อ่านโจทย์ซ้ำ, ตรวจชื่อ actor/field/status ให้ตรง, trace ลูกศรทุกเส้น, ใส่ guard ให้ decision, เว้นช่องรอบ label และไม่ให้เส้นตัดกล่อง. แนวคิด Git ใช้ลำดับ Before → command → After → Why และวาด parent/ref/HEAD ให้ครบ (ดูบัตร Git ต่อจากนี้).

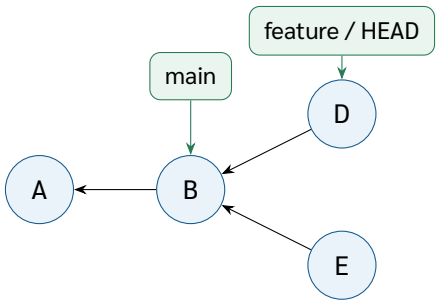
บทที่ 13

Git Exam Reference: คิดเป็น transaction

ใช้บท Git พื้นฐานเพื่อเข้าใจคำสั่ง; หน้านี้รวบวิธีวาดกราฟให้ตรวจคำตอบได้

13.1 แยก object ออกจาก pointer

สิ่งที่วาด	ความหมาย
Commit	snapshot + parent link; commit ใหม่ไม่เปลี่ยนเนื้อหา node เก่า
Branch ref	ชื่อที่ชี้ commit หนึ่งและขยับเมื่อ commit/merge บน branch นั้น
HEAD	บอก context ที่ checkout; attached HEAD ตาม branch ปัจจุบัน
Index	staged snapshot สำหรับ commit ถัดไป; ไม่ใช่ commit
Remote-tracking ref	เช่น origin/main คือสถานะ remote ล่าสุดที่ local รู้ ไม่ใช่ remote ที่เปลี่ยนสด



main และ feature แยกจาก common parent B. ถ้า HEAD อยู่ feature แล้ว commit ใหม่ F จะมี parent D; feature/HEAD ขยับ F ส่วน main ยัง B. กราฟไม่บอก working tree, staged files, conflict หรือสถานะ server.

13.2 วาดผลคำสั่ง: Before → command → After → Why

คำสั่ง	เปลี่ยนอะไร	ไม่ได้เปลี่ยนอะไรโดยตรง
git add	index รับ snapshot ที่เลือก	ไม่สร้าง commit/ref
git commit	สร้าง node จาก index; ขยับ current branch	ไม่ขยับ branch อื่นหรือ remote
git branch name	สร้าง ref ที่ commit ปัจจุบัน	ไม่ย้าย HEAD และไม่สร้าง node
git switch name	checkout context; HEAD ไปตาม branch	ไม่ขยับ ref ของ branch นั้น
git fetch	รับ object และอัปเดต remote-tracking ref	ไม่ merge เข้า local branch โดยอัตโนมัติ

13.3 Merge: เลือกผลจาก ancestry

- **Fast-forward:** target เป็น ancestor ของ source; target ref ขยับไป source tip, node ใหม่ไม่เกิด.
- **Three-way:** branches diverged; สร้าง merge commit ที่มี parent สองตัวเมื่อ merge สำเร็จ.
- **Conflict:** แก้ working tree, stage resolution, commit ตาม merge state; อย่าวาดผลสำเร็จก่อน resolve.

ตัวอย่างพร้อมวิธีคิด | ตัวอย่างคำตอบสั้นที่ตรวจย้อนกลับได้

Before: HEAD attached to staging at B; feature at D; B เป็น ancestor ของ D. Command: switch staging, merge feature. After: fast-forward staging/HEAD ไป D; feature ยัง D; ไม่มี merge commit. Why: branch เป้าหมายยังไม่มี commit ที่แยกจาก D.

เช็กก่อนส่ง: ระบุ checked-out branch, parent ของ node ใหม่, ref ที่ขยับ, ref ที่ไม่ขยับ และ remote ที่ไม่ได้เปลี่ยน. ถ้าโจทย์ไม่กำหนด fast-forward policy ให้บอกสมมติฐาน.

บทที่ 14

Git Worked Transactions: ExamScore repository

Exam Blueprint | L10 หน้า 32; Explanation / Intuition for worked graphs. Context จากโจทย์คือ Python CLI โหลดชื่อ/คะแนนจาก CSV, validate 0–100, คำนวณ statistics และ pass report. main คือ release; sprint_1_staging คือ integration. ไฟล์ตัวอย่าง score_data.py, scores.csv, validation_notes.txt. Commit labels A–F ต่อไปนี้สร้างเพื่อสอน pointer effects ไม่ใช่ commit IDs ที่ source แจก.

14.1 ก่อนตอบ: แยก graph, pointer, index และ working tree

ชิ้นงาน | สิ่งที่ command เปลี่ยนได้

Element	Meaning / evidence
Commit	snapshot พร้อม parent(s); commits เดิมไม่ย้ายเมื่อ switch branch
Local branch	reference เคลื่อนที่ เช่น main; commit ใหม่ขยับเฉพาะ branch ปัจจุบัน
HEAD	current checkout; ในโจทย์ปกติชี้ branch ไม่ใช่ชื่อ branch อิสระ
origin/main	local remote-tracking reference: ภาพล่าสุดที่รู้เกี่ยวกับ remote; ไม่ใช่ live query
Working tree / index	file edits / staged next snapshot; add ยังไม่สร้าง commit
Remote repository	มี refs/objects ของตนเอง; ต้องมี remote command จึงสื่อสาร

Notation ในบทนี้: ลูกศร commit ชี้ **จาก child ไป parent**; เวลาโดยทั่วไปเก่า → ใหม่ซ้าย → ขวา. กล่องสีเขียวคือ branch refs; HEAD → branch แสดง checkout. หลายกล่องเหนือ commit เดียวหมายถึงหลาย refs ที่ชี้ object เดียวกัน ไม่ใช่หลาย copies ของ commit. **Assumption / unresolved:** working tree clean เว้นโจทย์บอกว่าแก้ไฟล์; commands สำเร็จ; ไม่มี conflicts; merge style ปกติที่ยอม fast-forward. ถ้าโจทย์กำหนด –no-ff ผลต้องเปลี่ยน. ใช้ explicit filenames ใน git add ตาม exam instructions

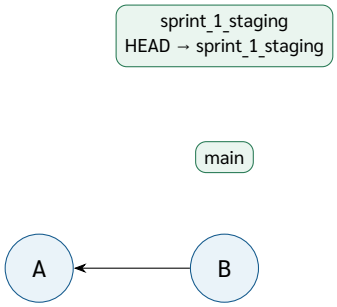
ตัวอย่างพร้อมวิธีคิด | ตอบหนึ่ง transaction ให้ครบ

1 อ่าน Before ว่า HEAD อยู่ไหนและ refs ชี้อะไร. 2 แยก file changes/add/commit. 3 ทำ command ตามลำดับ. 4 นับ nodes/parents ที่เพิ่ม. 5 ขยับ refs ที่เกี่ยวข้องเท่านั้น. 6 วาด After และ Why; ไม่พูดว่า GitHub เปลี่ยนถ้าไม่มี push/remote operation.

Source: L1 หน้า 29–33; L10 หน้า 32

14.2 1. Edit → stage → commit: branch ปัจจุบันขยับ

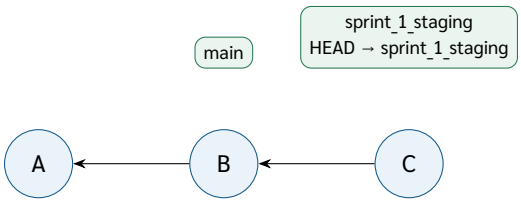
Input / Before: Before: HEAD อยู่ sprint staging ที่ B; main อยู่ B เช่นกัน. แก้ validation function กับ notes แต่ scores.csv ไม่ได้แก้. git add เลือกสองไฟล์นี้เข้า index; git commit สร้าง C จาก staged snapshot.



ชิ้นงาน | Command / transaction

```
git add score_data.py validation_notes.txt
git commit -m "Validate score range"
```

After:



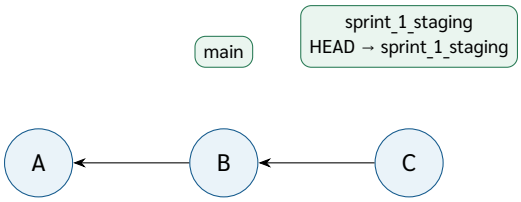
เหตุผล: C มี parent B. sprint staging ย้าย B → C; HEAD ตาม branch. main อยู่ B; remote ยังไม่เปลี่ยน. ถ้า add ไม่ครบ commit จะไม่มีการเปลี่ยนแปลงที่ยัง unstaged.

ข้อผิดพลาดที่พบบ่อย | Wrong answer → repair

เพิ่ม node ตอน git add; ขยับ main ทั้งที่ไม่ได้ checkout; หรือใช้ git add . ซึ่งขัดคำสั่งโจทย์. ตรวจ status/diff staged และ parent ของ C.

14.3 2. สร้าง branch โดยยังไม่ commit

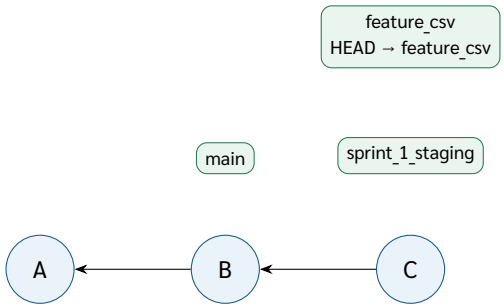
Input / Before: ต้องแยกงาน CSV ออกจาก integration branch. ก่อน command C คือ current commit; ยังไม่ได้แก้ไฟล์.



ชิ้นงาน | Command / transaction

```
git switch -c feature_csv
```

After:



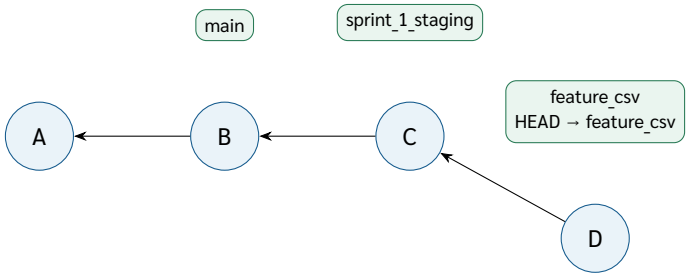
เหตุผล: สร้าง feature_csv reference ที่ C และ HEAD ชี้ branch ใหม่นี้. จำนวน commit และ parent links เท่าเดิม; staging ยัง C. git branch feature_csv อย่างเดียวจะสร้าง ref แต่ไม่ switch HEAD.

ข้อผิดพลาดที่พบบ่อย | Wrong answer → repair

วาด commit ใหม่เพียงเพราะชื่อ branch ใหม่. Branch คือ pointer ราคาถูก; commit เกิดจาก commit operation เท่านั้น.

14.4 3. Switch หลัง feature commit: refs คงที่

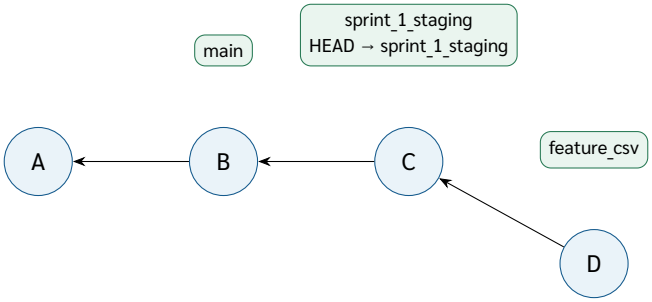
Input / Before: Feature เพิ่ม D มี parent C แล้วต้องกลับ integration branch. Working tree clean ตาม assumption.



ชิ้นงาน | Command / transaction

git switch sprint_1_staging

After:



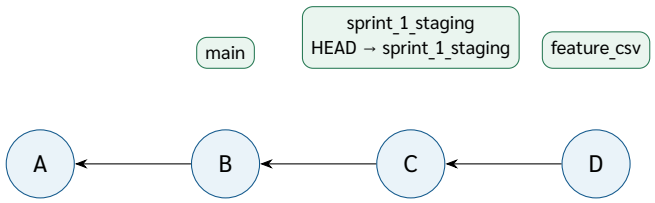
เหตุผล: HEAD เปลี่ยน feature_csv → staging; branch refs และ graph ไม่เปลี่ยน. Working tree/index แสดง snapshot C เมื่อ switch สำเร็จ. D ยังเข้าถึงผ่าน feature_csv.

ข้อผิดพลาดที่พบบ่อย | Wrong answer → repair

ย้าย staging ไป D เมื่อ switch; ลบ D; หรือคิดว่า switch ส่งงานไป GitHub. ถ้าโจทย์ให้ uncommitted edits ต้องพิจารณา checkout constraints แทน assumption นี้.

14.5 4. Fast-forward merge: ไม่มี merge commit

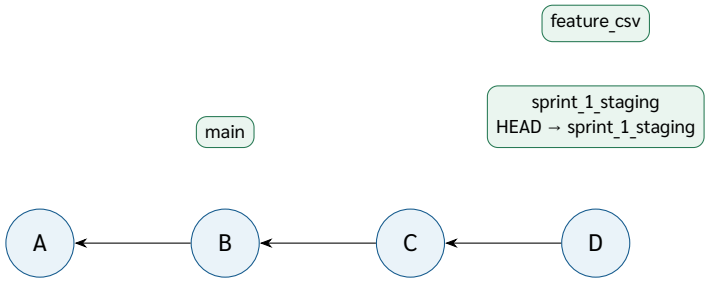
Input / Before: Current staging C เป็น ancestor ของ feature D; ไม่มี commit ใหม่บน staging หลังแตก branch.



ชิ้นงาน | Command / transaction

git merge feature_csv

After:



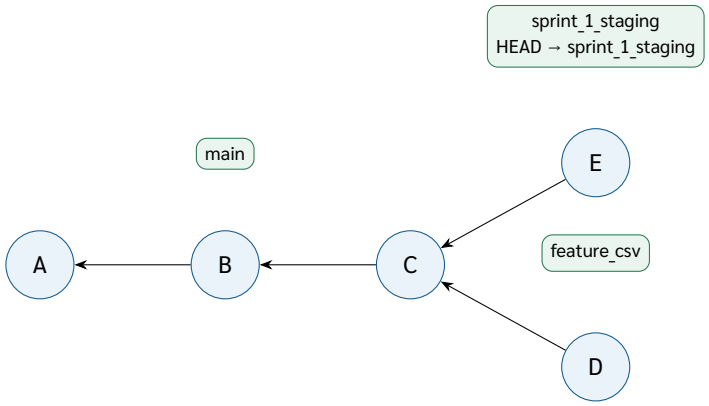
เหตุผล: ภายใต้ default fast-forward policy staging เคลื่อน C → D; ไม่มี node เพิ่ม. feature ยัง D; HEAD ยัง staging; main ยัง B. Merge target คือ branch ที่ checkout อยู่.

ข้อผิดพลาดที่พบบ่อย | Wrong answer → repair

สร้าง two-parent merge node ทั้งที่โจทย์ไม่ได้บังคับ –no-ff; หรือย้าย feature แทน target. ถ้า command ใช้ –no-ff ต้องเพิ่ม merge commit แม้ graph fast-forward ได้.

14.6 5. Diverged merge: commit ใหม่มีสอง parents

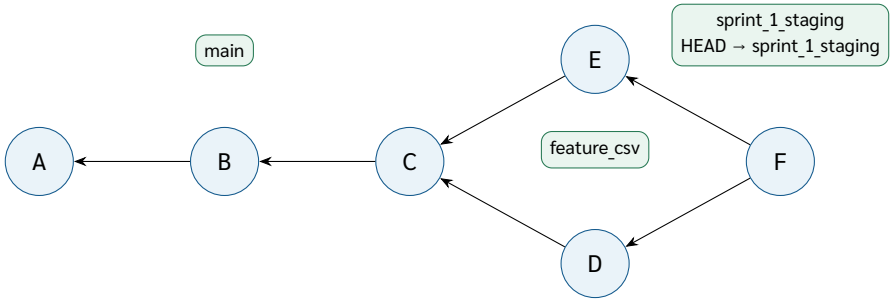
Input / Before: C เป็น common ancestor; staging มี E และ feature มี D. ทั้งสอง branch มีงานหลังจุดแตก; neither tip is ancestor of the other.



ชิ้นงาน | Command / transaction

git merge feature_csv

After:



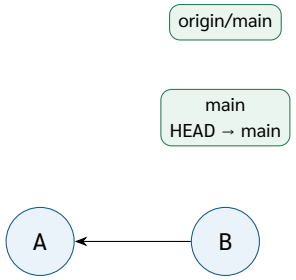
เหตุผล: Merge สำเร็จแบบไม่มี conflict สร้าง F: first parent E (current target), second parent D (merged branch). staging → F; feature ยัง D; commits D/E ไม่ถูกลบ. main ยัง B. Snapshot F รวมงานที่ merge แล้ว ไม่ใช่ snapshot ของ D อย่างเดียว.

ข้อผิดพลาดที่พบบ่อย | Wrong answer → repair

ลาก D เป็น parent ของ E เพื่อบังคับเส้นตรง; claim feature ref เคลื่อน; หรือวาด F มี parent เดียว. ถ้ามี conflict ต้อง resolve/add/commit ก่อน F จะเกิด; source assumption นี้ตัด conflict ออก.

14.7 6. Fetch → integrate → push: local และ remote คนละชุด

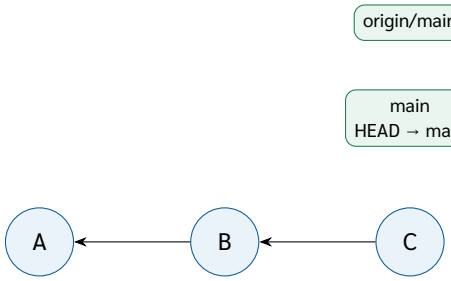
Input / Before: สมมติ remote main ได้ C (parent B) จากเพื่อน แต่ local ยังรู้ B. Before graph ของ local จึงยังไม่มี C. Remote ก่อน fetch: main=C; local main=B, origin/main=B.



ชิ้นงาน | Command / transaction

```
git fetch origin
git merge origin/main
git push origin main
```

After:



เหตุผล: fetch ดาวน์โหลด C และ update origin/main → C; local main/HEAD ยัง B. merge origin/main fast-forward local main → C; HEAD ยัง main. push ส่ง local main ไป remote; ในตัวอย่างนี้ remote C อยู่แล้วจึงไม่เพิ่ม commit. origin/main ไม่ใช่ branch ที่ควรพัฒนาโดยตรง.

ข้อผิดพลาดที่พบบ่อย | Wrong answer → repair

บอก fetch ขยับ local main หรือ working tree; บอก remote ต้องมีค่าเท่าที่ local diagram แสดงเสมอ. ถ้า remote advance ระหว่าง fetch/push ผลอาจ rejected; โจทย์นี้สมมติ remote ไม่เปลี่ยนระหว่าง commands.

14.8 Git revision: ใช้ graph ตอบโจทย์ใหม่

ชิ้นงาน | Command-to-effect comparison

Operation	Nodes / parents	Pointer / remote effect
add explicit files	no commit yet	index only
commit	new node, parent=current tip	current branch moves; HEAD follows
branch / switch -c	no new node	add ref; -c also switches HEAD
switch existing	no graph change	HEAD checkout changes
fast-forward merge	no node	target ref advances to descendant
diverged merge	new two-parent node	target ref moves; source ref stays
fetch	brings existing remote objects	tracking refs update; local branch stays
push	sends objects/updates remote refs	no new local commit merely from push

ตัวอย่างพร้อมวิธีคิด | Mini problem + reasoned answer

Before: A←B←C; main=B, staging=C, HEAD → main. ทำ switch staging แล้ว branch feature โดยไม่มี -c. **ตอบ:** HEAD → staging=C; main=B; feature=C; nodes A/B/C เดิมทั้งหมด. จากนั้นแก้ scores.csv, add scores.csv, commit D: staging=D, feature=C, main=B; D parent C. Feature ไม่ตาม staging อัตโนมัติแม้เริ่มที่ commit เดียวกัน.

Key decision: ถ้า command list ไม่บอก push อย่าวาด remote เปลี่ยน. ถ้า graph ไม่ระบุ current branch ต้องใส่ assumption ก่อน command เพราะ merge/commit target อาจเปลี่ยนผล. อ่าน pointer labels ก่อนดูรูปทรงเส้น; สาขาที่วาดสูงกว่าไม่ใช่ branch ที่สำคัญกว่า. **Exam mapping:** L10 ระบุ Git 20 คะแนน เลือก 3 task types ที่คิด 10 คะแนนต่อข้อ; สรุปตาม blueprint โดยไม่ตีความเองว่าได้ 30 คะแนน. Worked transactions ในบทนี้ใช้ฝึก semantics; ไม่รับประกันว่าข้อสอบเลือก commands เหล่านี้ทุกข้อ.

บทที่ 15

ทบทวนก่อนสอบ: แผนที่ 4 หน้า

ใช้เรียกคืนความเข้าใจ ไม่ใช่อ่านซ้ำแทนบทเต็ม

15.1 แผนที่เนื้อหาและ artifact

Lecture	คำถามหลัก	ชิ้นงานที่ต้องทำได้
1	ทำไม process และ version history สำคัญ?	เลือก SDLC/process, review, branch/commit graph
2	ใครต้องการแก้ปัญหาอะไร?	stakeholder, FR/NFR/rule, acceptance, proposal gap
3	จะระบุและพิสูจน์ requirement อย่างไร?	SDS/STS, test case, TDD, traceability
4	จะสื่อสาร behavior/data/UI แบบไหน?	Sequence, Activity, Class, State, Use Case, UI
5	ใคร/อะไรเชื่อถือได้และมีสิทธิ์?	authn/authz, threat boundary, controls + negative tests
6	ขอบเขต เวลาและ owner สมเหตุผลไหม?	charter, WBS tree, RACI, dependency, Gantt/PERT
7	จะ ส่ง งาน ย่อย และ ปรับ จาก feedback อย่างไร?	backlog, sprint goal, capacity, flow, Git integration
10	รูปแบบใดถูกใช้วัดผลสอบ?	blueprint คะแนนและประเภทงานที่ต้องฝึก

15.2 สูตรและการแยกคำที่มักสับสน

ห้ามสับสน	จำความต่าง
Project life cycle / SDLC	การบริหารโครงการ / การสร้างและเปลี่ยน software
Effort / Duration	ปริมาณงาน / เวลาปฏิทินที่ผ่านไป
WBS / Gantt	hierarchy ของ scope / วันและ dependency
Goal / Deliverable / Milestone	ผลลัพธ์ / สิ่งส่งมอบ / จุดเหตุการณ์ตรวจรับ
Authentication / Authorization	พิสูจน์ตัวตน / อนุญาต action ต่อ resource
Class / State / Sequence	โครงสร้างข้อมูล / lifecycle / ลำดับข้อความ
Story points / hours	ขนาดสัมพัทธ์ / เวลาทำงาน
Fast-forward / merge commit	ขยับ ref ไป descendant / สร้าง node ที่มีสอง parents

$$ROI = (Gain - Cost)/Cost, \qquad E_{PERT} = (O + 4M + P)/6, \qquad SD = (P - O)/6$$

$$Float = LS - ES = LF - EF, \qquad \text{Sprint capacity ใช้หน่วยเดียวกับ estimate}$$

15.3 Part 1–2: จุดให้คะแนนและข้อผิดพลาด

งาน	ต้องมี	อย่าทำ
Requirement	type, purpose, quality reason, observable check/result	เติม threshold/role ที่ไม่มีในโจทย์
Proposal	evidence ของปัญหา/ผู้ใช้/value/scope/risk/effort/done	เรียก deadline ว่า effort; อ้างประโยชน์ที่ยังไม่วัดเป็น fact
Security	actor, asset, boundary, observed impact, control, test	บอกเพียง “ใช้ HTTPS” หรือซ่อนปุ่ม
WBS/RACI	milestone tree, ทุก task ครั้งเดียว, role ตาม evidence	เอา schedule ไปปน hierarchy; เดา owner
Gantt	workday, predecessor, parallel, buffer, milestone	นับ weekend เป็นวันทำงาน; milestone มี duration
Backlog/Sprint	role-value, AC สังเกตได้, goal, capacity และเหตุผลที่ไม่เลือก	points=hours; เลือกงานให้เต็มโดยไม่ดู goal

ก่อนส่ง

ขีดเส้นใต้เงื่อนไขในโจทย์ → สร้าง artifact → อธิบายเหตุผล → ใส่ assumption/unresolved → ตรวจ consistency และ expected evidence.

15.4 Part 3: ตรวจภาพและ Git

ตรวจ diagram	คำถามก่อนส่ง
Sequence	ลูกศรหันถูกทิศและเรียงตามเวลาไหม? ข้อความ response/guard ชัดไหม?
Activity	ทุก decision มี guard ครบไหม? branch จบหรือกลับเข้ากระบวนการอย่างถูกต้องไหม?
Use Case	actor อยู่นอกขอบเขตไหม? goal เป็นความสามารถระดับผู้ใช้ไหม?
State	state เป็นสภาพคงอยู่หรือเผลอใส่ event? มี valid/invalid transition ตามกติกาไหม?
Class	multiplicity/ownership ตรงกับ lifecycle หรือยังระบุ assumption?
UI	label, field state, read-only/editable, validation และ action ครบไหม?
Git	Before → command → After → Why; parent/ref/HEAD ระบุครบหรือไม่?

15.5 โครงคะแนน Lecture 10

Part	คะแนน	โครงงาน
Part 1	25	Requirement/proposal และ security/role policy
Part 2	25	WBS 3, RACI 2, Gantt 5; grooming 10; sprint planning 5
Part 3 UML/UI	เลือก 3 จาก 5 = 30	Sequence, Activity, UI List/Form, Use Case, State; งานละ 10
Part 3 Git	เลือก 2 จาก 3 = 20	predict, explain, commands/redraw; งานละ 10

ทำโจทย์ก่อนเปิดเฉลย; ถ้าคำตอบอาศัยข้อมูลที่ไม่ได้ให้ ให้เขียนเป็น assumption และบอกว่าจะยืนยันอย่างไร.

ภาคผนวก A

สูตรและแนวเขียนคำตอบแบบเร็ว

เรื่อง	สูตรหรือแนวเขียนคำตอบ
FR	ระบบต้องทำ Action กับ Object ภายใต้ Condition ที่ระบุ ผลต้องตรวจได้
NFR	Quality + Metric + Threshold + เงื่อนไขทำงาน; ค่าที่ยังไม่ยืนยันต้องแสดงไว้
Quality	Clear / Testable / Feasible / Relevant / Atomic / Traceable
AC	Given สภาพเริ่ม / When เหตุการณ์ / Then ผลที่สังเกตได้
Proposal	Problem / Users / Solution / Value / Scope / Risk / Effort / Evidence
Traceability	Need → Requirement → AC → Design → Task → Code → Test → Evidence
เลือก UML	Use Case ผู้ใช้/เป้าหมาย; Activity Workflow; Sequence Scenario; State Lifecycle; Class โครงสร้าง
Security	Authentication ยืนยันตัวตน; Authorization ตรวจสิทธิ์ต่อ Action/Object/Context
RACI	R ลงมือ; A รับผิดชอบผลสำเร็จ; C ให้ข้อมูลก่อนจบ; I รับแจ้ง
WBS / Gantt	WBS แตก Milestone→Task→Subtask; Gantt แสดงวัน Dependency, Buffer และจุด Milestone
CPM	EF=ES+Duration; ES=max(EF ของงานก่อน); LS=LF−Duration; Float=LS−ES
PERT	$E = (O + 4M + P)/6$, $SD = (P - O)/6$; รวม Variance เมื่อสมมติให้ค่าประมาณเป็นอิสระ
Scrum	Sprint Goal + Story ที่พร้อม + Capacity; Velocity เฉพาะทีม Points ไม่ใช่ชั่วโมง
Git	Before → อ่าน HEAD/Refs → Command → ผลต่อ Node/Parent/Pointer → After → เหตุผล

แหล่งที่มาและขอบเขตการอ้างอิง

ดูดัชนีต้นฉบับก่อนบท 1 และ source_page_inventory.csv สำหรับทุกหน้า; source_trace.md สำหรับ mapping ระหว่างหัวข้อกับแหล่งข้อมูล และ figure_manifest.md สำหรับภาพที่ redraw/สร้างใหม่และตำแหน่งในเล่ม. ไม่ใช้ external references เป็น authority ของเนื้อหาในฉบับนี้.